Next

The following classes contain this method:

- Node
- Segment
- SubPath

Copyright 2014 Corel Corporation. All rights reserved.

Object

The following classes contain this method:

- SnapPoints

Copyright 2014 Corel Corporation. All rights reserved.

Offset

The following classes contain this method:

- Rect

Copyright 2014 Corel Corporation. All rights reserved.

Open

The following classes contain this method:

- OLE
- Palette
- Palettes
- RecentFile

Copyright 2014 Corel Corporation. All rights reserved.

OpenCorelScriptFile

The following classes contain this method:

- Application

Copyright 2014 Corel Corporation. All rights reserved.

OpenDocument

The following classes contain this method:

- Application

Copyright 2014 Corel Corporation. All rights reserved.

OpenDocumentAsCopy

The following classes contain this method:

- Application

Copyright 2014 Corel Corporation. All rights reserved.

OpenDocumentEx

The following classes contain this method:

- Application

Copyright 2014 Corel Corporation. All rights reserved.

OrderBackOf

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

OrderBackOne

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

OrderForwardOne

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

OrderFrontOf

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

OrderIsInFrontOf

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

OrderReverse

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

OrderToBack

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

OrderToFront

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

PaletteAssign

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

Paste

The following classes contain this method:

- Layer
- SymbolLibrary
- TextRange

Copyright 2014 Corel Corporation. All rights reserved.

PasteEx

The following classes contain this method:

- Layer

Copyright 2014 Corel Corporation. All rights reserved.

PasteSpecial

The following classes contain this method:

- Layer

Copyright 2014 Corel Corporation. All rights reserved.

PlaceShape

The following classes contain this method:

- TableCell

Copyright 2014 Corel Corporation. All rights reserved.

PlaceShapeRange

The following classes contain this method:

- TableCell

Copyright 2014 Corel Corporation. All rights reserved.

PlaceTextInside

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

Play

The following classes contain this method:

- CorelScriptFile

Copyright 2014 Corel Corporation. All rights reserved.

Previous

The following classes contain this method:

- Node
- Segment
- SubPath

Copyright 2014 Corel Corporation. All rights reserved.

PrintColorProof

The following classes contain this method:

- Document
- PrintSettings

Copyright 2014 Corel Corporation. All rights reserved.

PrintOut

The following classes contain this method:

- Document
- PrintJob
- PrintSettings

Copyright 2014 Corel Corporation. All rights reserved.

PSet

The following classes contain this method:

- PatternCanvas

Copyright 2014 Corel Corporation. All rights reserved.

PublishToPDF

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

PurgeUnusedSymbols

The following classes contain this method:

- SymbolLibrary

Copyright 2014 Corel Corporation. All rights reserved.

PutCopy

The following classes contain this method:

- PatternCanvas

Copyright 2014 Corel Corporation. All rights reserved.

PutCurveInfo

The following classes contain this method:

- Curve
- SubPath

Copyright 2014 Corel Corporation. All rights reserved.

PutHandles

The following classes contain this method:

- DuotoneInk

Copyright 2014 Corel Corporation. All rights reserved.

Quit

The following classes contain this method:

- Application

Copyright 2014 Corel Corporation. All rights reserved.

Range

The following classes contain this method:

- Nodes
- Segments
- ShapeRange
- Shapes
- SnapPoints
- TableCells
- Text
- TextRange

Copyright 2014 Corel Corporation. All rights reserved.

Redo

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Refresh

The following classes contain this method:

- Application
- Window
- Windows

Copyright 2014 Corel Corporation. All rights reserved.

RegisterObject

The following classes contain this method:

Copyright 2014 Corel Corporation. All rights reserved.

RegistrationAssign

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

RegistryQuery

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

Remove

The following classes contain this method:

- ArrowHeads
- NodeRange
- OutlineStyles
- PatternCanvases
- SegmentRange
- ShapeRange
- SnapPointRange
- TableCellRange

Copyright 2014 Corel Corporation. All rights reserved.

RemoveAll

The following classes contain this method:

- NodeRange
- SegmentRange
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

RemoveByReference

The following classes contain this method:

- SnapPointRange

Copyright 2014 Corel Corporation. All rights reserved.

RemoveCell

The following classes contain this method:

- TableCellRange

Copyright 2014 Corel Corporation. All rights reserved.

RemoveColor

The following classes contain this method:

- Palette

Copyright 2014 Corel Corporation. All rights reserved.

RemoveFromContainer

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

RemoveFromSelection

The following classes contain this method:

- Document
- NodeRange
- SegementRange
- Shape
- ShapeRange
- SnapPointRange

Copyright 2014 Corel Corporation. All rights reserved.

RemoveHandle

The following classes contain this method:

- DuotoneInk

Copyright 2014 Corel Corporation. All rights reserved.

RemoveOverlaps

The following classes contain this method:

- Curve

Copyright 2014 Corel Corporation. All rights reserved.

RemovePluginCommand

The following classes contain this method:

- Application

Copyright 2014 Corel Corporation. All rights reserved.

RemoveRange

The following classes contain this method:

- NodeRange
- SegmentRange
- ShapeRange
- TableCellRange

Copyright 2014 Corel Corporation. All rights reserved.

Rename

The following classes contain this method:

- CorelScriptTools
- Style

Copyright 2014 Corel Corporation. All rights reserved.

Reorder

The following classes contain this method:

- DataField

Copyright 2014 Corel Corporation. All rights reserved.

Repeat

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Replace

The following classes contain this method:

- ArrowHeads
- Text
- TextRange

Copyright 2014 Corel Corporation. All rights reserved.

ReplaceWide

The following classes contain this method:

- TextRange

Copyright 2014 Corel Corporation. All rights reserved.

ReplaceWith

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

Resample

The following classes contain this method:

- Bitmap

Copyright 2014 Corel Corporation. All rights reserved.

Reset

The following classes contain this method:

- DuotoneInk
- DuotoneOverprint
- ExportFilter
- HatchPattern
- HatchPatterns
- ImportFilter
- PDFVBASettings
- PrintSettings
- PSScreenOptions

Copyright 2014 Corel Corporation. All rights reserved.

ResetCropEnvelope

The following classes contain this method:

- Bitmap
- EPS

Copyright 2014 Corel Corporation. All rights reserved.

ResetOverprints

The following classes contain this method:

- Duotone

Copyright 2014 Corel Corporation. All rights reserved.

ResetSettings

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Resize

The following classes contain this method:

- BSplineControlPoints

Copyright 2014 Corel Corporation. All rights reserved.

ResolveAllBitmapsLinks

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

ResolveLink

The following classes contain this method:

- Bitmap

Copyright 2014 Corel Corporation. All rights reserved.

RestoreAllLinks

The following classes contain this method:

- CloneLink

Copyright 2014 Corel Corporation. All rights reserved.

RestoreCloneLink

The following classes contain this method:

- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

RestoreSettings

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Reverse

The following classes contain this method:

- TextRanges

Copyright 2014 Corel Corporation. All rights reserved.

ReverseDirection

The following classes contain this method:

- Curve
- SubPath

Copyright 2014 Corel Corporation. All rights reserved.

Revert

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

RevertToShapes

The following classes contain this method:

- Symbol

Copyright 2014 Corel Corporation. All rights reserved.

RGBAssign

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

RmFolder

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

Rotate

The following classes contain this method:

- EffectExtrude
- NodeRange
- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

RotateArea

The following classes contain this method:

- PatternCanvas

Copyright 2014 Corel Corporation. All rights reserved.

RotateEx

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

RunMacro

The following classes contain this method:

- GMSManager

Copyright 2014 Corel Corporation. All rights reserved.

Save

The following classes contain this method:

- Document
- Duotone
- DuotoneInk
- OutlineStyles
- Palette
- PDFVBASettings
- PrintSettings

Copyright 2014 Corel Corporation. All rights reserved.

SaveAs

The following classes contain this method:

- Bitmap
- Document
- Palette

Copyright 2014 Corel Corporation. All rights reserved.

SaveAsCopy

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

SaveSettings

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

SaveStyle

The following classes contain this method:

- ColorManager

Copyright 2014 Corel Corporation. All rights reserved.

SaveStyleSheet

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

SaveStyleSheetAsDefault

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Scallop

The following classes contain this method:

- Node
- NodeRange
- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

ScreenToDocument

The following classes contain this method:

- Window

Copyright 2014 Corel Corporation. All rights reserved.

Select

The following classes contain this method:

- EffectEnvelope
- HatchFill
- PatternCanvas
- PostScriptFill
- PSScreenOptions
- TableCell
- TableCellRange
- TableColumn
- TableColumns
- TableRow
- TableRows
- TableShape
- TextRange
- TextureFill

Copyright 2014 Corel Corporation. All rights reserved.

Selection

The following classes contain this method:

- Curve
- Document
- SubPath

Copyright 2014 Corel Corporation. All rights reserved.

SelectPrinter

The following classes contain this method:

- PrintSettings

Copyright 2014 Corel Corporation. All rights reserved.

SelectShapesAtPoint

The following classes contain this method:

- Page

Copyright 2014 Corel Corporation. All rights reserved.

SelectShapesFromRectangle

The following classes contain this method:

- Page

Copyright 2014 Corel Corporation. All rights reserved.

SelectSize

The following classes contain this method:

- Page

Copyright 2014 Corel Corporation. All rights reserved.

Separate

The following classes contain this method:

- Effect
- Shape

Copyright 2014 Corel Corporation. All rights reserved.

SetActualSize

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

SetAllMargins

The following classes contain this method:

- TableCell

Copyright 2014 Corel Corporation. All rights reserved.

SetAngle

The following classes contain this method:

- FountainFill

Copyright 2014 Corel Corporation. All rights reserved.

SetAutoSnap

The following classes contain this method:

- SnapPointRange

Copyright 2014 Corel Corporation. All rights reserved.

SetBasedOn

The following classes contain this method:

- Style

Copyright 2014 Corel Corporation. All rights reserved.

SetBevel

The following classes contain this method:

- EffectExtrude

Copyright 2014 Corel Corporation. All rights reserved.

SetBorders

The following classes contain this method:

- TableBorders

Copyright 2014 Corel Corporation. All rights reserved.

SetBoundingBox

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetCellSpacing

The following classes contain this method:

- TableShape

Copyright 2014 Corel Corporation. All rights reserved.

SetCenterPosition

The following classes contain this method:

- Ellipse

Copyright 2014 Corel Corporation. All rights reserved.

SetColorCount

The following classes contain this method:

- TraceSettings

Copyright 2014 Corel Corporation. All rights reserved.

SetColorMode

The following classes contain this method:

- TraceSettings

Copyright 2014 Corel Corporation. All rights reserved.

SetColumns

The following classes contain this method:

- TextFrame

Copyright 2014 Corel Corporation. All rights reserved.

SetCustomPaperSize

The following classes contain this method:

- PrintSettings

Copyright 2014 Corel Corporation. All rights reserved.

SetDistanceFromObject

The following classes contain this method:

- Dimension

Copyright 2014 Corel Corporation. All rights reserved.

SetEdgePad

The following classes contain this method:

- FountainFill

Copyright 2014 Corel Corporation. All rights reserved.

SetEndingControlPointPosition

The following classes contain this method:

- Segment

Copyright 2014 Corel Corporation. All rights reserved.

SetFillMode

The following classes contain this method:

- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetFillScale

The following classes contain this method:

- HatchFill

Copyright 2014 Corel Corporation. All rights reserved.

SetFrequency

The following classes contain this method:

- Grid

Copyright 2014 Corel Corporation. All rights reserved.

SetHeight

The following classes contain this method:

- TableRow

Copyright 2014 Corel Corporation. All rights reserved.

SetLayout

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

SetLight

The following classes contain this method:

- EffectExtrude

Copyright 2014 Corel Corporation. All rights reserved.

SetLineScale

The following classes contain this method:

- HatchFill

Copyright 2014 Corel Corporation. All rights reserved.

SetLineSpacing

The following classes contain this method:

- TextRange

Copyright 2014 Corel Corporation. All rights reserved.

SetMargins

The following classes contain this method:

- TableCell

Copyright 2014 Corel Corporation. All rights reserved.

SetMatrix

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

SetName

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

SetNoBackColor

The following classes contain this method:

- HatchFill

Copyright 2014 Corel Corporation. All rights reserved.

SetNoOutline

The following classes contain this method:

- Outline

Copyright 2014 Corel Corporation. All rights reserved.

SetOffset

The following classes contain this method:

- EffectDropShadow

Copyright 2014 Corel Corporation. All rights reserved.

SetOrigin

The following classes contain this method:

- HatchPattern

Copyright 2014 Corel Corporation. All rights reserved.

SetOutlineProperties

The following classes contain this method:

- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetOverhang

The following classes contain this method:

- Dimension

Copyright 2014 Corel Corporation. All rights reserved.

SetPaperSize

The following classes contain this method:

- PrintSettings

Copyright 2014 Corel Corporation. All rights reserved.

SetPixelAlignedRendering

The following classes contain this method:

- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetPointAndAngle

The following classes contain this method:

- Guide

Copyright 2014 Corel Corporation. All rights reserved.

SetPoints

The following classes contain this method:

- Guide

Copyright 2014 Corel Corporation. All rights reserved.

SetPosition

The following classes contain this method:

- BSplineControlPoint
- Node
- Shape
- ShapeRange
- SnapPoint
- SubPath

Copyright 2014 Corel Corporation. All rights reserved.

SetPositionEx

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetProgressMessage

The following classes contain this method:

- AppStatus

Copyright 2014 Corel Corporation. All rights reserved.

SetProperties

The following classes contain this method:

- BSplineControlPoint
- HatchPattern
- Outline
- PostScriptFill
- PSScreenOptions
- TextureFill

Copyright 2014 Corel Corporation. All rights reserved.

SetProperty

The following classes contain this method:

- Style

Copyright 2014 Corel Corporation. All rights reserved.

SetPropertyAsString

The following classes contain this method:

- Style

Copyright 2014 Corel Corporation. All rights reserved.

SetRadius

The following classes contain this method:

- Ellipse
- Rectangle

Copyright 2014 Corel Corporation. All rights reserved.

SetRange

The following classes contain this method:

- TextRange

Copyright 2014 Corel Corporation. All rights reserved.

SetRect

The following classes contain this method:

- Rect

Copyright 2014 Corel Corporation. All rights reserved.

SetReverseTerminatorLength

The following classes contain this method:

- DimensionLinear

Copyright 2014 Corel Corporation. All rights reserved.

SetRotationCenter

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetRoundness

The following classes contain this method:

- Rectangle

Copyright 2014 Corel Corporation. All rights reserved.

SetSize

The following classes contain this method:

- Page
- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetSizeEx

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SetStartingControlPointPosition

The following classes contain this method:

- Segment

Copyright 2014 Corel Corporation. All rights reserved.

SetTextLabelGap

The following classes contain this method:

- Dimension

Copyright 2014 Corel Corporation. All rights reserved.

SetType

The following classes contain this method:

- NodeRange
- SegmentRange

Copyright 2014 Corel Corporation. All rights reserved.

SetValues

The following classes contain this method:

- DuotoneOverprint

Copyright 2014 Corel Corporation. All rights reserved.

SetViewArea

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

SetViewPoint

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

SetWidth

The following classes contain this method:

- TableColumn

Copyright 2014 Corel Corporation. All rights reserved.

AlignAndDistribute

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

ShapeUngroupAllEx

The following classes contain this method:

- ShapeUngroupAllEx

Copyright 2014 Corel Corporation. All rights reserved.

Share

The following classes contain this method:

- ExtrudeVanishingPoint

Copyright 2014 Corel Corporation. All rights reserved.

ShowDialog

The following classes contain this method:

- ExportFilter
- ImportFilter
- PDFVBASettings
- Printer
- PrintSettings
- TraceSettings

Copyright 2014 Corel Corporation. All rights reserved.

Skew

The following classes contain this method:

- NodeRange
- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

SkewEx

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

Smoothen

The following classes contain this method:

- NodeRange

Copyright 2014 Corel Corporation. All rights reserved.

SnapPointsOfType

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

Sort

The following classes contain this method:

- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

Split

The following classes contain this method:

- EffectBlend
- TableCell

Copyright 2014 Corel Corporation. All rights reserved.

SpotAssign

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

SpotAssignByName

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

StepAndRepeat

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

Straighten

The following classes contain this method:

- TextRange

Copyright 2014 Corel Corporation. All rights reserved.

Stretch

The following classes contain this method:

- NodeRange
- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

StretchEx

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

StringAssign

The following classes contain this method:

- Color
- Style

Copyright 2014 Corel Corporation. All rights reserved.

SwapData

The following classes contain this method:

- TreeNode

Copyright 2014 Corel Corporation. All rights reserved.

SwapGroupData

The following classes contain this method:

- TreeNode

Copyright 2014 Corel Corporation. All rights reserved.

TextFind

The following classes contain this method:

- Page

Copyright 2014 Corel Corporation. All rights reserved.

TextReplace

The following classes contain this method:

- Page

Copyright 2014 Corel Corporation. All rights reserved.

ToCentimeters

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

ToCiceros

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

ToDidots

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

ToFitAllObjects

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToFitArea

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToFitPage

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToFitPageHeight

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToFitPageWidth

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToFitSelection

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToFitShape

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToFitShapeRange

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ToInches

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

ToPicas

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

ToPoints

The following classes contain this method:

- CorelScriptTools

Copyright 2014 Corel Corporation. All rights reserved.

ToString

The following classes contain this method:

- Color
- Style

Copyright 2014 Corel Corporation. All rights reserved.

ToUnits

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Trace

The following classes contain this method:

- Bitmap

Copyright 2014 Corel Corporation. All rights reserved.

TransformMatrix

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

Translate

The following classes contain this method:

- CorelScriptFile
- FountanFill

Copyright 2014 Corel Corporation. All rights reserved.

Trim

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

UnadviseEvents

The following classes contain this method:

- Application
- Document

Copyright 2014 Corel Corporation. All rights reserved.

Undo

The following classes contain this method:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Unfreeze

The following classes contain this method:

- EffectLens
- Transparency

Copyright 2014 Corel Corporation. All rights reserved.

Ungroup

The following classes contain this method:

- EffectLens
- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

UngroupAll

The following classes contain this method:

- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

UngroupAllEx

The following classes contain this method:

- ShapeRange
- Shape
- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

Union

The following classes contain this method:

- Rect

Copyright 2014 Corel Corporation. All rights reserved.

UnLink

The following classes contain this method:

- TextFrame
- TreeNode

Copyright 2014 Corel Corporation. All rights reserved.

Unlock

The following classes contain this method:

- ShapeRange

Copyright 2014 Corel Corporation. All rights reserved.

UnlockAllShapes

The following classes contain this method:

- Page

Copyright 2014 Corel Corporation. All rights reserved.

Unmerge

The following classes contain this method:

- TableCell
- TableCellRange

Copyright 2014 Corel Corporation. All rights reserved.

UpdateLink

The following classes contain this method:

- Bitmap
- OLE

Copyright 2014 Corel Corporation. All rights reserved.

UpdateLinks

The following classes contain this method:

- SymbolDefinition

Copyright 2014 Corel Corporation. All rights reserved.

UpdateProgress

The following classes contain this method:

- AppStatus

Copyright 2014 Corel Corporation. All rights reserved.

User

The following classes contain this method:

- SnapPoints

Copyright 2014 Corel Corporation. All rights reserved.

UserAssign

The following classes contain this method:

- Color
- Fill
- Outline
- PSScreenOptions
- Transparency

Copyright 2014 Corel Corporation. All rights reserved.

UserAssignEx

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

Weld

The following classes contain this method:

- Shape

Copyright 2014 Corel Corporation. All rights reserved.

WeldWith

The following classes contain this method:

- Curve

Copyright 2014 Corel Corporation. All rights reserved.

YIQAssign

The following classes contain this method:

- Color

Copyright 2014 Corel Corporation. All rights reserved.

ZoomIn

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ZoomInAtPoint

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

ZoomOut

The following classes contain this method:

- ActiveView

Copyright 2014 Corel Corporation. All rights reserved.

AfterExport

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

AfterPrint

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

AfterSave

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

BeforeExport

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

BeforePrint

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

BeforeSave

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Close

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

DocumentAfterExport

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentAfterPrint

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentAfterSave

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentBeforeExport

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentBeforePrint

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentBeforeSave

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentClose

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentNew

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

DocumentOpen

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

LayerActivate

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

LayerChange

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

LayerCreate

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

LayerDelete

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

OnPluginCommand

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

OnUpdatePluginCommand

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

Open

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

PageActivate

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

PageChange

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

PageCreate

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

PageDelete

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

QueryClose

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

QueryDocumentClose

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

QueryDocumentExport

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

QueryDocumentPrint

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

QueryDocumentSave

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

QueryExport

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

QueryPrint

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

QueryQuit

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

QuerySave

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Quit

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

SelectionChange

The following classes contain this event:

- Application
- Document
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

ShapeChange

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

ShapeCreate

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

ShapeDelete

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

ShapeDistort

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

ShapeMove

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

ShapeTransform

The following classes contain this event:

- Document

Copyright 2014 Corel Corporation. All rights reserved.

Start

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

WindowActivate

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

WindowDeactivate

The following classes contain this event:

- Application
- GlobalMacroStorage

Copyright 2014 Corel Corporation. All rights reserved.

Importing files into layers

Files of all supported formats can be imported.

- Files are imported at the **Layer** level because each imported object is assigned to a specified layer on a specified page. However, files are exported at the **Document** level because the range of exported objects can extend over multiple layers and multiple pages (see Exporting files from documents).

The **Layer** class has two file-import methods: **Import** and **ImportEx**.

- The wide selection of supported file formats is due to the vast number of filters that are available to the application. Each filter lets you work with the files from another graphics application. To learn more about working with these filters, see Working with import filters and export filters.

Understanding the Layer.Import method

The **Layer.Import** method provides basic functionality for importing files.

The following VBA code imports the file **C:\logotype.gif** into the active layer at the center of the page:

```
ActiveLayer.Import "C:\logotype.gif"
```

Importing a file selects the contents of that file and deselects any other selected objects in the document. You can therefore reposition or resize the imported objects by getting the document selection:

```
ActiveDocument.Unit = cdrInch
ActiveSelection.SetSize 3, 2
```

Some file formats can be imported by using one of several filters, so it is important to understand the benefits of each available filter. For example, when importing an Encapsulated PostScript (EPS) file, you can choose between the EPS filter and the PDF filter. The EPS filter lets you do the following:

- import an EPS file as a placeable object that can be printed but not modified
- interpret the PostScript portion of the file, so that you can import the original artwork from within the file rather than its low-resolution header

To specify which filter to use, you can include the optional parameter **Filter**, as in the following VBA example:

```
ActiveLayer.Import "C:\map.eps", cdrPSInterpreted
```

Understanding the Layer.ImportEx method

The **Layer.ImportEx** method provides much better control over the import filter through its optional use of a **StructImportOptions** object. The following VBA code imports the specified file as a linked file:

```
Dim iFilt As ImportFilter
Dim importProps As New StructImportOptions
importProps.LinkBitmapExternally = True
Set iFilt = ActiveLayer.ImportEx("C:\world-map.epsf", cdrAutoSense, importProps)
iFilt.Finish
```

Copyright 2014 Corel Corporation. All rights reserved.

Including queries in macros

CorelDRAW supports Corel Query Language (CQL), a syntax for including queries in macros.

This section contains the following topics:

- Understanding CQL
- Understanding the query syntax
- Understanding the available query criteria

Copyright 2014 Corel Corporation. All rights reserved.

ItemByIndex

The following classes contain this property:

- Properties

Copyright 2014 Corel Corporation. All rights reserved.

Locking and hiding layers

Layer objects feature the properties **Editable** and **Visible**, which control (respectively) whether the layer is editable and whether its contents are visible. Both properties are Boolean. By setting both the properties to **True**, you unlock and display the layer for editing. By setting either property to **False**, however, you lock the layer such that it cannot be edited.

The following sample VBA code locks, but displays, the layer on the active page:

```
ActivePage.Layers("Layer 1").Visible = True
ActivePage.Layers("Layer 1").Editable = False
```

The result of any changes to these properties is immediately displayed in the Object Manager.

The preceding example affects only the active page. You can access the layer settings for a given page by specifying a page from the **Document.Pages** collection, or by referencing the **Document.ActivePage** property. To make the changes to all pages in a document, use the **Document.MasterPage** property:

```
ActiveDocument.MasterPage.Layers("Layer 1").Visible = True
```

- For more information on working with pages, see Working with pages.

Copyright 2014 Corel Corporation. All rights reserved.

Making macros user-friendly

An important part of many macro solutions is the user interface. A well-designed interface improves the ease-of-use, power, and acceptance of a macro solution. Simple user interfaces can be created with toolbars, while more complex interfaces can be created with dialog boxes or dockers — and can even allow the user to interact with the mouse.

However, for some macro solutions, a user interface alone is not enough. To make a macro solution as user-friendly as possible, you can provide documentation for it.

In this section

This section contains the following topics:

- Providing toolbars for macros
- Providing dialog boxes for macros
- Providing user interaction for macros
- Providing documentation for macros

Copyright 2014 Corel Corporation. All rights reserved.

A

The methods in this section begin with the letter *A*.

Copyright 2014 Corel Corporation. All rights reserved.

B

The methods in this section begin with the letter *B*.

Copyright 2014 Corel Corporation. All rights reserved.

C

The methods in this section begin with the letter *C*.

Copyright 2014 Corel Corporation. All rights reserved.

D

The methods in this section begin with the letter *D*.

Copyright 2014 Corel Corporation. All rights reserved.

E

The methods in this section begin with the letter *E*.

Copyright 2014 Corel Corporation. All rights reserved.

F

The methods in this section begin with the letter *F*.

Copyright 2014 Corel Corporation. All rights reserved.

G

The methods in this section begin with the letter *G*.

Copyright 2014 Corel Corporation. All rights reserved.

H

The methods in this section begin with the letter *H*.

Copyright 2014 Corel Corporation. All rights reserved.

I

The methods in this section begin with the letter *I*.

Copyright 2014 Corel Corporation. All rights reserved.

J

The methods in this section begin with the letter *J*.

Copyright 2014 Corel Corporation. All rights reserved.

K

The methods in this section begin with the letter *K*.

Copyright 2014 Corel Corporation. All rights reserved.

L

The methods in this section begin with the letter *L*.

Copyright 2014 Corel Corporation. All rights reserved.

M

The methods in this section begin with the letter *M*.

Copyright 2014 Corel Corporation. All rights reserved.

Methods

The CorelDRAW object model contains several methods, which are arranged alphabetically in this section.

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Copyright 2014 Corel Corporation. All rights reserved.

N

The methods in this section begin with the letter *N*.

Copyright 2014 Corel Corporation. All rights reserved.

O

The methods in this section begin with the letter *O*.

Copyright 2014 Corel Corporation. All rights reserved.

P

The methods in this section begin with the letter *P*.

Copyright 2014 Corel Corporation. All rights reserved.

Q

The methods in this section begin with the letter Q.

Copyright 2014 Corel Corporation. All rights reserved.

R

The methods in this section begin with the letter *R*.

Copyright 2014 Corel Corporation. All rights reserved.

S

The methods in this section begin with the letter *S*.

Copyright 2014 Corel Corporation. All rights reserved.

T

The methods in this section begin with the letter *T*.

Copyright 2014 Corel Corporation. All rights reserved.

U

The methods in this section begin with the letter *U*.

Copyright 2014 Corel Corporation. All rights reserved.

W

The methods in this section begin with the letter *W*.

Copyright 2014 Corel Corporation. All rights reserved.

Y

The methods in this section begin with the letter *Y*.

Copyright 2014 Corel Corporation. All rights reserved.

Z

The methods in this section begin with the letter *Z*.

Copyright 2014 Corel Corporation. All rights reserved.

Modifying documents

You can modify a document regardless of whether it is active.

- Modifying an inactive document does not activate that document. To activate a document, you must use its **Activate** method (see Activating documents).

The following VBA code adds a layer named “fooLayer” to the third open document:

```
Dim doc As Document
Set doc = Documents(3)
doc.ActivePage.CreateLayer "fooLayer"
```

The following VBA code uses the **findDocument()** function to add a layer named “fooLayer” to the inactive document named **barDoc.cdr**:

```
Dim doc As Document
Set doc = findDocument("barDoc.cdr")
If Not doc Is Nothing Then doc.ActivePage.CreateLayer "fooLayer"
```

Copyright 2014 Corel Corporation. All rights reserved.

Modifying pages

You can modify a page regardless of whether it is active.

- Activating a page in an inactive document does not activate that document. To activate a document, you must use its **Activate** method (see Activating documents).

By explicitly referencing the page that you want to modify, you can make those changes without activating the page. The following VBA code deletes all shapes on page 3 of the active document without activating that page:

```
Public Sub DeleteShapesFromPage3()
Dim doc As Document
Set doc = ActiveDocument
doc.Pages(3).Shapes.All.Delete
End Sub
```

Copyright 2014 Corel Corporation. All rights reserved.

Modules

The CorelDRAW object model contains several modules, which are arranged alphabetically in this section.

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Copyright 2014 Corel Corporation. All rights reserved.

U

The modules in this section begin with the letter *U*:

- UICategories

Copyright 2014 Corel Corporation. All rights reserved.

Object Model Reference

This Object Model Reference for CorelDRAW is divided into eight sections:

- Classes
- Modules
- Types
- Enumerations
- Properties
- Methods
- Events
- Constants

Topics within each section are listed in alphabetical order for easy navigation and contain links to other relevant topics. In addition, code samples are provided for selected properties, methods, and events in the CorelDRAW object model.

- The CorelDRAW application object is not referenced in the sample code. The application object does not need to be included when you are referencing objects within the application; however, if you are using VBA to reference the CorelDRAW application from within another application, the reference must include the CorelDRAW application object.

Legend

This Object Model Reference uses the following legend:

— default property or method

— read-only property

Copyright 2014 Corel Corporation. All rights reserved.

Opening documents

To open a document, you can use the **Application.OpenDocument** method.

```
Dim doc As Document
Set doc = OpenDocument("C:\graphic1.cdr")
```

If you want, you can use event handlers to respond to events that are triggered by opening a document:

- **Application.DocumentOpen**
- **Document.Open**
- **GlobalMacroStorage.DocumentOpen**

Copyright 2014 Corel Corporation. All rights reserved.

Organizing and deploying macros

When you've finished developing your macro solution, you can make it available to other users.

In this section

This section contains the following topics:

- Organizing macros
- Deploying macros

Copyright 2014 Corel Corporation. All rights reserved.

Organizing macros

To make your macro solutions easy to deploy, you can organize them. Here are some tips:

- To sort your macros, use a separate code module for each macro, and then group related macros into a single GMS file.
- To help users find the entry point to a macro, place all public subroutines into a single code module so that the macro can be called from within the application.

Copyright 2014 Corel Corporation. All rights reserved.

Printing documents

Using VBA to print documents is straightforward: almost all settings that are available in the **Print** dialog box are available to the **Document.PrintSettings** property. When these properties are set, printing the document is simply a matter of calling the **Document.PrintOut** method.

For example, the following VBA code prints three copies of pages 1, 3, and 4 to a level-3 PostScript printer:

```
With ActiveDocument.PrintSettings
.Copies = 3
.PrintRange = prnPageRange
.PageRange = "1, 3-4"
.Options.PrintJobInfo = True
With .PostScript
.DownloadType1 = True
.Level = prnPSLevel3
End With
End With
ActiveDocument.PrintOut
```

Each page in the **Print** dialog box has a corresponding object-model class that contains all settings for that page. The following table lists these classes.

Page in Print dialog box	Class in object model
General	**PrintSettings**
Layout	**PrintSettings** and **PrintLayout**
Separations	**PrintSeparations** and **PrintTrapping**
Prepress	**PrintPrepress**
PostScript	**PrintPostScript**
Misc	**PrintOptions**

- You cannot set layout options in VBA. However, if necessary, you can open the **Print** dialog box by using the **PrintSettings.ShowDialog** method.

- You can print only the selected objects in a document by setting the **PrintSettings.PrintRange** property to **prnSelection**.
- You can use a specific printer in the **Application.Printers** collection by specifying it in the **PrintSettings.Printer** property.
- You can save a printing profile by using the **PrintSettings.Save** method.
- You can access a saved printing profile by using the **PrintSettings.Load** method, but be sure to specify the full path to the profile.
- You can reset the print settings by using the **PrintSettings.Reset** method.

If you want, you can use event handlers to respond to events that are triggered by opening the **Print** dialog box:

- **Application.DocumentBeforePrint**
- **Document.BeforePrint**
- **GlobalMacroStorage.DocumentBeforePrint**

You can also use event handlers to respond to events that are triggered by printing a document and closing the **Print** dialog box:

- **Application.DocumentAfterPrint**
- **Document.AfterPrint**
- **GlobalMacroStorage.DocumentAfterPrint**

Finally, you can also use event handlers to respond to events that are triggered when the user responds to a request to print a document:

- **Application.QueryDocumentPrint**
- **Document.QueryPrint**
- **GlobalMacroStorage.QueryDocumentPrint**

Copyright 2014 Corel Corporation. All rights reserved.

A

The properties in this section begin with the letter *A*.

Copyright 2014 Corel Corporation. All rights reserved.

B

The properties in this section begin with the letter *B*.

Copyright 2014 Corel Corporation. All rights reserved.

C

The properties in this section begin with the letter *C*.

Copyright 2014 Corel Corporation. All rights reserved.

D

The properties in this section begin with the letter *D*.

Copyright 2014 Corel Corporation. All rights reserved.

E

The properties in this section begin with the letter *E*.

Copyright 2014 Corel Corporation. All rights reserved.

F

The properties in this section begin with the letter *F*.

Copyright 2014 Corel Corporation. All rights reserved.

G

The properties in this section begin with the letter *G*.

Copyright 2014 Corel Corporation. All rights reserved.

H

The properties in this section begin with the letter *H*.

Copyright 2014 Corel Corporation. All rights reserved.

I

The properties in this section begin with the letter *I*.

Copyright 2014 Corel Corporation. All rights reserved.

J

The properties in this section begin with the letter *J*.

Copyright 2014 Corel Corporation. All rights reserved.

K

The properties in this section begin with the letter *K*.

Copyright 2014 Corel Corporation. All rights reserved.

L

The properties in this section begin with the letter *L*.

Copyright 2014 Corel Corporation. All rights reserved.

M

The properties in this section begin with the letter *M*.

Copyright 2014 Corel Corporation. All rights reserved.

Properties

The CorelDRAW object model contains several properties, which are arranged alphabetically in this section.

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Copyright 2014 Corel Corporation. All rights reserved.

N

The properties in this section begin with the letter *N*.

Copyright 2014 Corel Corporation. All rights reserved.

O

The properties in this section begin with the letter *O*.

Copyright 2014 Corel Corporation. All rights reserved.

P

The properties in this section begin with the letter *P*.

Copyright 2014 Corel Corporation. All rights reserved.

Q

The properties in this section begin with the letter Q.

Copyright 2014 Corel Corporation. All rights reserved.

R

The properties in this section begin with the letter *R*.

Copyright 2014 Corel Corporation. All rights reserved.

S

The properties in this section begin with the letter *S*.

Copyright 2014 Corel Corporation. All rights reserved.

T

The properties in this section begin with the letter *T*.

Copyright 2014 Corel Corporation. All rights reserved.

U

The properties in this section begin with the letter *U*.

Copyright 2014 Corel Corporation. All rights reserved.

V

The properties in this section begin with the letter *V*.

Copyright 2014 Corel Corporation. All rights reserved.

W

The properties in this section begin with the letter *W*.

Copyright 2014 Corel Corporation. All rights reserved.

X

The properties in this section begin with the letter *X*.

Copyright 2014 Corel Corporation. All rights reserved.

Y

The properties in this section begin with the letter *Y*.

Copyright 2014 Corel Corporation. All rights reserved.

Z

The properties in this section begin with the letter *Z*.

Copyright 2014 Corel Corporation. All rights reserved.

Providing dialog boxes for macros

A dialog box provides a user-friendly interface for more complex macro solutions.

For best results, all dialog boxes must provide the following:

- a meaningful title
- an obvious function for cancelling or closing the dialog box
- an easy-to-use layout
- a Help button from which users can access how-to documentation
- a tooltip (that is, a **ControlTipText** string) for every control

There are two types dialog boxes: modal and modeless.

Understanding modal dialog boxes

A modal dialog box locks the application until the user acts on and then closes the dialog box. Most built-in dialog boxes for macro solutions are modal, and most modal dialog boxes provide the following buttons:

- **OK** — performs an action and then closes the dialog box. This button is the default.
- **Cancel** — closes the dialog box without performing an action. This button provides the same functionality as the **Close** button in the upper-right corner of a dialog box.

In addition, some modal dialog boxes provide the following button:

- **Apply** — performs an action that can be commited by clicking the **OK** button or cancelled by clicking the **Cancel** button

Finally, most wizard-style dialog boxes provide the following buttons:

- **Previous** — returns to the previous page. This button can be disabled on the first page of the dialog box.
- **Next** — advances to the next page. This button can be replaced by a **Finish** button on the last page of the dialog box.
- **Finish** — performs the action for the dialog box and then closes the dialog box

Understanding modeless dialog boxes

A modeless dialog box does not lock the application, so the user can leave the dialog box open and continue working in the application. In this way, modeless dialog boxes behave like dockers. Most modeless dialog boxes provide the following buttons:

- **Apply** or **Create** — performs an action (and can, in fact, be specially labeled to describe that action). This button is typically the default.
- **Close** — closes the dialog box. This button is used after the action is applied.

Choosing between modal and modeless dialog boxes

Before you can create a dialog box for your macro solution, you must decide whether to make it modal or modeless by considering what you want the dialog box to achieve.

For example, let's say that you are creating a "one-shot" end-to-end solution such as a customized **Print** dialog box or **Save** dialog box. In this case, you would provide a modal dialog box because it is unlikely that the user would want to apply the specified settings repeatedly.

On the other hand, let's say that you are creating a solution for setting up an effect to apply to a selection of shapes. To let the user specify the desired settings and then apply them repeatedly, you would provide a modeless dialog box.

After choosing which type of dialog box to provide, you are ready to set it up. For information, see Setting up dialog boxes.

After setting up a dialog box, you are ready to code it. For information, see Coding dialog boxes.

Copyright 2014 Corel Corporation. All rights reserved.

Providing documentation for macros

To make a macro as user-friendly as possible, you can provide documentation for it.

One solution is to create a Readme file or a printed manual. Another solution is to incorporate the documentation directly into the user interface for the macro, but this method uses up valuable on-screen “real estate.” Yet another solution is to create an online Help system, but this method requires special tools and a fair amount of additional work.

Perhaps the simplest way to provide macro documentation is in the form of a plain-text file. In fact, upon installation, a macro project can create a registry value that points to the location of this file. In VBA, the following function can be used to open a plain-text file (where the parameter `file` provides the full path to the file, such as **C:\ReadMe.txt**):

```
Public Sub launchNotepad(file As String)
Shell "Notepad.exe" & " " & file, vbNormalFocus
End Sub
```

A much more powerful solution is to provide documentation in HTML format. HTML provides numerous benefits over plain-text, including support for graphics and for hypertext links (such as to specific locations in the document — for example, **index.html#middle**). In VBA, the following function can be used to open an HTML file (where the parameter `url` provides the full path to the file — such as **C:\ReadMe.txt** — or a URL for the file):

```
' Put this Declare statement before all Subs and Functions!
Declare PtrSafe Function ShellExecute Lib "shell32.dll" _
Alias "ShellExecuteA" (ByVal hwnd As LongPtr, _
ByVal lpOperation As String, ByVal lpFile As String, _
ByVal lpParameters As String, ByVal lpDirectory As String, _
ByVal nShowCmd As Long) As LongPtr
Public Sub launchBrowser(url As String)
ShellExecute 0, vbNullString, url, vbNullString, vbNullString, 5
End Sub
```

Copyright 2014 Corel Corporation. All rights reserved.

Providing toolbars for macros

A toolbar provides a basic interface that enhances the user's experience with your macro solution. Toolbars are useful because their buttons are memorable even if small, and because those buttons can be set to display meaningful captions and helpful tooltips.

Designing toolbars for macros

When creating toolbars, you should plan carefully. Having multiple small toolbars containing a few related buttons is better than having one big toolbar containing all of the buttons for all of your macros. By breaking your buttons into small groups, it is much easier to deploy them with the projects to which they belong.

For more information, see the following procedures:

- To create a macro toolbar
- To add buttons to a macro toolbar

Associating images or icons with macros

Macro commands can have an image or icon associated with them. This image or icon can be displayed or hidden on toolbars and menus, and it can be sized as small (16 × 16 pixels), medium (32 × 32 pixels), or large (48 × 48 pixels).

For more information, see the following procedure:

- To associate an image or icon with a macro

Setting captions and tooltips for macros

Each macro can have both a caption and a tooltip. The caption is displayed whenever the menu command is used and can be displayed as part of a button, while the tooltip appears when the pointer hovers over the button or menu item.

For more information, see the following procedures:

- To set a caption for a macro
- To set a tooltip for a macro

Copyright 2014 Corel Corporation. All rights reserved.

Providing user interaction for macros

One way to make your macro solutions more user-friendly is to optimize them for user interaction, such as mouse input. A macro that captures mouse actions gives users real-time influence on the result of that macro.

The CorelDRAW object model provides three main ways to receive mouse input from users, as explained in the following topics:

- Capturing mouse clicks
- Capturing mouse drags
- Capturing coordinates

Copyright 2014 Corel Corporation. All rights reserved.

Publishing documents to PDF

Publishing documents to PDF is a two-step process. The first step is to specify the PDF settings (although this step can be skipped by specifying those settings from the application or by using the default settings). The second step is to export the file.

To specify the PDF settings, you can use the **Document.PDFSettings** property. This property is an object of type **PDFVBASettings** and contains properties for all PDF settings that can be set in the **Publish To PDF** dialog box.

The following VBA code exports pages 2, 3, and 5 to a PDF file named **MyPDF.pdf**:

```
Dim doc As Document
Set doc = ActiveDocument
With doc.PDFSettings
.Author = "Corel Corporation"
.Bookmarks = True
.ColorMode = pdfRGB
.ComplexFillsAsBitmaps = False
.CompressText = True
.DownsampleGray = True
.EmbedBaseFonts = True
.EmbedFonts = True
.Hyperlinks = True
.Keywords = "Test, Example, Corel, CorelDRAW, PublishToPDF"
.Linearize = True
.PageRange = "2-3, 5"
.pdfVersion = pdfVersion13
.PublishRange = pdfPageRange
.TrueTypeToType1 = True
End With
doc.PublishToPDF "C:\MyPDF.pdf"
```

The following VBA example gives more control to the user by displaying the **Publish to PDF Settings** dialog box:

```
Dim doc As Document
Set doc = ActiveDocument
If doc.PDFSettings.ShowDialog = True Then
doc.PublishToPDF "C:\MyPDF.pdf"
End If
```

- Profiles for PDF settings can be saved and loaded by using the **PDFVBASettings.Save** method and **PDFVBASettings.Load** method (respectively).

Copyright 2014 Corel Corporation. All rights reserved.

Querying arrays

Arrays support all common query criteria (see Understanding the available query criteria).

Arrays also support the following query criteria:

Query criteria	Description	Example and result
array.Add(object)	Adds an object to the end of the array	
array.Convert(expression, [object])	Changes the value of each element of the array to the result of evaluation of the given expression	`array(1,2,3,4).convert ($item * 2)` Yields **array(2, 4, 6, 8)**
array.Count()	Returns the number of elements in the array	`array(1,2,3).count` Yields **3**

array.Empty()	Returns **True** if the array contains no elements	`array(1,2,3).empty`
		Yields **False**
		
		`array().empty`
		Yields **True**

array.Filter(expression, [object])	Creates a subset of array elements by including only the elements for which the specified Boolean expression evaluates to **True**. The expression can use the same iterator object as in the **ForEach** method to get the value of the current element and the custom data passed in as the second parameter.	`array(1,2,3,4,5).filter($item > 3)` Yields **array(4, 5)**
array.Find(object)	Finds an object in the array and returns **True** if it is found	`array(1,2,3,4).find(3)` Yields **True**
array.ForEach(object, expression, [object])	Evaluates the specified expression for each element of the array and returns the result of the evaluation. This is an iterative process; the result of evaluation of the previous iteration is passed to the current one.	`array(1,2,3,4).foreach (0, $lasteval + $item)` Yields the sum of all array elements: **10**
	The expression being evaluated can use a special iterator object, which has the following member variables: • **$item** — the current array element • **$index** — the 1-based index of the current array element • **$lasteval** — the result of evaluation of the previous iteration • **$data** — additional data object passed in by the caller The first parameter specifies the initial value passed in through the **$lasteval** variable to the first element iteration. The second parameter is the expression being evaluated for each array element. The last optional parameter is the additional data object passed in as the **$data** variable; if this parameter is omitted, **$data** variable will contain a null object.	`array('this','is','an', 'example').foreach ('Result:', $lasteval + ' ' + $item.ToTitle)` Yields **Result: This Is An Example**
array.IndexOf(object)	Finds an object in the array and returns the 1-based index of the array element found, or **0** if not found	`array('apple','orange','pear').indexof ('orange')` Yields **2**
array.Item(int)	Gets the object identified by the index. The index is 1-based.	`array(1,3,4).item(2)` Yields **3**
array.Join(string)	Joins the array elements into a string by using the specified delimiter string	`array(1,3,4).join(', ')` Yields **'1, 3, 4'**

Copyright 2014 Corel Corporation. All rights reserved.

Querying Boolean objects

Boolean objects support all common query criteria (see Understanding the available query criteria).

Boolean objects do not support any additional query criteria.

Copyright 2014 Corel Corporation. All rights reserved.

Querying colors

You can query the color properties of the shapes on a page or layer.

To access the color properties of the current shape, you must prepend the property name with the at sign (`@`). Thus, `@type` calls the **Color.Type** property of the current shape.

- You can call an expression without specifying the current shape. However, in this case, the `@` operator is unavailable.

Color queries can return values of any of the other supported object types (see Understanding the available query criteria).

Colors support the following query criteria:

Query criteria	Description	Example and result
color.CMY()	Converts the color to CMY and returns a special version of the CQL color object that has three additional properties: **C**, **M**, and **Y**. These properties provide access to individual color components.	`@fill.color.cmy.c = @outline.color.cmy.c` Checks whether the fill color and outline color have the same **C** value
color.CMYK()	Converts the color to CMYK and returns a special version of the CQL color object that has four additional properties: **C**, **M**, **Y**, and **K**. These properties provide access to individual color components.	`@fill.color.cmyk[.c = .m]` Checks whether the fill color has the same **C** value and **M** value
color.COM()	Returns the COM object for the color. This allows access to properties and methods of the color object that are not exposed through CQL.	
color.Gray()	Converts the color to grayscale and returns the gray- level value as an integer	`@fill.gray < 128` Checks whether the gray-level value for the fill color is less than 128
color.HLS()	Converts the color to HLS and returns a special version of the CQL color object that has three additional properties: **H**, **L**, and **S**. These properties provide access to individual color components.	`@fill.color.hls.s = 0` Checks whether the **S** value for the fill color is 0
color.HSB()	Converts the color to HSB and returns a special version of the CQL color object that has three additional properties: **H**, **S**, and **B**. These properties provide access to individual color components.	`@fill.color.hsb.h = 270` Checks whether the **H** value for the fill color is 270
color.IsCMYK()	Return **True** if this is a CMYK color	
color.IsGray()	Return **True** if the color contains only black component	
color.IsSpot()	Returns **True** if the color is a spot color	
color.IsTintable()	Returns **True** if the color can have tints	
color.IsWhite()	Returns **True** if the color is pure white	
color.Lab()	Converts the color to LAB and returns a special version of the CQL color object that has three additional properties: **L**, **A**, and **B**. These properties provide access to individual color components.	`@fill.color.lab.l = 0 and @fill.color.lab.a = -10` Checks whether the **L** value for the fill color is 0 and the **A** value is -10
color.Name()	Returns the name of the color, or **'unnamed color'** if the color doesn't have a name	
color.Palette()	Returns the name of the palette that the color belongs to, or an empty string if the color belongs to a custom palette	

color.PaletteIndex()	Returns the index for the palette color. This method is available only for colors from fixed palettes.	
color.RGB()	Converts the color to RGB and returns a special version of the CQL color object that has three additional properties: **R**, **G**, and **B**. These properties provide access to individual color components.	`@fill.color.rgb.r = 255` Checks whether the **R** value for the fill color is 255
color.SourcePalette()	Returns the original palette name for userink colors	
color.Tint()	Returns the tint value of a spot color	
color.Type()	Returns a string representing the color model. This could be a value such as **'rgb'**, **'cmyk'**, or **'hsb'**.	
color.YIQ()	Converts the color to YIQ and returns a special version of the CQL color object that has three additional properties: **Y**, **I**, and **Q**. These properties provide access to individual color components.	`@fill.color.yiq.y < 10` Checks whether the **Y** value for the fill color is less than 10

Copyright 2014 Corel Corporation. All rights reserved.

Querying double objects

Double objects support all common query criteria (see Understanding the available query criteria).

Double objects also support the following query criteria:

Query criteria	Description	Example and result
double.Abs()	Returns the absolute value of the double	`(-2.5).abs` Yields **2.5**
double.ToDegrees()	Converts the value of radians to degrees	`pi.ToDegrees` Yields **180.0**
double.ToRadians()	Converts the value of degrees to radians	`(12.5).ToRadians` Yields **0.218166156499291**

Copyright 2014 Corel Corporation. All rights reserved.

Querying fills

You can query the fill properties of the shapes on a page or layer.

To access the fill properties of the current shape, you must prepend the property name with the at sign (`@`). Thus, `@type` calls the **Fill.Type** property of the fill for the current shape.

- You can call an expression without specifying the current shape. However, in this case, the `@` operator is unavailable.

Fill queries can return values of any of the other supported object types (see Understanding the available query criteria).

Fills support the following query criteria:

Query criteria	Description	Example and result
fill.Color()	Returns the color of the uniform fill, or null if the fill is not uniform	`@fill.color = 'blue'` Selects all shapes with a blue uniform fill
fill.Fountain()	Returns the properties of a fountain fill, or null if the fill is not fountain	
fill.Type()	Returns a string representing the fill type. Possible values are **'none'**, **'uniform'**, **'fountain'**, **'postscript'**, **'pattern'**, **'texture'**, and **'hatch'**.	`@fill.type = 'fountain' and @fill.fountain.angle = 45` Selects all shapes with a 45-degree fountain fill

Copyright 2014 Corel Corporation. All rights reserved.

Querying global objects

Query criteria that are not preceded by an object name are assumed to belong to the global object. For example, the expression `method(2)` assumes that `global.method(2)` is called.

The global object type provides functions and constants that are common in mathematics, such as pi, e, sin, cos, min, and max.

- `pi` — evaluates to **3.14159265358979**
- `sin(2)` — evaluates to **0.909297426825682**
- `max(1, 2, 4, 3, 0)` — evaluates to **4**

- The query criteria for global objects are available directly and do not require qualification. For example, specifying only `pi` executes **global.pi**.

Global objects support the following query criteria:

Query criteria	Description	Example and result
global.array([object, ...])	Creates an array of objects specified as parameters	`array('a','b','c') .convert($item.repeat($index)).join(',')` Yields **'a,bb,ccc'**
global.atan(double)	Returns the arctangent of the number	
global.bool()	Returns the type definition of the **bool** class. Type definition is a special instance of the class that doesn't have the data associated with it. Only static methods (such as **TypeName**) can be called on type-definition objects.	
global.chr(int)	Returns a character by the specified Unicode character code	`chr(65)` Yields **'A'**
global.cos(double)	Returns the cosine of the number	
global.double()	Returns the type definition of the **double** class	
global.e()	Returns a double value of natural logarithm base	
global.exp(double)	Returns the exponent of the number (e^x)	
global.iif(bool, expression, expression)	Acts as an inline if function. The first parameter is a Boolean expression. If the expression evaluates to **True**, then iif returns the value of the seconds parameter; otherwise, it returns the value of the third parameter.	`iif('This is an apple' .contains('apple'), 'apple','orange')` Yields **'apple'**
global.int()	Returns the type definition of the **int** class	
global.log(double)	Returns the natural logarithm of the value	
global.max(numeric, ...)	Returns the maximum value of the parameters	`max(2,3,4,1,0,13,5)` Yields **13**
global.min(numeric, ...)	Returns the minimum value of the parameters	`min(2,3,4,1,0,13,5)` Yields **0**
global.pi()	Returns a double value of pi	
global.sin(double)	Returns the sine of the number	
global.sqrt(double)	Returns the square root of the number	
global.string()	Returns the type definition of the **string** class	

global.tan(double)	Returns the tangent of the number

Global objects also support the following color-related query criteria:

Query criteria	Description	Example and result
global.cmy(int, int, int)	Returns a CMY color with specified components	`@colors.find(cmy(0,0,0))` Finds shapes with a white CMY color
global.cmyk(int, int, int, int)	Returns a CMYK color with specified components	`@colors.find(cmyk(0,0,0,100))` Finds shapes with a black CMYK color
global.hls(int, int, int)	Returns an HLS color with specified components	`@colors.find(hls(0,0,0))` Finds shapes with a (0,0,0) HLS fill
global.hsb(int, int, int)	Returns an HSB color with specified components	`@fill.color.hsb = hsb(90,100,100)` Checks whether the shape has a (90, 100, 100) HSB fill
global.lab(double, int, int)	Returns a LAB color with specified components	`@outline.color.lab = lab(100,127,127)` Checks whether the shape has a (100, 127, 127) LAB outline
global.rgb(int, int, int)	Returns an RGB color with specified components	`@fill.color = rgb(255,0,0)` Checks whether the shape has a red RGB fill
global.yiq(int, int, int)	Returns a YIQ color with specified components	`@colors.find(yiq(0,0,0))` Finds shapes with a (0,0,0) YIQ color

You can include additional queries that filter the global properties of shapes on a page or layer.

To access the properties and methods of the current object, you must prepend the property name or method name with the at sign (`@`). Thus, `@type` calls the **Type** property of the current object for which the expression is evaluated.

- You can call an expression without specifying the current shape. However, in this case, the `@` operator is unavailable.

Global queries can return values of any of the other supported object types (see Understanding the available query criteria).

Calling macro functions

You can include queries that call macro functions. The **vba** property of the VBA global object provides access to public macro functions through the following syntax:

```
vba.Project.Module.Function(parameters)
```

For example, you can add the following VBA function to **GlobalMacros>Module1**:

```
Public Function Sum(ByVal x As Double, ByVal y As Double) As Double
Sum = x + y
End Function
```

You can then call this function from a query such as the following, yielding the result **5**:

```
MsgBox Application.Evaluate("vba.GlobalMacros.Module1.Sum(2, 3)")
```

Only public functions in code modules can be called in this way. Private functions, functions in class modules or form modules, and subroutines cannot be called from queries.

Copyright 2014 Corel Corporation. All rights reserved.

Querying integers

Integers support all common query critera (see Understanding the available query criteria).

Integers also support the following query criteria:

Query criteria	Description	Example and result
int.Abs()	Returns the absolute value of the integer	`(-2).abs` Yields **2**
int.ToDegrees()	Converts the value of radians to degrees	`1.ToDegrees` Yields **57.2957795130823**
int.ToRadians()	Converts the value of degrees to radians	`180.ToRadians` Yields **3.14159265358979**

Copyright 2014 Corel Corporation. All rights reserved.

Querying null objects

Null objects support all common query criteria (see Understanding the available query criteria).

You can call any method on a null object; the call will succeed, and value **null** will be returned. Thus, `null.somemethod()` returns **null**.

Using query operators with null objects

False is returned when a null object is used with the following comparison operators: =, ==, <, >, >=, and <=. For example, `null = 5` returns **False**. However, please note the following exceptions:

- When two null objects are compared by using the equality operator = or ==, **True** is returned.
- When a null object is compared to a non-null object by using the inequality operator <>, **True** is returned.

Example	Result
`null = 3`	**False**
`null = 'test'`	**False**
`null = null`	**True**
`null <> 3`	**True**
`null <> null`	**False**
`3 > null`	**False**
`null > null`	**False**
`null >= null`	**False**

Copyright 2014 Corel Corporation. All rights reserved.

Querying outlines

You can query the outline properties of the shapes on a page or layer.

To access the outline properties of the current shape, you must prepend the property name with the at sign (`@`). Thus, `@type` calls the **Outline.Type** property of the current shape.

- You can call an expression without specifying the current shape. However, in this case, the `@` operator is unavailable.

Outline queries can return values of any of the other supported object types (see Understanding the available query criteria).

Outlines support the following query criteria:

Query criteria	Description	Example and result
outline.BehindFill()	Returns **True** if the **Behind Fill** option is set	
outline.Color()	Returns the color of the outline, or null if the shape does not have an outline	
outline.IsHairline()	Returns **True** if the outline is hairline	
outline.LineCaps()	Returns a string representing the line-cap type of the outline. Possible values are **'butt'**, **'round'**, and **'square'**.	
outline.LineJoin()	Returns a string representing the corner type of the outline. Possible values are **'miter'**, **'round'**, and **'bevel'**.	
outline.ScaleWithObject()	Returns **True** if the **Scale Outline with Object** option is set	
outline.Type()	Returns a string representing the outline type. The possible values are **'none'**, **'solid'**, **'dot-dash'**, and **'enhanced'**.	`@outline[.type = 'solid' or .type = 'dot-dash']` Selects all shapes with solid or dashed outlines
outline.Width()	Returns the width of the outline, in points	`@outline > {1 pt}` Checks whether the shape outline is wider than 1 pt

Copyright 2014 Corel Corporation. All rights reserved.

Querying shapes

You can query the shapes on a page or layer.

To access the properties and methods of the current shape, you must prepend the property name or method name with the at sign (`@`). Thus, `@type` calls the **Shape.Type** property of the current shape.

- You can call an expression without specifying the current shape. However, in this case, the `@` operator is unavailable.

Shape queries can return values of any of the other supported object type (see Understanding the available query criteria). For example, if the expression is evaluated on a shape object in a document and that shape object exposes a fill property that returns a color object, the following expression can be called to determine whether the current shape has a red fill:

```
@fill.color.name = 'Red'
```

Shapes support the following query criteria:

Query criteria	Description	Example and result
shape.Bottom()	Returns the vertical position of the bottom edge of the shape on the page	`@bottom < {4 in}` Checks whether the bottom edge of the shape is above the vertical 4-inch mark on the page
shape.CenterX()	Returns the horizontal position of the center of the shape on the page	`@centerX = {4.25 in}` Checks whether the center of the shape is at the horizontal 4.25-inch mark on the page
shape.CenterY()	Returns the vertical position of the top center of the shape on the page	`@CenterY = {5.5 in}` Checks whether the top center of the shape is at the vertical 5.5-inch mark on the page

shape.Colors()	Returns an array of colors used in the fill and outline of the shape. This array contains only unique colors, so if the fill and the outline have the same color, only one color is returned.	`@colors.find(cmyk(0,0,0,100))`
		Checks whether the shape contains a CMYK black color
		
		`!@color.filter($item.cmyk.k <> 0).empty`
		Checks whether the shape has any colors with a non-empty black color component

shape.COM()	Returns the COM object for the shape. This method allows access to any property and function of the shape object and its subobject — items that are not directly accessible by CQL.	`(@com.transparency.type = 1)` Checks whether the shape has a uniform transparency applied to it. Refer to the Object Model Reference. section for details on objects and methods.
shape.Fill()	Returns the fill object representing the shape fill	`@fill.type = 'uniform'` Checks whether the shape has a uniform fill applied to it
shape.Height()	Returns the height of the shape, in the current unit of measurement	`@height = {20 mm}` Checks whether the shape is 20 mm high
shape.Left()	Returns the horizontal position of the left edge of the shape on the page	`@left = {0 mm}` Checks whether the left edge of the shape is at the horizontal 0-millimeter mark on the page
shape.Name()	Returns the name of the current shape	`@name = 'MyShape'` Checks whether the shape is named **MyShape**
shape.Outline()	Returns the outline object representing the shape outline	`@outline.color = 'red'` Checks whether the shape has a red outline applied to it
shape.Right()	Returns the horizontal position of the right edge of the shape on the page	`@right > {8.5 in}` Checks whether the right edge of the shape is past the horizontal 8.5-inch mark on the page

shape.Top()	Returns the vertical position of the top edge of the shape on the page	`@top = {11 in}` Checks whether the top edge of the shape is at the vertical 11-inch mark on the page
shape.Type()	Returns a string representing the shape type. The value returned is **'rectangle'**, **'ellipse'**, **'curve'**, **'polygon'**, and so on.	`@type = 'polygon'` Checks whether the shape is a polygon
shape.Width()	Returns the width of the shape, in the current unit of measurement	`@width > {1 in}` Checks whether the shape is less than 1 inch wide

Copyright 2014 Corel Corporation. All rights reserved.

Querying strings

Strings support all common query criteria (see Understanding the available query criteria).

Strings also support the following query criteria:

Query criteria	Description	Example and result
string.CharAt(int)	Gets the character at the given index in the string. The index is zero-based.	`'Test'.CharAt(1)` Yields **'e'**
string.CharCode()	Gets the Unicode character code of the first string character	`'Test'.CharCode` Yields **84**
string.CharCodeAt(int)	Gets the Unicode character code of the character at the given index. The index is zero-based.	`'ABCD'.CharCodeAt(3)` Yields **68**
string.Contains(string)	Returns **True** if the string contains the specified string	`'Dogs'.Contains('Dog')` Yields **True**

string.Delete(int, [int])	Removes the specified characters from the string. The meaning of the parameters is the same as those of **SubStr** function.	`'abcdefg'.Delete(2, 3)`
		Yields **'abfg'**
		
		`'abcdefg'.Delete(-2)`
		Yields **'abcdeg'**

string.Empty()	Returns **True** if the string is empty	`'test'.empty` Yields **False**
string.EndsWith(string)	Returns **True** if the string ends with the specified string	`'Doing'.EndsWith('ing')` Yields **True**
string.IndexOf(string)	Finds a substring and returns the character index of its first occurrence. If it is not found, **-1** is returned.	`'this is a test' .IndexOf('is')` Yields **2**
string.LastIndexOf(string)	Finds a substring and returns the character index of its last occurrence. If it is not found, **-1** is returned.	`'this is a test' .LastIndexOf('is')` Yields **5**
string.Length()	Returns the length of the string	`'test'.length` Yields **4**
string.LTrim([string])	Similar to the **Trim** function, but removes the characters only from the beginning of the string	`' test '.trim()` Yields **'test '**
string.Repeat(int)	Repeats the string the specified number of times	`'test'.repeat(3)` Yields **'testtesttest'**
string.Replace(string, string)	Replaces all instances of the search substring (the first parameter) with the replacement string (the second parameter)	`'green pen, blue pen' .Replace('pen', 'pencil')` Yields **'green pencil, blue pencil'**
string.RTrim([string])	Similar to the **Trim** function, but removes the characters only from the end of the string	`' test '.trim()` Yields **' test'**
string.Split(string)	Splits the string delimited with the specified characters into an array of strings	`'Design<->Implementation <->Testing'.Split('<->')` Yields **array('Design', 'Implementation', 'Testing')**
string.StartsWith(string)	Returns **True** if the string starts with the specified string	`'Apple'.StartsWith('A')` Yields **True**

string.SubStr(int, [int])	Gets a substring of a string. The first parameter is a zero-based index of the first character to include in the substring. The second optional parameter is the length of the substring to retrieve. If it is omitted, the whole remainder of the string is returned.	`'Hello World'.SubStr(6)` Yields **'World'**
		`'Hello World'.SubStr(6,3)` Yields **'Wor'**
	The first index can be negative, to specify the index from the end of the string.	`'This is a test'.SubStr(-4)` Yields **'test'**
	If the last parameter is negative, it indicates the last character from the end of the string.	`'Some words'.SubStr(5, -1)` Yields **'word'**
string.ToCharArray()	Converts the string into an array of characters	`'abc'.ToCharArray` Yields **array('a', 'b', 'c')**
string.ToLower()	Converts the string to lowercase	`'TEST'.ToLower` Yields **'test'**
string.ToTitle()	Converts the string to title case	`'TEST'.ToTitle` Yields **'Test'**
string.ToUpper()	Converts the string to uppercase	`'test'.ToUpper` Yields **'TEST'**
string.Trim([string])	Removes the specified characters from the beginning and end of the string. If the optional string parameter is omitted, then space (code 32) and tab (code 9) characters are removed.	`' test '.trim()` Yields **'test'**
		`'aaaatestbbcc'.trim('abc')` Yields **'test'**

Copyright 2014 Corel Corporation. All rights reserved.

Querying units of measurement

You can query units of measurement, such as to convert values. Linear-dimension units (such as millimeters, inches, meters, centimeters, and points) are supported, as are angle units (such as degrees and radians) and file-size units (such as bytes, kilobytes, megabytes, and gigabytes).

To provide a constant unit of measurement, you must include the unit value and unit of measurement in braces (`{}`):

- `{1 in}` — equals one inch
- `{1 mm}` — equals one millimeter

Certain math operations can be performed on units of measurement, as in the following examples:

- `{1 cm} + {1 mm}` — yields **{1.1 cm}**
- `{1 pt} * 3` — yields **{3 pt}**

Units of measurement support the following query criteria:

Query criteria	Description	Example and result
unit.BaseValue()	Returns the base value of the unit, in base units. Base units are chosen for the category, and all the other units are internally represented in terms of these base units.	`{1 in}.baseValue` Yields **254000**
unit.Category()	Returns a string describing the unit category	`{1 in³}.category` Yields **'Linear Dimensions'**
unit.Convert(string)	Converts the unit value to another unit of measurement of the same category	`{1 in³}.convert('mm')` Yields **16387.064 mm³**
unit.DisplayUnit()	Returns the unit name and possible unit power (such as **'mm'** or **'cm³'**)	`{1 in³}.displayUnit` Yields **'in³'**
unit.Power()	Returns the power of the unit	`{1 in³}.power` Yields **3**
unit.Unit()	Returns the unit name as a string	`{1 in³}.unit` Yields **'in'**
unit.Value()	Returns the value of the unit, in the current units of measurement	`{1 in³}.value` Yields **1**

Handling exponents when querying units of measurement

In addition to multiplication and division, queries on units of measurement support exponents. Units of different degrees can be specified by using the following general syntax:

```
{value unit^power}
```

Consider the following examples:

- `{2 mm^2}` — yields 2 square millimeters
- `{3 in^3}` — yields 3 cubic inches

Units of power zero (such as `{2 mm^0}`) evaluate to regular numeric values. For example, `{2 mm^0}` equals 2.

Powers of 2 and 3 can be specified by using (respectively) the special characters **²** (**U+00B2**) and **³** (**U+00B3**) after the unit of measurement, as in the following examples:

- `{2 mm²}` — yields 2 square millimeters
- `{3 in³}` — yields 3 cubic inches

Multiplying units increases the power of the unit, while dividing them decreases the power:

- `{2 mm} * {2 mm}` — yields **{4 mm²}**
- `{4 mm²} / {1 mm}` — yields **{4 mm}**

Comparing units can be done only with units of the same power:

- `{2 mm} * {2 mm} = {4 mm²}` — yields **True** (because the comparison is correct)
- `{2 mm} * {2 mm} = {4 mm}` — yields an error (because `mm²` is being compared to `mm`)

However, different units of the same unit category can be used in expressions:

- `{1 in} = {72 pt}` — yields **True** (because inches and points belong to the same unit category)

To eliminate rounding error, comparisons of units of measurement are performed with reduced precision. The default precision level is three decimal places of the largest unit in the comparison. For precise comparison, the precise-equality operator (`==`) can be used. Consider the following expressions, given that one millimeter equals 2.83464566929134 points:

- `{1 mm} = {2.835 pt}` — yields **True**
- `{1 mm} == {2.835 pt}` — yields **False**

When converting units, the unit power must be considered. To convert values between different units, the **unit.convert** method must be used. Consider the following examples:

- `{1 in}.convert('mm')` — yields **{25.4 mm}**
- `{1 in²}.convert('mm')` — yields **{645.16 mm²}**

Copyright 2014 Corel Corporation. All rights reserved.

Recording macros

CorelDRAW offers a recording feature that lets you create a macro without needing to manually code it. For many simple and repetitive tasks, recorded macros are a quick, efficient solution: They store the sequence of keys that you press and the mouse actions that you perform within the application. You may prefer to create macros by recording them if you are not familiar with the object model for the application, or if you are not sure which objects and methods to use.If you want to store a recorded macro for future use, you can save it by using the **Record Macro** dialog box. Saving a recorded macro is particularly useful if you want to extend or customize its functionality by editing it in the Macro Editor.

*The **Record Macro** dialog box*

However, if you want to use a recorded macro during the current session only, you can record a temporary macro.

For details on recording macros, see the following procedures:

- To record and save a macro
- To record a temporary macro

Copyright 2014 Corel Corporation. All rights reserved.

Renaming layers

You can rename a layer by editing its **Name** property.

The following VBA code renames “Layer 1” as “Layer with a New Name”:

```
ActivePage.Layers("Layer 1").Name = "Layer with a New Name"
```

Copyright 2014 Corel Corporation. All rights reserved.

Reordering layers

You can reorder layers by using the following two methods of the **Layer** class: **MoveAbove** and **MoveBelow**. Both methods move the specified layer above or below the layer that is referenced as a parameter.

The following VBA code moves the layer called “Layer 1” to immediately below the layer “Guides”:

```
Dim pageLayers As Layers
Set pageLayers = ActivePage.Layers
pageLayers("Layer 1").MoveBelow pageLayers("Guides")
```

The change is immediately reflected in the Object Manager (although it may be apparent only in the Layer Manager).

Copyright 2014 Corel Corporation. All rights reserved.

Reordering pages

A page can be moved to another location in a document by using its **MoveTo** method. The following VBA code moves page 2 to the position of page 4:

```
ActiveDocument.Pages(2).MoveTo 4
```

- Activating a page in an inactive document does not activate that document. To activate a document, you must use its **Activate** method (see Activating documents).

Copyright 2014 Corel Corporation. All rights reserved.

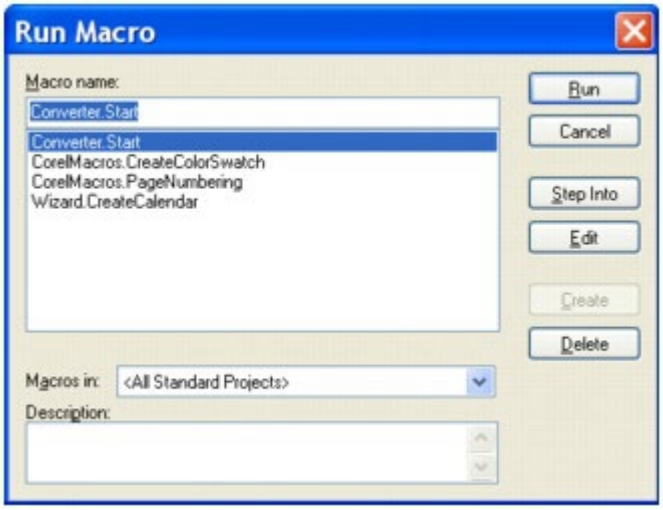
Additional information on CorelDRAW Graphics Suite is available from the following online resources:

- Corel Knowledge Base — a repository of articles written by the Corel Technical Support Services team in response to questions by CorelDRAW Graphics Suite users
- CorelDRAW Graphics Suite community — an online community for CorelDRAW Graphics Suite users to share their experience with the product, ask questions, and receive help and suggestions from other users

Running macros

You can run saved macros in one of two ways:

- from directly within CorelDRAW
- from within the Macro Editor



*The **Run Macro** dialog box*

You can also run any temporary macro recorded in CorelDRAW.

For details on running macros, see the following procedures:

- To run a saved macro
- To run a temporary macro

Copyright 2014 Corel Corporation. All rights reserved.

Saving documents

Two methods can be used for saving documents: **Document.SaveAs** and **Document.Save**.

The **Document.SaveAs** method saves a document by using the specified file path and filename. You can use this method to save a previously unsaved document or to save an existing document to a different file.

- The **Document.SaveAs** method provides an optional parameter that lets you access the **StructSaveAsOptions** class to specify additional settings.

The **Document.Save** method saves over an existing document file — that is, by using the existing file path and filename for the document.

If you want, you can use event handlers to respond to events that are triggered by opening the **Save** dialog box:

- **Application.DocumentBeforeSave**
- **Document.BeforeSave**
- **GlobalMacroStorage.DocumentBeforeSave**

You can also use event handlers to respond to events that are triggered by saving a document and closing the **Save** dialog box:

- **Application.DocumentAfterSave**
- **Document.AfterSave**
- **GlobalMacroStorage.DocumentAfterSave**

Finally, you can also use event handlers to respond to events that are triggered when the user responds to a request to save a document:

- **Application.QueryDocumentSave**
- **Document.QuerySave**
- **GlobalMacroStorage.QueryDocumentSave**

Copyright 2014 Corel Corporation. All rights reserved.

Searching for shapes

In your macros, you can include queries that search for shapes that have specific shape properties, fill properties, outline properties, or color properties. To do this, you use Corel Query Language (CQL) in conjunction with one of the following methods:

- **Shape.Evaluate** — returns the result of a given expression that evaluates the properties of the current shape
- **Shapes.FindShape** — returns a single shape that has the specified properties
- **Shapes.FindShapes** — returns, as a shape range, all shapes that have the specified properties

You can specify the shape properties for which to search. For example, the expression ActiveShape.Evaluate("@name") searches searches the **Name** property of all selected shapes.

Consider the following VBA code sample, in which the **Type** property and the **Width** property are used to select all rectangles that are wider than two inches:

```
ActivePage.Shapes.FindShapes(Query := "@type = 'rectangle' and _
@width > {2 in}").CreateSelection
```

For comprehensive information on using CQL, see Including queries in macros.

Copyright 2014 Corel Corporation. All rights reserved.

Selecting shapes

Each shape is a member of the **Layer.Shapes** collection for the layer on which it appears. The shapes in a **Layer.Shapes** collection appear in the order in which they appear on that layer — the first shape is the one at the top of the "stack," and the last shape is the one at the bottom. If shapes are added, reordered, or deleted, the affected **Layer.Shapes** collection is immediately updated to reflect the new shape order of that layer.

- In addition, each document page has a **Shapes** collection, which contains all **Layer.Shapes** collections for that page. The first shape in a **Page.Shapes** collection is the one at the very top of that page, and the last shape is the one at the very bottom.

If you want to access individual shapes, you can select them. When you select shapes, you create a "selection" that contains only those shapes.

The **Shape.Selected** property takes a Boolean value that indicates whether a shape is selected: **True** if the shape is selected, **False** otherwise. You can select a shape by changing the value of its **Selected** property to **True**; this technique adds the shape to the current selection — that is, rather than creating a new selection that contains only that shape.

If you want to create a new selection from a shape — that is, by selecting a specified shape and deselecting any other selected shapes — you can use the **Shape.CreateSelection** method, as in the following VBA example:

```
Dim sh As Shape
Set sh = ActivePage.Shapes(1)
If sh.Selected = False Then sh.CreateSelection
```

You can select multiple shapes by using the **ShapeRange.CreateSelection** method. The following VBA code uses this method — in combination with the **Shapes.All** method — to select all shapes on the active page (except those on locked or hidden layers):

```
ActivePage.Shapes.All.CreateSelection
```

You can access a selection in one of two ways:

- Use the **Document.Selection** method to return a special **Shape** object that contains the actual selection. This **Shape** object is automatically refreshed when the selection is updated.
- Use the **Document.SelectionRange** property to return a **ShapeRange** object that contains a copy of the selection. This **ShapeRange** object represents a "snapshot" of the selection (at the time when the **ShapeRange** object was created), so it is not automatically refreshed when the selection is updated.

To summarize, you can access a selection directly, or you can access a copy of that selection; alternatively, you can access a subset of the shapes in a selection. You can also reorder the shapes in a selection. When you no longer require a selection, you can deselect one or all of its shapes.

For more information on selecting shapes, see the following subtopics:

- Accessing selections directly
- Accessing copies of selections
- Accessing the shapes in a selection
- Reordering the shapes in a selection
- Deselecting shapes

Accessing selections directly

As previously discussed, you can use the **Document.Selection** method to access the contents of a selection directly. A **Shape** object is returned, and this **Shape** object is updated to reflect any changes made to the selection.

The following VBA code returns the selection for the active document:

```
Dim sel As Shape
Set sel = ActiveDocument.Selection
```

The shortcut for `ActiveDocument.Selection` is `ActiveSelection`, which returns a **Shape** object of subtype **cdrSelectionShape**. The **Shape** subtype has a member collection called **Shapes**, which represents a collection of all the selected shapes in the document. The shapes in the `ActiveSelection.Shapes` collection can be accessed as in the following VBA example:

```
Dim sh As Shape, shs As Shapes
Set shs = ActiveSelection.Shapes
For Each sh In shs
sh.Rotate 15 'Rotate each shape by 15 degrees counterclockwise
Next sh
```

- After you use the `ActiveSelection` command to select shapes, you cannot subsequently use the command to access those shapes. Instead, you must create a copy of the selection by using one of the following methods:
  - Recreate the selection as an array of **Shape** objects.
  - Recreate the selection as a **Shapes** collection.
  - Create a "snapshot" of the selection as a **ShapeRange** object (see Accessing copies of selections).

Accessing copies of selections

As previously discussed, you can use the **Document.SelectionRange** property to make a copy of the shapes in a selection. However, the returned **ShapeRange** object is not refreshed when the selection is updated because it represents a "snapshot" of the selection at the moment when that

ShapeRange object was created.

The following VBA code returns a copy of the selection for the active document:

```
Dim selRange As ShapeRange
Set selRange = ActiveDocument.SelectionRange
```

The shortcut for the `ActiveDocument.SelectionRange` command is `ActiveSelectionRange`, which returns a **ShapeRange** object. This object contains a collection of references to the shapes that were selected at the moment when the property was invoked. The shapes in the `ActiveSelectionRange` collection can be accessed as in the following VBA example:

```
Dim sh As Shape, shRange As ShapeRange
Set shRange = ActiveSelectionRange
For Each sh In shRange
sh.Skew 15 ' Skew each shape thru 15° counterclockwise
Next sh
```

After you use the `ActiveSelectionRange` command to create a copy of the current document selection, you can subsequently access the returned **ShapeRange** object to access any of its shapes. You can even add shapes to or remove shapes from the returned **ShapeRange** object. You can then use the **ShapeRange.CreateSelection** method if you want to replace the current selection with the modified **ShapeRange** object.

The following VBA code creates a **ShapeRange** object from the current document selection, removes the first and second shapes from that shape range, and then replaces the original selection with this modified **ShapeRange** object:

```
Dim shRange As ShapeRange
Set shRange = ActiveSelectionRange
shRange.Remove 1
shRange.Remove 2
shRange.CreateSelection
```

- If you want to add a specified **ShapeRange** object to the current selection (rather than use it to replace the current selection), you can use the **ShapeRange.AddToSelection** method.

Accessing the shapes in a selection

You can use a similar process for accessing the shapes in a selection as you can for accessing the shapes in a selection range.

Here is a a VBA code sample for accessing the shapes in a selection:

```
Dim shs As Shapes, sh As Shape
Set shs = ActiveSelection.Shapes
For Each sh In shs
' Do something with the shape, sh
Next sh
```

Here is a VBA code sample for accessing the shapes in a selection range:

```
Dim sRange As ShapeRange, sh As Shape
Set sRange = ActiveSelectionRange
For Each sh In sRange
' Do something with the shape, sh
Next sh
```

- Remember that the `ActiveSelection.Shapes` command provides direct access to the current selection, while the `ActiveSelectionRange` command provides a copy of the current selection. Use `ActiveSelection.Shapes` if you want to access the current selection; use `ActiveSelectionRange` if you want to create a “snapshot” of the current selection that you can access later.

Reordering the shapes in a selection

The `ActiveSelection.Shapes` command and the `ActiveSelectionRange` command return shapes in the reverse order from which they were selected: the first shape is the last one selected, and the last shape is the first one selected. Please keep this fact in mind when reordering the shapes in a selection.

Deselecting shapes

You can deselect any shape by changing the value of its **Shape.Selected** property to **False**.

You can deselect all shapes by using the **Document.ClearSelection** method. The following VBA code uses the ClearSelection method to deselect all shapes in the active document:

```
ActiveDocument.ClearSelection
```

If you want, you can use event handlers to respond to events that are triggered by deseleting a shape:

- **Document.ShapeChange**

You can also use event handlers to respond to events that are triggered by deactivating a selection:

- **Application.SelectionChange**
- **Document.SelectionChange**
- **GlobalMacroStorage.SelectionChange**

Copyright 2014 Corel Corporation. All rights reserved.

Setting document properties

You can specify the reference point, unit of measurement, and drawing scale for a document by using the corresponding properties of the **Document** class.

The **Document.ReferencePoint** property specifies the reference point for a document. This point is referenced when positioning the objects in that document.

The **Document.Unit** property specifies the unit of measurement for a document. This unit is used to position and size the objects in that document.

The **Document.WorldScale** property specifies the drawing scale for a document. The drawing scale lets you make the distances in a drawing proportionate to real-world distances; for example, you can specify that 1 inch in the drawing corresponds to 1 meter in the physical world.

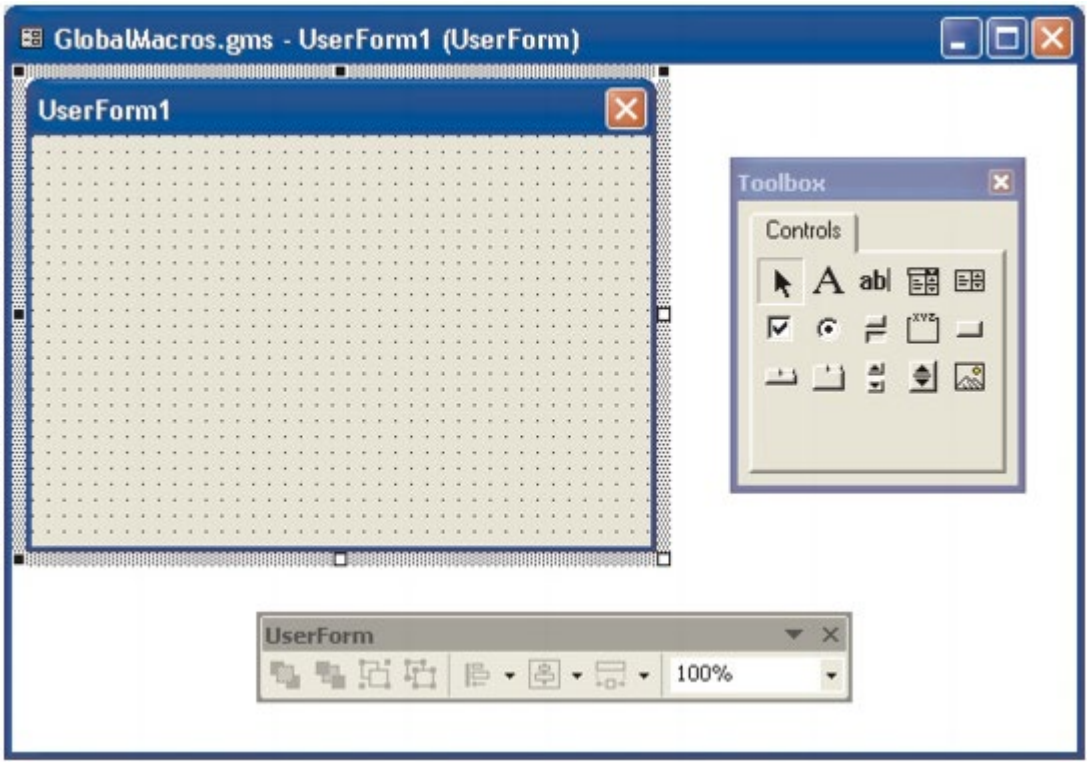
- These properties affect all elements in your document, such as the objects that you draw. For optimal results, choose the settings that best fit your macro solution.

Copyright 2014 Corel Corporation. All rights reserved.

Setting up dialog boxes

The Form Designer in the Macro Editor provides easy access to the tools for setting up a dialog box.

You can access the Form Designer by creating a new, blank dialog box. In the Project Explorer, right-click the project to which you want to add a dialog box, and then click **Insert** } **UserForm**.



A blank form in the Form Designer

The Form Designer provides two main features for designing dialog boxes:

- the **Form Designer** toolbox
- The **UserForm** toolbar

The Form Designer also provides functions for naming and testing dialog boxes.

Designing dialog boxes

The **Form Designer** toolbox is the main utility for designing dialog boxes. It lets you add controls to a dialog box by dragging them from the toolbox.

*The **Form Designer** toolbox*

The **Form Designer** toolbox lets you add the following controls to a dialog box:

Icon	Control name	Function
	Label	Provides static text, such as instructions or captions

TextBox	Provides an area for typing text. For information on coding this control, see Providing text boxes in dialog boxes.
ComboBox	Provides a list from which a single item can be selected; optionally, also provides an area for typing text. For information on coding this control, see Providing combination boxes and list boxes in dialog boxes.
ListBox	Provides a list from which multiple items can be selected. For information on coding this control, see Providing combination boxes and list boxes in dialog boxes.
CheckBox	Provides a check box that can be enabled (by clicking to display a check mark), disabled (by clicking to remove the check mark), or grayed (that is, made unavailable)
OptionButton	Provides a “radio button” that is linked to other radio buttons with the same **GroupName** property, such that only one of the buttons can be enabled at a time
ToggleButton	Provides a button that can be toggled (to appear pressed or not pressed)
Frame	Groups items together so that they move with the frame
CommandButton	Provides a button that can be clicked to commit an assigned action. For information on coding this control, see Providing buttons in dialog boxes.
TabStrip	Provides separate views of related controls
MultiPage	Provides multiple pages of controls
ScrollBar	Provides immediate access to a range of values by scrolling
SpinButton	Enhances another control (such as a **TextBox** control) so that the value for that control can be set more quickly
Image	Provides an image. For information on coding this control, see Providing images in dialog boxes.

- The **Form Designer** toolbox also features a **Pick** tool , which lets you select and move the controls on a dialog box.

- To display a Help topic containing information about a selected dialog-box control in the Form Designer, press **F1**.

The Form Designer also provides access to the **UserForm** toolbar, which you can use when designing a dialog box. For information on this toolbar, see Using the Macro Editor toolbars.

Naming dialog boxes

After you have finished designing your dialog box, you may want to change its title. Click the dialog box to select it, and then in the **Properties** window, change the **Caption** property.

- For clarity, you can give each dialog box a unique, descriptive name by using the **Properties** window. However, remember to follow the standard

programming conventions for naming variables.

Testing dialog boxes

At any time, you can test your dialog box by pressing **F5** to run it.

After you finish setting up your dialog box, you can start coding it. For information, see Coding dialog boxes.

Copyright 2014 Corel Corporation. All rights reserved.

Setting up the automation feature

Before you can develop and run macros in the software, you may need to set up the automation features for VBA and VSTA.

When you perform a “typical installation” of the software, the VBA and VSTA features are installed by default. However, you can manually install them if necessary. For example, if you want to edit VSTA projects, and you installed Microsoft Visual Studio 2012 after installing CorelDRAW Graphics Suite, you must reinstall the VBA and VSTA features manually.

You can also specify various options for VBA.

For details on setting up the automation feature, see the following procedures:

- To install the VBA and VSTA features
- To specify VBA options

Copyright 2014 Corel Corporation. All rights reserved.

Sizing pages

You can specify the size and orientation of pages, specifying the default page size, and use defined page sizes.

Specifying the size and orientation of pages

You can size a page by using its **SetSize** method, which applies two size values (width and height) to the page.

The following VBA code changes the size of the active page in the active document to A4:

```
ActiveDocument.Unit = cdrMillimeter
ActivePage.SetSize 210, 297
ActivePage.Orientation = cdrLandscape
```

For the **SetSize** method, the first number is always the page width and the second number is always the page height. Reversing the two numbers switches the orientation of the page.

Specifying the default page size

The default page size for a document is determined by the value of the item that has an index of **0** in the **Document.Pages** collection. You can specify the default page size by changing the value of this item:

```
Dim doc As Document
Set doc = ActiveDocument
doc.Unit = cdrMillimeter
doc.Pages(0).SetSize 297, 210
```

Alternatively, you can use the **Document.MasterPage** property to specify the default page size:

```
Dim doc As Document
Set doc = ActiveDocument
doc.Unit = cdrMillimeter
doc.MasterPage.SetSize 297, 210
```

Using defined page sizes

Page sizes can be defined by either the application or the user. All defined page sizes are stored in the **Application.PageSizes** collection, and the name of each **PageSize** object in that collection is defined by its **Name** property:

```
Dim pageSizeName As String
pageSizeName = Application.PageSizes(3).Name
```

Page sizes can be specified by name. For example, the following VBA code gets the **PageSize** object named “Business Card”:

```
Dim thisSize As PageSize
Set thisSize = Application.PageSizes("Business Card")
```

You can get the actual dimensions of a **PageSize** object by using its **Width** and **Height** properties. The following VBA code retrieves the width and height (in millimeters) of the third **PageSize** object:

```
Dim pageWidth As Double, pageHeight As Double
Application.Unit = cdrMillimeter
pageWidth = Application.PageSizes(3).Width
pageHeight = Application.PageSizes(3).Height
```

Although each **PageSize** object provides a **Delete** method, this method can be used only on user-defined page sizes. You can determine whether a **PageSize** object is user-defined by testing its **BuiltIn** Boolean property:

```
Public Sub deletePageSize(thisSize As PageSize)
If Not thisSize.BuiltIn Then thisSize.Delete
End Sub
```

- You can specify a particular unit of measurement for a page size by setting the units for the document before getting its width and height.

Copyright 2014 Corel Corporation. All rights reserved.

To add a code module to a macro project

- Do one of the following:
  - In the **Macro Manager** docker, click the project in the list, click **New**, and then click **New module**.
  - In the **Macro Manager** docker, right-click the project in the list, and then click **New module**.
  - In the Project Explorer of the Macro Editor, right-click the project, and then click **Insert** } **Module**.

You can also

Display or hide all code modules in the **Macro Manager** docker	In the **Macro Manager** docker, click the **Simple mode** button .
Edit a code module	In the **Macro Manager** docker, do one of the following: • Click the module in the list, and then click the **Edit** button . • Right-click the module in the list, and then click **Edit**. The code module opens in the Macro Editor.
Rename a code module	In the **Macro Manager** docker, right-click the module in the list, and then click **Rename**.
Delete a code module	In the **Macro Manager** docker, do one of the following: • Click the module in the list, and then click the **Delete** button . • Right-click the module in the list, and then click **Delete**.

- Some macro projects are locked and cannot be modified.

Copyright 2014 Corel Corporation. All rights reserved.

To add a class module to a macro project

1. In the Project Explorer of the Macro Editor, right-click the project.
2. Click **Insert** } **Class Module**.

   A new class module is added to the **Class Modules** folder for the project.

- Some macro projects are locked and cannot be modified.
- Detailed documentation on creating class modules is beyond the scope of this documentation.

Copyright 2014 Corel Corporation. All rights reserved.

To add a dialog box to a macro project

1. In the Project Explorer of the Macro Editor, right-click the project.
2. Click **Insert** } **UserForm**.

   A form is added to the **Forms** folder for the project.

- Some macro projects are locked and cannot be modified.

- For more information, see Providing dialog boxes for macros.

Copyright 2014 Corel Corporation. All rights reserved.

To add a macro to a macro project

- In the **Macro Manager** docker, do one of the following:
  - Click the desired container module in the macro project, click **New**, and then click **New macro**.
  - Right-click the desired container module in the macro project, and then click **New macro**.

- Some macro projects are locked and cannot be modified.

Copyright 2014 Corel Corporation. All rights reserved.

To add buttons to a macro toolbar

1. Click **Workspace** } **Customization** } **Commands**.
2. Choose **Macros** from the **Commands** list box.
   The list displays the fully qualified names of all of the public, parameter-free subroutines from all of the installed project (GMS) files.
3. Drag a macro from the list to the toolbar.
   The macro appears on the toolbar with the default macro icon.

Copyright 2014 Corel Corporation. All rights reserved.

To associate an image or icon with a macro

1. Click **Workspace** } **Customization** } **Commands**.
2. Choose **Macros** from the **Commands** list box.

   The list displays the fully qualified names of all of the public, parameter-free subroutines from all of the installed project (GMS) files.
3. Select a macro in the **Command** list.
4. Click the **Appearance** tab, and then do one of the following:
   - To apply a Windows bitmap (BMP) image to the macro, click **Import**, navigate to where the image is stored, and select it. The colors in the image are mapped to their closest available matches.
   - To create a customized icon for the macro, edit the pixels displayed in the **Image** area.

Copyright 2014 Corel Corporation. All rights reserved.

To create a macro project

- In the **Macro Manager** docker, do one of the following:
  - Click **Visual Studio Tools for Applications** in the list, click **New**, and then click **New macro project**.
  - Click **Visual Basic for Applications** in the list, click **New**, and then click **New macro project**.

- Project names must follow normal variable-naming conventions: They must begin with an alphabetic character, and they must not contain spaces nor special characters other than underscores (_).

You can also

Load a macro project	Do one of the following: • Click **Visual Studio Tools for Applications** in the list, click **Load**, and then choose the project. • Click **Visual Basic for Applications** in the list, click **Load**, and then choose the project.
Rename a macro project	Right-click the project in the list, and then click **Rename**. You can also rename a macro project from within the Macro Editor. Click the project in the Project Explorer, and then edit the **(Name)** value in the **Properties** window. Press **Enter** to commit your changes.
Copy a GMS-based macro project	Right-click the project in the list, click **Copy to**, and then choose the target location for the copy. **NOTE:** You cannot copy a document-based macro project. Such projects are stored within a document and cannot be managed separately from that document.
Unload a GMS-based macro project	Right-click the macro project in the list, and then click **Unload macro project**. **NOTE:** You can close a document-based macro project only by closing the document in which it is stored.

- Some macro projects are locked and cannot be modified.

Copyright 2014 Corel Corporation. All rights reserved.

To create a macro toolbar

1. Click **Tools** } **Options**.
2. Click **Workspace** } **Customization** } **Command bars**.
3. Click New.
4. Type a name for the toolbar.
5. Enable the check box next to the name of the toolbar.

Copyright 2014 Corel Corporation. All rights reserved.

To delete a VBA macro

- In the **Macro Manager** docker, do one of the following:
  - Click the macro in the list, and then click the **Delete** button .
  - Right-click the macro in the list, and then click **Delete**.

- Some macro projects are locked and cannot be modified.

Copyright 2014 Corel Corporation. All rights reserved.

To edit a VBA macro

- In the **Macro Manager** docker, do one of the following:
  - Click the macro in the list, and then click the **Edit** button .
  - Right-click the macro in the list, and then click **Edit**.

  The macro opens in the Macro Editor.

- Some macro projects are locked and cannot be modified.

- For detailed information on manually coding macros, see How is automation coding structured?.

Copyright 2014 Corel Corporation. All rights reserved.

To export a GMS file

- Locate the GMS file on your computer, and make it available to your users.

  For help locating this folder, see Using GMS files.

Copyright 2014 Corel Corporation. All rights reserved.

To export workspace features

1. Right-click the menu bar, and click **Customize** } **Workspace** } **Export Workspaces**.
2. In the list, enable the check boxes next to the workspace features that you want to export:
   - **Dockers** — includes the sizes and positions of dockers
   - **Menus** — lets you choose which menus to include
   - **Shortcut Keys** — includes all available shortcut keys
   - **Status Bar** — includes the status bar
   - **Toolbars** — lets you choose which toolbars to include

   Enable all check boxes if you want to export the entire workspace.
3. Click **Save**.
4. In the **File name** box, type a filename.

   The specified workspace features are saved to a single Corel workspace (XSLT) file with the specified filename.

- If you want, you can export each workspace feature to a separate file. Simply export one item at a time to create a series of XSLT files.
- When you export shortcut keys, you export all shortcut keys. If you want to distribute only a few keys, create a new workspace, remove all shortcut keys from it, and then add only the desired keys.

Copyright 2014 Corel Corporation. All rights reserved.

To import a GMS file

1. Save the GMS file to a **GMS** folder on your computer.
   For help locating this folder, see Using GMS files.
2. In the **Macro Manager** docker, do one of the following:
   - Click **Visual Basic for Applications** in the list, click **Load**, and then choose the project.
   - Right-click **Visual Basic for Applications** in the list, and then click **Load macro project**.

Copyright 2014 Corel Corporation. All rights reserved.

To import workspace features

1. Right-click the toolbar, and click **Customize** } **Workspace** } **Import Workspaces**.
2. Click **Browse**.
3. Select the desired Corel workspace (XSLT) file, and then click **Next**.
4. In the list, enable the check boxes next to the workspace features that you want to import:
   - **Dockers** — includes the sizes and positions of dockers
   - **Menus** — lets you choose which menus to include
   - **Shortcut Keys** — includes all available shortcut keys
   - **Status Bar** — includes the status bar
   - **Toolbars** — lets you choose which toolbars to include

   Enable all check boxes if you want to import the entire workspace.
5. Click **Next**.
6. Choose a destination for the workspace features by doing one of the following:
   - Enable the **Current workspace** option to import the specified workspace features into the current workspace, and then click **Next**.
   - Enable the **New workspace** option to import the specified workspace features into a new workspace. Click **Next**, and provide details about the workspace. Click **Next**.
7. Confirm the details of the import, and then click **Finish**.

   The specified workspace features are imported into the specified workspace.

- If the name of an incoming toolbar is the same as an existing toolbar, the incoming toolbar is renamed.
- If an imported command calls an uninstalled macro, it does not function.

Copyright 2014 Corel Corporation. All rights reserved.

To install the VBA and VSTA features

1. On the Windows Control Panel, click **Uninstall a program**.
2. Double-click on the **Uninstall or change a program** page.
3. Enable the **Modify** option in the wizard that appears, and follow the instructions.
4. On the **Features** page (page 2) of the installation wizard, enable the **Utilities** check box, and then enable the following check boxes in the **Includes** list box:
   - **Visual Basic for Applications**
   - **Visual Studio Tools for Applications**

Copyright 2014 Corel Corporation. All rights reserved.

To record a temporary macro

1. Do one of the following:
   - Click **Tools** } **Macros** } **Record temporary macro**.
   - Press **Ctrl + Shift + R**.
2. Perform the actions that you want to record.

   The application begins recording your actions. If you want to pause recording, do one of the following:
   - Click **Tools** } **Macros** } **Pause recording**. Repeat this step to resume recording.
   - Click the **Pause recording** button on the **Macros** toolbar or in the **Macro Manager** docker. Repeat this step to resume recording.
3. To stop recording, do one of the following:
   - Click **Tools** } **Macros** } **Stop recording**.
   - Click the **Stop recording** button on the **Macros** toolbar or in the **Macro Manager** docker.
   - Press **Ctrl + Shift + O**.

   The macro is temporarily saved to the default recording project. When the current session is ended, the macro is deleted from that project.

- You cannot record a temporary macro if all available macro projects are locked.
- Not all actions can be recorded.

- You can specify the default recording project by right-clicking the project in the **Macro Manager** docker, and then clicking **Set as recording project**. (You cannot specify a locked macro project.) If you want, you can create multiple temporary recordings by assigning each one to its own macro project.
- You can cancel recording a macro, and discard any commands recorded thus far, by clicking **Tools** } **Macros** } **Cancel recording**.

Copyright 2014 Corel Corporation. All rights reserved.

To record and save a macro

1. Do one of the following:
   - Click **Tools** } **Macros** } **Start recording**, or click the **Start recording** button on the **Macros** toolbar, to store the macro in the default macro project for recordings.
   - In the **Macro Manager** docker, click the project in which to store the macro, and then click the **Record** button .

   The **Record Macro** dialog box appears.
2. In the **Macro name** box, type a name for the macro.

   Macro names can contain numerals, but they must begin with a letter. Macro names cannot contain spaces or non-alphanumeric characters other than underscores (_).
3. Type a description of the macro in the **Description** box, and then click **OK**.
4. Perform the actions that you want to record.

   The application begins recording your actions. If you want to pause recording, do one of the following:
   - Click **Tools** } **Macros** } **Pause recording**. Repeat this step to resume recording.
   - Click the **Pause recording** button on the **Macros** toolbar or in the **Macro Manager** docker. Repeat this step to resume recording.
5. To stop recording, do one of the following:
   - Click **Tools** } **Macros** } **Stop recording**.
   - Click the **Stop recording** button on the **Macros** toolbar or in the **Macro Manager** docker.
   - Press **Ctrl + Shift + O**.

- You cannot record a macro if all available macro projects are locked..
- Not all actions can be recorded — some because of their complexity (although many such actions can be manually coded in the Macro Editor). When an action cannot be recorded, a comment is placed in the macro code ("`The recording of this command is not supported`."), but the recording process continues until you stop it. You can view any comments in the code by opening the macro in the Macro Editor.

- You can specify the default macro project for recordings by right-clicking the project in the **Macro Manager** docker, and then clicking **Set as recording project**. However, you cannot specify a locked macro project.
- You can cancel recording a macro, and discard any commands recorded thus far, by clicking **Tools** } **Macros** } **Cancel recording**.

You can also

Save the actions in the **Undo** list as a VBA macro	Click **Tools** } **Undo**, perform the actions that you want to record, and then click the **Save list to a VBA macro** button in the **Undo** docker.

Copyright 2014 Corel Corporation. All rights reserved.

To run a saved macro

- Do one of the following:
  - Click **Tools** } **Macros** } **Run macro**, or click the **Run macro** button on the **Macros** toolbar. From the **Macros in** list box, choose the project or file in which the macro is stored. From the **Macro name** list, choose the macro. Click **Run**.
  - In the **Macro Manager** docker, double-click the macro in the list.
  - In the **Macro Manager** docker, click the macro in the list, and then click the **Run** button .
  - In the **Macro Manager** docker, right-click the macro in the list, and then click **Run**.
  - In the Macro Editor, click anywhere in the subroutine that forms the macro, and then click **Run** } **Run macro**.

Copyright 2014 Corel Corporation. All rights reserved.

To run a temporary macro

- Do one of the following:
  - Click **Tools** } **Macros** } **Run temporary macro**.
  - Press **Ctrl + Shift + P**.

- This option is enabled only after you record a temporary macro.

- If you have created multiple temporary macros, you must specify which macro project contains the one that you want to run. Right-click the project in the **Macro Manager** docker, and then click **Set as recording project**.

Copyright 2014 Corel Corporation. All rights reserved.

To set a caption for a macro

1. Click **Workspace** } **Customization** } **Commands**.
2. Choose **Macros** from the Commands list box.

   The list displays the fully qualified names of all of the public, parameter-free subroutines from all of the installed project (GMS) files.
3. Select a macro in the **Command** list.
4. Click the **Appearance** tab, and then type the caption in the **Caption** box.

   To specify a character in the caption as an accelerator that can be activated in combination with the **Alt** key, type an ampersand (**&**) in front of that character. This accelerator key applies only to menu commands, which display accelerator characters with an underscore (**_**).

Copyright 2014 Corel Corporation. All rights reserved.

To set a tooltip for a macro

1. Click **Workspace** } **Customization** } **Commands**.
2. Choose **Macros** from the **Commands** list box.
   The list displays the fully qualified names of all of the public, parameter-free subroutines from all of the installed project (GMS) files.
3. Select a macro in the **Command** list.
4. Click the **General** tab, and then type the tooltip in the **Tooltip help** box.

Copyright 2014 Corel Corporation. All rights reserved.

To specify VBA options

1. Click **Tools** } **Options**.
2. In the **Workspace** list of categories, click **VBA**.
3. In the **Security** area, specify how to control the risk of running malicious macros by clicking **Security options**.

   If you want to bypass this security feature, enable the **Trust all installed GMS modules** check box, and then proceed to step 6.
4. On the **Security level** page of the **Security** dialog box, enable one of the following options:
   - **Very high** — allows only macros installed in trusted locations to run. All other signed and unsigned macros are disabled.
   - **High** — allows only signed macros from trusted sources to run. Unsigned macros are automatically disabled.
   - **Medium** — lets you choose which macros run, even if they are potentially harmful.
   - **Low (not recommended)** — allows all potentially unsafe macros to run. Enable this setting if you have virus-scanning software installed, or if you check the safety of all documents that you open.
5. On the **Trusted publishers** page of the **Security** dialog box, review which macro publishers are trusted. Click **View** to display details on the selected macro publisher, or click **Remove** to delete the selected macro publisher from the list.

   If desired, you can enable or disable the **Trust access to Visual Basic project** check box for the selected macro publisher.
6. Disable the **Delay load VBA** check box if you want to load the VBA feature at start-up.

Copyright 2014 Corel Corporation. All rights reserved.

Transforming shapes

You can transform shapes in various ways, as explained in the following topics:

- Sizing shapes
- Stretching shapes
- Skewing shapes
- Rotating shapes
- Positioning shapes

If you want, you can use event handlers to respond to events that are triggered by transforming a shape:

- **Document.ShapeTransform**

Sizing shapes

You can return the width and height of a shape (in document units) by using the **Shape.SizeWidth** and **Shape.SizeHeight** properties, as in the following VBA example:

```
Dim width As Double, height As Double
ActiveDocument.Unit = cdrMillimeter
width = ActiveShape.SizeWidth
height = ActiveShape.SizeHeight
```

You can also use the **Shape.SizeWidth** and **Shape.SizeHeight** properties to resize an existing shape by specifying new values for those properties. The following VBA example uses these properties to set the size of the active shape to a width of 50 millimeters and a height of 70 millimeters:

```
ActiveDocument.Unit = cdrMillimeter
ActiveShape.SizeWidth = 50
ActiveShape.SizeHeight = 70
```

You can return both the width and the height of a shape (in document units) by using the **Shape.GetSize** method, as in the following VBA example:

```
Dim width As Double, height As Double
ActiveDocument.Unit = cdrMillimeter
ActiveShape.GetSize width, height
```

You can resize a shape by using the **Shape.SetSize** method to specify a new width and new height for it, as in the following VBA example:

```
ActiveDocument.Unit = cdrMillimeter
ActiveShape.SetSize 50, 70
```

You can also resize a shape by using the **Shape.SetSizeEx** method. Besides the new width and new height for the shape, this method takes a reference point for the resize (instead of using the center point of the shape). The following VBA code uses the **SetSizeEx** method to resize the current selection to 10 inches wide by 8 inches high about the reference point (6, 5) in the document:

```
ActiveDocument.Unit = cdrInch
ActiveSelection.SetSizeEx 6, 5, 10, 8
```

If you want to take the outline of a shape into account when returning the size of that shape, you must use the **Shape.GetBoundingBox** method. The bounding box for a shape surrounds both the shape and its outline; however, the actual dimensions of a shape specify its width and height irrespective of the size of its outline. The following VBA example uses the **GetBoundingBox** method to return the size of the active shape:

```
Dim width As Double, height As Double
Dim posX As Double, posY As Double
ActiveDocument.Unit = cdrInch
ActiveDocument.ReferencePoint = cdrBottomLeft
ActiveShape.GetBoundingBox posX, posY, width, height, True
```

The **Shape.GetBoundingBox** method takes parameters that specify the position of the lower-left corner of the shape, the width of the shape, and the height of the shape. The final parameter is a Boolean value that indicates whether to include (**True**) or exclude (**False**) the outline of the shape. The **Shape.SetBoundingBox** method lets you set the size of a shape by specifying the size of its bounding box; however, this method lacks the parameter for specifying whether to include the outline in the new size. If you want to calculate the size and position of the bounding box of a shape without including its outline, you can use the **GetBoundingBox** method twice (once including the outline and once excluding it), as in the following VBA example:

```
Public Sub SetBoundingBoxEx(X As Double, Y As Double, _
Width As Double, Height As Double)
Dim sh As Shape
Dim nowX As Double, nowY As Double
Dim nowWidth As Double, nowHeight As Double
Dim nowXol As Double, nowYol As Double
Dim nowWidthol As Double, nowHeightol As Double
Dim newX As Double, newY As Double
Dim newWidth As Double, newHeight As Double
Dim ratioWidth As Double, ratioHeight As Double
Set sh = ActiveSelection
sh.GetBoundingBox nowX, nowY, nowWidth, nowHeight, False
sh.GetBoundingBox nowXol, nowYol, nowWidthol, nowHeightol, True
ratioWidth = Width / nowWidthol
ratioHeight = Height / nowHeightol
newWidth = nowWidth * ratioWidth
newHeight = nowHeight * ratioHeight
newX = X + (nowX - nowXol)
newY = Y + (nowY - nowYol)
sh.SetBoundingBox newX, newY, newWidth, newHeight, False, _
cdrBottomLeft
End Sub
```

Stretching shapes

You can stretch a shape (or scale it by stretching is proportionately) by using the **Shape.Stretch** method or the **Shape.StretchEx** method. Both methods take a decimal value for the stretch, where 1 is 100% (or no change); you cannot use zero, so you must use a very small value instead.

The following VBA code uses the **Shape.Stretch** method to stretch the selection to half its current height and twice its width, about the midpoint of the bottom edge of its bounding box:

```
ActiveDocument.ReferencePoint = cdrBottomMiddle
ActiveSelection.Stretch 2, 0.5
```

If you want to specify the reference point about which to perform a stretch, you can use the **Shape.StretchEx** method. The following VBA code performs the same stretch as the previous code, but it performs that stretch about the point (4, 5) on the page (in inches):

```
ActiveDocument.Unit = cdrInch
ActiveSelection.StretchEx 4, 5, 2, 0.5
```

- The **Shape.Stretch** and **Shape.StretchEx** methods provide an optional Boolean parameter that determines how to stretch paragraph text. A value of **True** stretches the characters, while **False** stretches the bounding box and re-flows the text within it.

Skewing shapes

You can skew a shape by using the **Shape.Skew** method or the **Shape.SkewEx** method. These methods let you specify the horizontal-skew angle (in degrees, where positive values move the top edge to the left and the bottom edge to the right) and the vertical-skew angle (in degrees, where positive values move the right edge upwards and the left edge downwards).

- Skews of angles close to or greater than 90° are not allowed.
- The horizontal skew is applied before the vertical skew.

The difference between the **Shape.Skew** and **Shape.SkewEx** methods is the point about which the skew is performed: **Skew** uses the center of rotation for the shape, while **SkewEx** uses the specified reference point.

- You can determine the center of rotation for a shape by returning the values of its **Shape.RotationCenterX** and **Shape.RotationCenterY** properties. Changing these values moves the center of rotation for that shape.

The following VBA code uses the **Shape.Skew** method to skew the selection (about its center of rotation) by 30° horizontally and by 15° vertically:

```
ActiveSelection.Skew 30, 15
```

Rotating shapes

You can rotate a shape by using the **Shape.Rotate** method or the **Shape.RotateEx** method. These methods rotate the shape by the given angle (in degrees). However, the difference between these methods is the point about which they perform the rotation: **Rotate** uses the center of rotation for the shape, while **RotateEx** uses the specified reference point.

- You can determine the center of rotation for a shape by returning the values of its **Shape.RotationCenterX** and **Shape.RotationCenterY** properties. Changing these values moves the center of rotation for that shape.

The following VBA code uses the **Shape.Rotate** method to rotate the selection (about its center of rotation) by 30°:

```
ActiveSelection.Rotate 30
```

The following VBA code uses the **Shape.RotateEx** method to rotate each selected shape by 15° clockwise about its lower-right corner:

```
Dim sh As Shape
ActiveDocument.ReferencePoint = cdrBottomRight
For Each sh In ActiveSelection.Shapes
sh.RotateEx -15, sh.PositionX, sh.PositionY
Next sh
```

Positioning shapes

You can return the horizontal and vertical position of a shape by using the **Shape.PositionX** and **Shape.PositionY** properties (respectively). Alternatively, you can use the **Shape.GetPosition** method to return both the horizontal position and the vertical position of a shape.

- You can use the **Shape.GetBoundingBox** method if you want to return the position of a shape, including its outline. For more information on this method, see Sizing shapes.

The following VBA code uses the **Shape.GetPosition** method to return the position of the selection relative to the current reference point of the active document, which the code explicitly sets to the lower-left corner:

```
Dim posX As Double, posY As Double
ActiveDocument.ReferencePoint = cdrBottomLeft
ActiveSelection.GetPosition posX, posY
```

You can also use the **Shape.PositionX** and **Shape.PositionY** properties to set the horizontal and vertical position of a shape (respectively), thereby moving that shape to the specified position. Alternatively, you can use the **Shape.SetPosition** method to move a shape to specified horizontal and vertical position, or you can use the **Shape.SetPositionEx** method to move the shape to a specified point.

- You can also use the **Shape.SetSizeEx** and **Shape.SetBoundingBox** methods to set the position of a shape. For more information on these methods, see Sizing shapes.

The following VBA code uses the **Shape.SetPosition** method to set the position of the lower-right corner of each selected shape in the active document to (3, 2) in inches:

```
Dim sh As Shape
ActiveDocument.Unit = cdrInch
ActiveDocument.ReferencePoint = cdrBottomRight
For Each sh In ActiveSelection.Shapes
sh.SetPosition 3, 2
Next sh
```

If you want, you can use event handlers to respond to events that are triggered by positioning a shape:

- **Document.ShapeMove**

Copyright 2014 Corel Corporation. All rights reserved.

C

The types in this section begin with the letter *C*:

- CurveElement

Copyright 2014 Corel Corporation. All rights reserved.

Types

The CorelDRAW object model contains several types, which are arranged alphabetically in this section.

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Copyright 2014 Corel Corporation. All rights reserved.

Understanding automation

Before you begin to work with macros, you need to understand the concept of automation. This section provides basic information about automation and about the macro-programming formats that are supported by the software.

In this section

This section contains the following topics:

- What is automation?
- Which automation environments are supported?
- What are the main elements of automation?
- How is automation coding structured?

Copyright 2014 Corel Corporation. All rights reserved.

Understanding CQL

Corel Query Language (CQL) is a simple, object-oriented, extensible language that is used to evaluate expressions. You can use CQL to include queries in macros — for example, to search for objects in a document.

You can query any of the following object types:

- Boolean objects — values of **True** or **False** that are returned by comparison operators such as <, >, <>, or =
- integers — values of **-2147483648** to **2147483647**
- double, or floating-point, objects — values of approximately **-10^308** to **10^308**
- strings — Unicode text strings that are empty or that contain one or more characters
- shapes — **Shape** objects
- fills — fill properties of **Shape** objects
- outlines — outline properties of **Shape** objects
- colors — color properties of **Shape** objects
- units of measurement — supported units for measurements such as distance, angle, and file size
- arrays — lists of objects of any supported type
- global objects
- null objects — nonexistent or missing objects

For details on the queries that you can perform on these object types, see Understanding the available query criteria.

Copyright 2014 Corel Corporation. All rights reserved.

Understanding the object-model hierarchy

The main structure of the CorelDRAW object model can be summarized as follows:

- The **Application** object contains a **Documents** collection of all open **Document** objects. When a CorelDRAW document is created or opened, a corresponding **Document** object is added to the **Documents** collection for the **Application** object.
- Each **Document** object contains a **Pages** collection of all the **Page** objects (or “pages”) in that document.
- Each **Page** object contains a **Layers** collection of all the **Layer** objects (or “layers”) on that page.
- Each **Layer** object contains a **Shapes** collection of all the **Shape** objects (or “shapes”) on that layer.

In addition, the object model contains a set of filter objects, which provide support for files from other technical-graphics applications. Import filters are governed by the **Layer** class, while export filters are governed by the **Document** class.

- For a visual representation of the object model, please see the object-model diagram at **X:\Program Files\Corel**<*folder*>**\Data** (where **X:** is the drive and ***<folder>*** is the folder where the software is installed):
  - **CorelDRAW Object Model Diagram.pdf**

Copyright 2014 Corel Corporation. All rights reserved.

Understanding the available query criteria

Each object type that can be queried (see Understanding CQL) has a set of query criteria. Some of these criteria are common to several object types, while others are unique to a single object type.

Most of the supported object types can be queried by using the following criteria:

Query criteria	Description
object.IsNull()	Returns **True** if the object is a null object
object.IsNumeric()	Returns **True** if the object can be converted to a numeric value (that is, an integer or double object)
object.This()	Returns a copy of the object
object.ToBool()	Converts the current object to a Boolean value (**True** or **False**)
object.ToDouble()	Converts the value of the object to a double object
object.ToInt()	Converts the value of the object to an integer
object.ToString()	Converts the value of the object to a string
object.TypeName()	Returns a string containing the type name of the object

For information on all the query criteria that are supported by each object type, see the following topics:

- Querying Boolean objects
- Querying integers
- Querying double objects
- Querying strings
- Querying shapes
- Querying fills
- Querying outlines
- Querying colors
- Querying units of measurement
- Querying arrays
- Querying global objects
- Querying null objects

- Each supported object type lets you use the **help** method to list all of the query criteria that are available to that object type. For example, to list the criteria that are available to the **int** object type, you can evaluate the expression int.help.

Copyright 2014 Corel Corporation. All rights reserved.

Understanding the CorelDRAW object model

In CorelDRAW, the **Application** object is the root of all other objects and is used if no other root object is specified.

Documents, pages, layers, shapes, and filters are among the most important objects in the CorelDRAW object model. Understanding these objects — and their relationships to one another — is the key to understanding the CorelDRAW object model.

In this section

This section contains the following topics:

- Understanding the object-model hierarchy
- Working with the Application object
- Working with documents
- Working with pages
- Working with layers
- Working with shapes
- Working with import filters and export filters
- Arranging open documents

- All code examples in this section are written in VBA.

- For a visual representation of the object model, please see the object-model diagram at **X:\Program Files\Corel**<*folder*>**\Data** (where **X:** is the drive and ***<folder>*** is the folder where the software is installed):
  - **CorelDRAW Object Model Diagram.pdf**

Copyright 2014 Corel Corporation. All rights reserved.

Understanding the query syntax

Each query expression yields a result object that belongs to one of the supported types (see Understanding CQL).

Here are some sample query expressions:

- `1 + 2`
- `2 * (2.3 + 1.2)`
- `'Hello' + ' world'`

Each operand in the expression is an object — even the constants such as `1` or `'Hello'`, for which the data type is determined automatically. In addition, each object has its own set of members (that is, properties and methods). You can use the following syntax to call an object member:

```
object.member(parameters)
```

Members that do not require any parameters can be called without the parenthetical content, so both of the following expressions are valid:

- `'world'.length()`
- `'world'.length`

The preceding expression evaluates to a numeric value of 5, the length of the string `'world'`.

Reducing repetition in expressions

In many cases, various properties of the same object must be inspected in an expression. To reduce repetition, a special construct involving square brackets (`[]`) can be used:

```
object[.method1 + .method2 * .method3]
```

For example, if you want to evaluate multiple fill-related properties for the current shape, you can use an expression such as the following:

```
@fill[.type = 'uniform' and .color = 'red']
```

The preceding expression is equivalent to the following expression:

```
@fill.type = 'uniform' and @fill.color = 'red'
```

Similarly, the following example returns **TESTtest**:

```
'Test'[.toUpper + .toLower]
```

Understanding the supported query operators

You can include the following operators in your queries:

Operators	Data type	Description	Example and result
+	numeric	addition	`1 + 2` Yields **3**
+	string	string concatenation	`'Lorem' + ' ipsum'` Yields **'Lorem ipsum'**
-	numeric	subtraction	`10-4` Yields **6**
*	numeric	multiplication	`2 * 5` Yields **10**
/	numeric	division	`3 / 2` Yields **1.5**
\	numeric integer	division	`3 \ 2` Yields **1**
^	numeric	power	`2 ^ 3` Yields **8**
>	any	greater than	`3 > 2` Yields **True**
<	any	less than	`10 < 3` Yields **False**

`>=`	any	greater than or equal to	`2 >= 2` Yields **True**
`<=`	any	less than or equal to	`3 <= 10` Yields **True**
`=` `==`	any	equal to	`2 = 3` Yields **False**
`<>`	any	not equal to	`2 <> 3` Yields **True**
`=`	string	case-insensitive equality	`'Lorem' = 'lorem'` Yields **True**
`==`	string	case-sensitive equality	`'Lorem'=='lorem'` Yields **False**
`<>`	string	case-insensitive inequality	`'Lorem'<>'lorem'` Yields **False**

`>`
`>=`
`=`
`<=`
`<`
	string	case-sensitive comparison	`'Lorem' >= 'lorem'`
			Yields **False**

`&` `and`	Boolean	logical AND	`2 = 2 and 1 = 1` Yields **True**
`|` `or`	Boolean	logical OR	`2 = 1 or 1 = 1` Yields **True**
`!` `not`	Boolean	logical NOT	`not (2 = 2)` Yields **False**
`.`	object	object method call	`(-1).abs()` Yields **1**

As in most programming languages, operators have certain precedence:

- level 1 — `.` (object method call)
- level 2 — `^`
- level 3 — `*`, `/`, `\`
- level 4 — `+`, `-`
- level 5 — `>`, `<`, `=`, `==`, `<=`, `>=`, `<>`
- level 6 — `Not`, `!`
- level 7 — `And`, `Or`, `&`, `|`

For example, the expression `2 * 2 + 3` performs the multiplication first and the addition second because the multiplication operator (`*`) has a higher precedence than the addition operator (`+`). To modify the precedence and perform the addition first, you must use parentheses (`()`) to modify the expression as follows: `2 * (2 + 3)`.

Searching for objects

You can use queries to search for shapes, fills, outlines, colors, and global objects. For more information, please see the following topics:

- Querying shapes
- Querying fills
- Querying outlines
- Querying colors
- Querying global objects

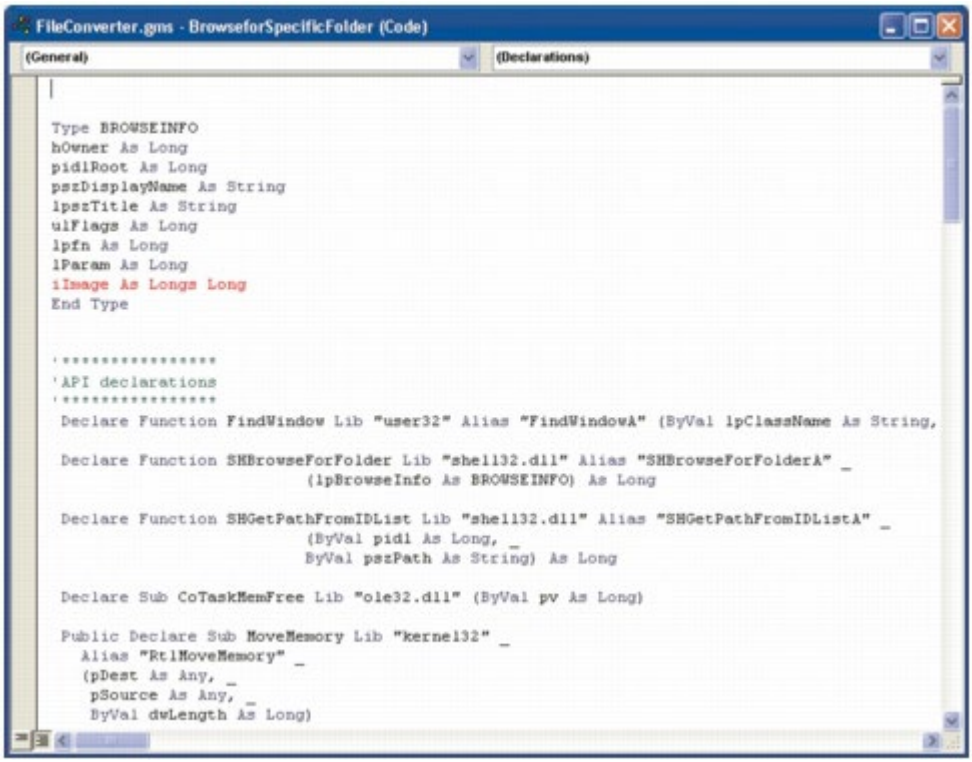
Copyright 2014 Corel Corporation. All rights reserved.

Using the Code window

The **Code** window is where you spend most of your time when working on macros. A standard code editor in the style of Microsoft Visual Studio, the **Code** window lets you do the following:

- format code automatically
- color syntax automatically
- check syntax automatically
- jump to definitions
- use contextual pop-up lists and automatic completion

If you are already familiar with any of the Microsoft Visual Studio editors, the **Code** window will be entirely familiar to you.



*The **Code** window*

To display the **Code** window, do one of the following:

- Click **View** } **Code**.
- Press **F7**.

Formatting code automatically

The Macro Editor formats code automatically for you. Even the capitalization of keywords, functions, subroutines, and variables is taken care of by the Macro Editor, irrespective of what you type. You cannot custom-format code, although you can set the indentation for each line, as well as the placing of custom line breaks.

If you use the returned value when calling a function, the parentheses around the parameters are mandatory (just as in most modern programming languages):

```
a = fooFunc (b, c)
```

However, if the returned value from a function call is being discarded, or if you are calling a subroutine, the parentheses must be left out (unlike in most other languages):

```
barFunc d, e
fooBarSub f
```

If you prefer always to see the parentheses, use the `Call` keyword before the function call or subroutine call:

```
Call barFunc (d, e)
Call fooBarSub (f)
```

Coloring syntax automatically

As you develop code in the **Code** window, the Macro Editor colors each word according to its classification.

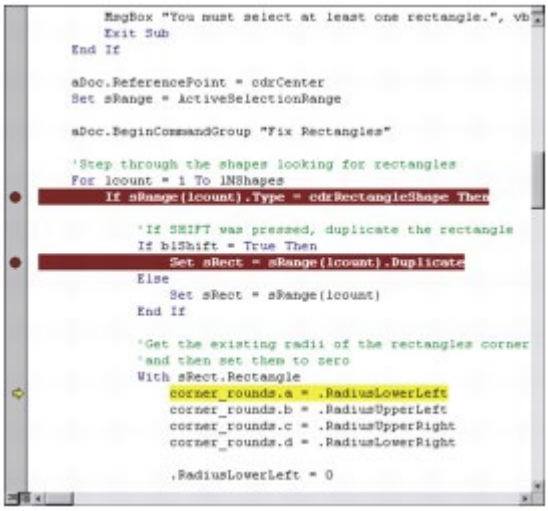
Word color	Classification

Blue	Automation keyword or programming statement
Green	Comment
Black	All other text

The **Code** window also uses the following colorization techniques:

Colorization technique	Classification
Red text	Line of code containing errors
White text on blue background	Selected text
Text highlighted in yellow	Line where execution is paused for debugging
White text on red background and red dot in the left margin	Breakpoint set for debugging purposes For more information, see Setting breakpoints.
Blue dot in the left margin	Bookmark set in the code

These syntax-colorization techniques make the code much easier to read.



Syntax coloring and highlighting

- Breakpoints and bookmarks are lost when you quit the application.

- The Macro Editor lets you modify the default colors for syntax highlighting. Click **Tools** } **Options**, and choose your settings on the **Editor Format** page.

Checking syntax automatically

Every time you move the cursor out of a line of code, the Macro Editor checks the syntax of the code in that line; if an error is found, the line is colored red and a pop-up warning is displayed. This real-time checking is useful (particularly when you are learning to program macros) because it indicates many possible errors in the code without having you run the code.

- The Macro Editor lets you disable pop-up warnings. Click **Tools** } **Options**, **click** the **Editor** tab, and then disable the **Auto Syntax Check** check box. Although the Macro Editor still checks the syntax and colors erroneous lines red, it stops displaying a warning when you paste text from another line of code.

Jumping to definitions

The Macro Editor lets you jump directly to the definition of a variable, function, or object.

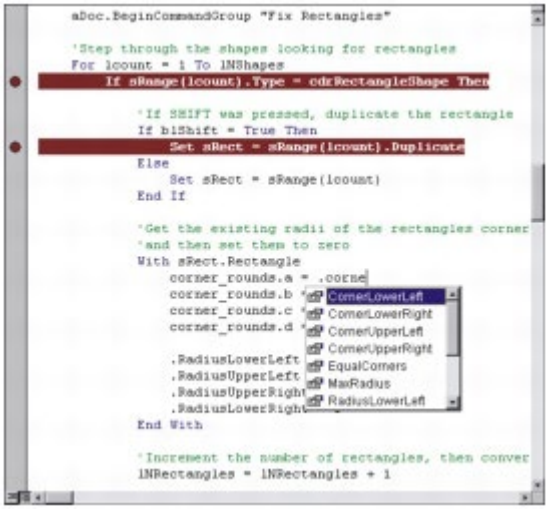
Desired definition	Procedure	Destination

Variable	Right-click the variable in the **Code** window, and then click **Definition**.	The definition of the variable in the code
Function	Right-click the function in the **Code** window, and then click **Definition**.	The definition of the function in the code
Object	Right-click the object in the **Code** window, and then click **Definition**.	The definition of the object in the Object Browser

- To return to where you requested the definition, right-click anywhere in the **Code** window, and then click **Last Position**.

Using contextual pop-up lists for automatic completion

The Macro Editor adds the functions you write and the variables you define to an internal list that contains all built-in keywords and enumerated values. As you type, the Macro Editor displays a contextual list of words that are valid candidates for insertion at the current position. This auto-completion feature makes code development quicker and more convenient.



An auto-completion pop-up list

If you type the first few characters of the word you want to use, the pop-up list advances to the nearest candidate that matches those characters. Select the desired word, and then do one of the following:

- type the character to follow the word (typically a space, line feed, parenthesis, period, or comma)
- enter only the word by pressing **Tab** or **Ctrl + Enter**

- To force the pop-up menu display, press **Ctrl + Spacebar**; the menu scrolls to the word that most closely matches the characters that you have typed so far. This technique is particularly useful for filling parameter lists when calling a function or subroutine. If there is only one exact match, the Macro Editor inserts the word without popping up the list; to display the pop-up list for the selected keyword at any time without auto-filling it, press **Ctrl + J**.

Copyright 2014 Corel Corporation. All rights reserved.

Using the Macro Editor

CorelDRAW provides an integrated development environment (IDE) for creating VBA macro projects. Called the Macro Editor, this IDE is similar to the one included with the full version of Visual Basic. You can use the Macro Editor to perform various tasks related to VBA macros, such as the following:

- browsing the contents of a VBA macro project
- developing and debugging VBA macro code
- setting object properties for VBA macros
- creating dialog boxes or “forms” for VBA macros

- You cannot use the Macro Editor to compile executable (EXE) program files.

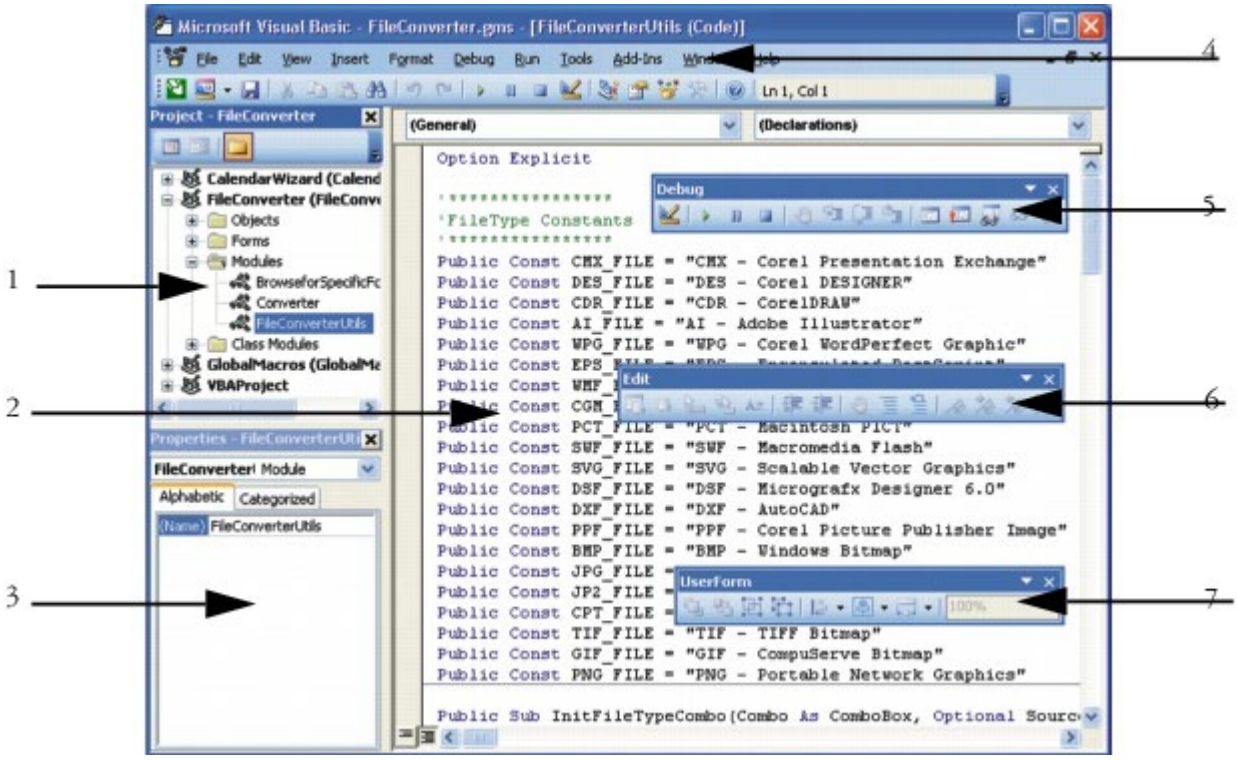
The Macro Editor features three main areas:

- Project Explorer — lets you browse macro projects and their contents. For more information, see Using the Project Explorer.
- **Code** window — lets you work with macro code. For more information, see Using the Code window.
- **Properties** window — lists all editable properties for the selected object. For more information, see Using the Properties window.

The Macro Editor also features four main toolbars:

- **Standard** toolbar — is the default toolbar
- **Debug** toolbar — contains buttons for common debugging tasks
- **Edit** toolbar — contains buttons for common code-editing tasks
- **UserForm** toolbar — contains buttons specific to designing dialog boxes

For more information on these toolbars, see Using the Macro Editor toolbars.



The Macro Editor (CorelDRAW version shown) features the following:
1) Project Explorer; 2) ***Code*** *window; 3)* ***Properties*** *window;*
4) ***Standard*** *toolbar; 5)* ***Debug*** *toolbar; 6)* ***Edit*** *toolbar;*
7) ***UserForm*** *toolbar*

The Macro Editor also lets you access the Object Browser, which displays the entire object model of each referenced component and of the host application. For more information, see Using the Object Browser.

Although the Macro Editor opens in a separate window from its host application, it runs within the process of that application. To display the Macro Editor, do any of the following:

- Click **Tools** } **Macros** } **Macro Editor** on the main menu in the application.
- Click the **Macro Editor** button on the **Macros** toolbar.
- Right-click **Visual Basic for Applications** in the **Macro Manager** docker, and then click **Show IDE**.

- Press **Alt + F11**.

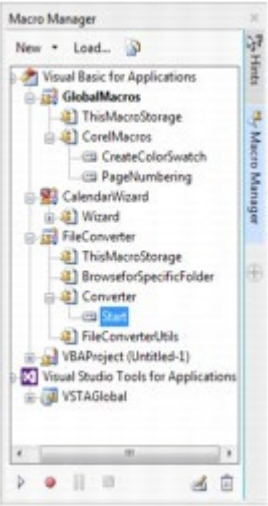
- To switch between the Macro Editor and the application, use the Windows taskbar, or press **Alt + F11** or **Alt + Tab**.

For more detailed information on constructing code procedures and setting properties, please see the Microsoft Visual Basic for Applications Help file, which is available from the **Help** menu in the Macro Editor.

Copyright 2014 Corel Corporation. All rights reserved.

Using the Macro Manager docker

In CorelDRAW, macros are stored in code modules, which are stored in macro projects. The **Macro Manager** docker provides a list of all existing VBA macro projects, plus the code modules and macros that are stored in them. You can use the **Macro Manager** docker to perform various tasks related to macro projects, code modules, and macros (see Creating macros).



*The **Macro Manager** docker, with a macro selected*

To display the **Macro Manager** docker, do one of the following:

- Click **Tools** } **Macros** } **Macro Manager**.
- Click the **Macro Manager** button on the **Macros** toolbar.

Copyright 2014 Corel Corporation. All rights reserved.

Using the Macros toolbar

CorelDRAW features a **Macros** toolbar that provides easy access to several macro-related featues, such as the Macro Editor.

*The **Macros** toolbar*

The Macros toolbar features the following buttons:

The **Macros** toolbar features the following buttons:

- **Macro Manager** button — opens the **Macro Manager** docker
- **Run Macro** button — runs a macro
- **Macro Editor** button — opens the Macro Editor
- **Disable application events** button — switches the Macro Editor between its modes for designing and running macros
- **Start Recording** button — records a macro
- **Pause Recording** button — pauses the recording of a macro
- **Stop Recording** button — stops the recording of a macro

To display the **Macros** toolbar, click **Window** } **Toolbars** } **Macros**. A check mark next to the command indicates that the toolbar is displayed.

Copyright 2014 Corel Corporation. All rights reserved.

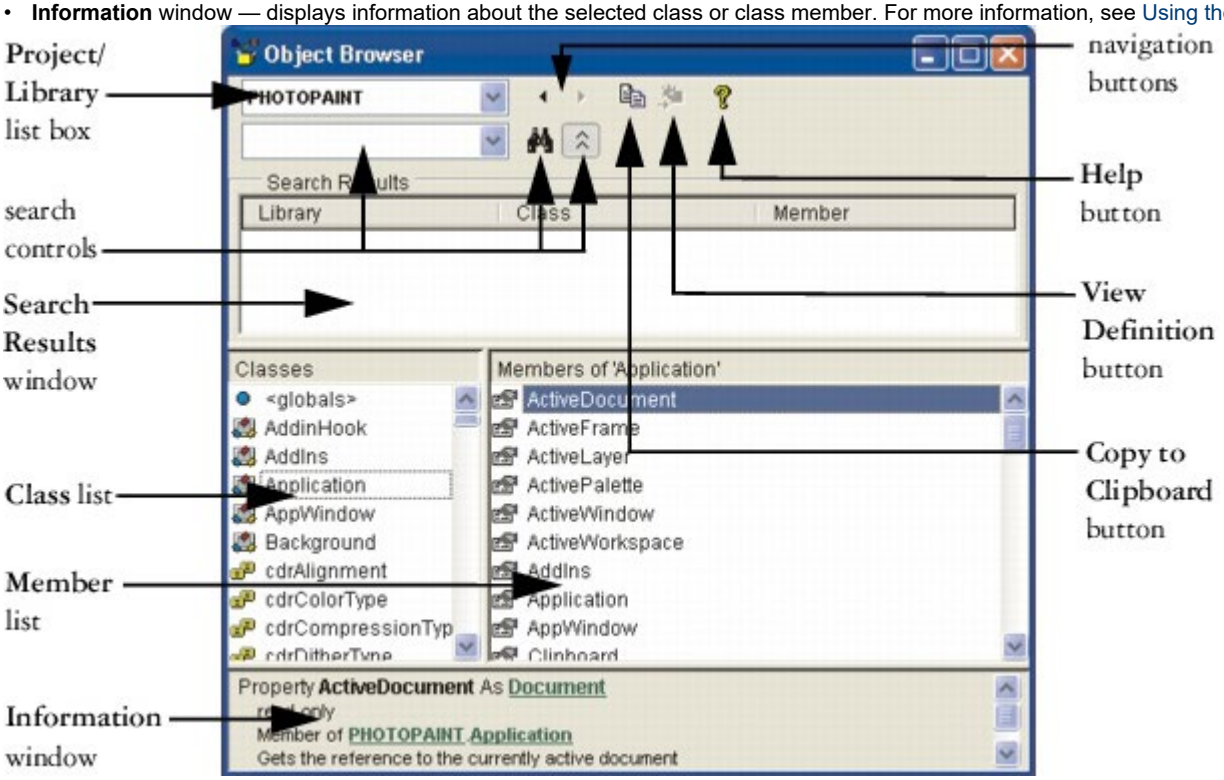
Using the Object Browser

The Object Browser is one of the most useful tools that is provided by the Macro Editor. In an easy-to-use, structured format, the Object Browser displays the entire object model of each referenced component and, most importantly, of CorelDRAW.

- Referenced components include all ActiveX or Object Linking and Embedding (OLE) objects that are used by the project.

The **Object Browser** window features the following items:

- **Project/Library** list box — lists all referenced components (projects and libraries). Choosing a project or library from this list updates the Object Browser to display only the items for that project or library. Generally, displaying only one project or library at a time makes it easier to use the Object Browser.
- navigation buttons — let you cycle through your selections from the Object Browser
- **Copy to Clipboard** button — copies the current selection to the Clipboard
- **View Definition** button — displays where the current selection is defined in the **Code** window
- **Help** button — displays a Help topic for the current selection. You can also access this Help topic by pressing **F1**.
- search controls — let you search the selected project or library for a given string. For more information, see Using the search controls.
- **Search Results** window — displays the results of a search. For more information, see Using the search controls.
- **Class** list — displays all class-related items for the selected project or library. For more information, see Using the Class list.
- **Member** list — displays the members of the selected class. For more information, see Using the Member list.
- **Information** window — displays information about the selected class or class member. For more information, see Using the Information window.



*The **Object Browser** window (Corel PHOTO-PAINT version shown)*

To open the Object Browser from within the Macro Editor, do any of the following:

- Click **View** } **Object Browser**.
- Click the **Object Browser** button on the **Standard** toolbar.
- Press **F2**.

- To reference the object model of another application, click **Tools** } **References**. Referenced components can be accessed by the macro code.

Using the Class list

Every project and library has an object model that contains the following class-related items, which are displayed in the **Class** list:

- global values

- classes
- modules
- types
- enumerations

Global values apply to an entire project or library, and they include individual members from enumerations (such as text-paragraph alignments, shape types, and import/export filters).

Classes contain properties, methods, and events. For more information, see What is a class?.

- For documentation on all classes available to CorelDRAW, see Classes.

Modules contain macro code.

- For documentation on all modules available to CorelDRAW, see Modules.

Types are customized data types that supplement the built-in data types that are provided by the automation environment (see Declaring variables).

- For documentation on all types available to CorelDRAW, see Types.

Enumerations represent fixed values in the procedures and functions of the coding for a macro. For more information, see What is an enumeration?.

- For documentation on all enumerations available to CorelDRAW, see Enumerations.

Each type of item in the **Class** list has an icon assigned to it:

Icon	Item
	global value
	class
	module
	type
	enumeration

- To access the Help topic for a selected item, click the **Help** button, or press **F1**.

Using the Member list

When you choose an item from the **Class** list, the members of that item appear in the **Member** list. Class members include the following:

- properties
- methods
- events
- constants

A property can be a simple type (such as a Boolean, integer, or string), or it can be a class or enumeration from the **Class** list. A property that is based on a class from the **Class** list inherits all members of that class.

Many classes have a default property. The default property is implied if no property name is given when getting or setting the value of the parent object. For example, collection types have the default property **Item**, which can be indexed; in such instances, it is not necessary to specify the **Item** property. Here,

```
ActiveSelection.Shapes.Item(1).Selected = False
```

is the same as the shorter

```
ActiveSelection.Shapes(1).Selected = False
```

because **Item** is the default or implied property of a collection of shapes.

- For documentation on all properties available to CorelDRAW, see Properties.

Methods are commonly known as “member functions” — functions that a class can perform on itself. A good example is the **Move** method of the **Shape** class, which moves a shape by using an [x, y] vector. The following code moves the selected shapes **2** measurement units to the right and **3** measurement units upwards:

```
ActiveSelection.Move 2, 3
```

If the return value of a function is not used, the function call does not take parentheses around the argument list unless the `Call` keyword is used.

- For documentation on all methods available to CorelDRAW, see Methods.

Events are associated with some classes. You can set up an event handler that is called when that event occurs in the application; this functionality lets you develop sophisticated applications that respond automatically to what is happening within the application. Commonly handled events include the **BeforePrint**, **BeforeSave**, **PageActivate**, **SelectionChange**, and **ShapeMove** events of the **Document** class.

- For documentation on all events available to CorelDRAW, see Events.

The constants displayed in the **Member** list are members of enumerations or are defined as public in a module. Enumerations group related items from a closed list — such as shape types, import/export filters, and alignments — for use anywhere an integer value is required.

In CorelDRAW, many constants begin with **cdr** (for example, **cdrEPS** and **cdrLeftAlignment**), while others begin with **clr**, **pdf**, **pnt**, or **prn**. Visual Basic also has its own constants, including ones (such as **vbKeyEnter**) for keystrokes and ones (such as **vbOK**) for dialog-box buttons.

- For documentation on all constants available to CorelDRAW, see Constants.

Each type of item in the **Member** list has an icon assigned to it:

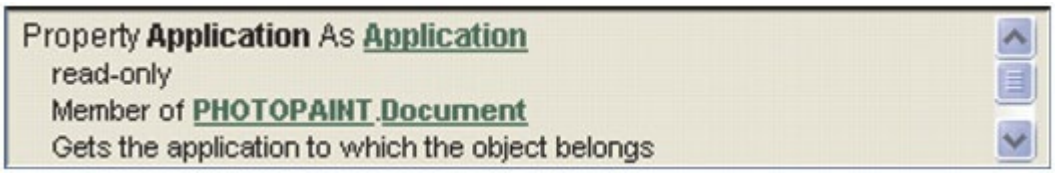
Icon	Item
	property
	default property
	method
	event
	constant

- To access the Help topic for a selected item, click the **Help** button, or press **F1**.

Using the Information window

The **Information** window provides information about the selected class or class member. This information includes the following:

- a prototype of the item
- an indication of whether the item is a read-only property
- the parent of the item
- a short description of the item



*The **Information** window for the **Document.Application** property in Corel PHOTO-PAINT*

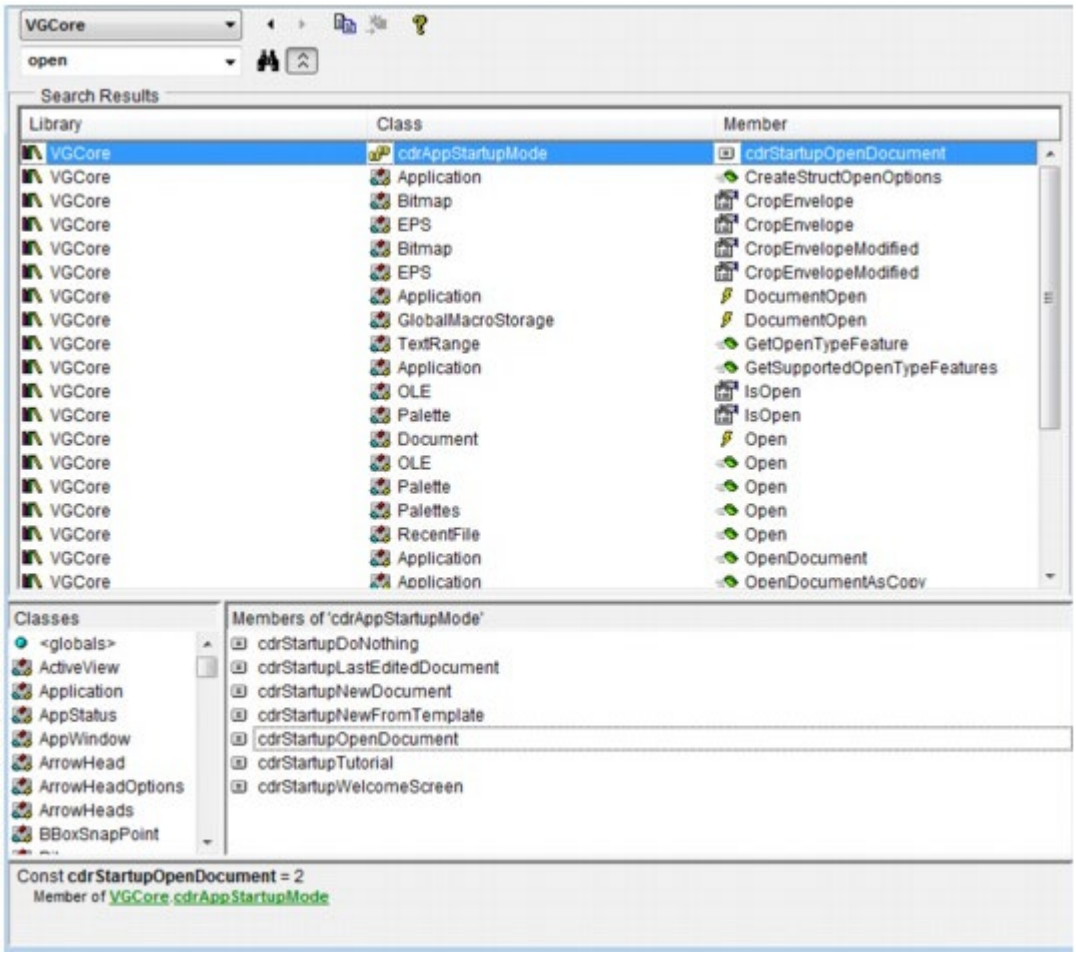
The **Information** window provides hyperlinks to all referenced types and classes that are defined within the current object model. For example, the information for the **Document.Application** property in Corel PHOTO-PAINT (see the preceding figure) includes the following hyperlinks:

- **Application** — accesses the **Application** class, because **Application** is both the type of the **Document.Application** property and a class in the **PHOTOPAINT** library
- **PHOTOPAINT** — accesses the class for the **PHOTOPAINT** library, which contains all classes in the Corel PHOTO-PAINT object model
- **Document** — accesses the **Document** class, which is the parent of the **Application** property

- When the **Information** window is not tall enough to reveal its complete contents, a scroll bar is provided. To increase the height of the **Information** window, drag the top border of the window upwards.

Using the search controls

The search controls let you search the selected project or library for a given string. Searching is useful when you can only partly remember the name of a class or class member, or when you want to find classes and members that have similar names (such as those containing the word “open”).



Searching the CorelDRAW object model

To search the classes and members of the selected object model, type a string into the **Search** box, and then click the **Search** button . The **Search Results** window displays, in alphabetical order, all matches. Clicking a match advances the **Class** list and **Member** list to that item and displays the **Information** window for that item.

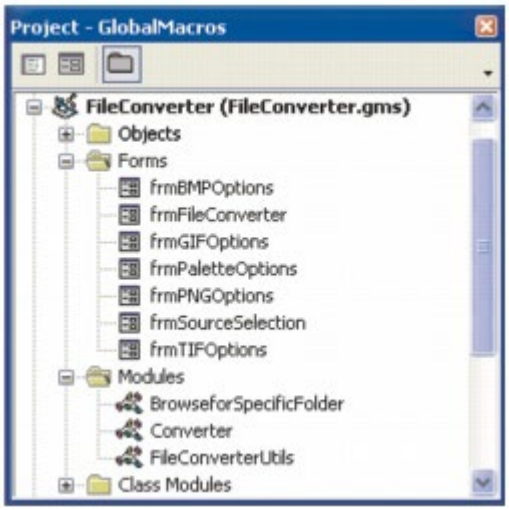
- Matching class names have a blank **Member** column in the **Search Results** window.

- To hide the **Search Results** window, click the **Hide Search Results** button .

Copyright 2014 Corel Corporation. All rights reserved.

Using the Project Explorer

The Project Explorer is essential for navigating macro projects and their contents: documents and objects, forms, modules, and class modules (or “classes”).



Each type of component in the Project Explorer has an icon assigned to it:

Icon	Item
	macro project
	folder
	document or object
	form
	module
	class module (or “class”)

To display or hide the Project Explorer, do any of the following:

- Click **View** } **Project Explorer**.
- Click the **Project Explorer** button on the **Standard** toolbar.
- Press **Ctrl + R**.

Copyright 2014 Corel Corporation. All rights reserved.

Using the Properties window

The **Properties** window lists all editable properties for the selected object. Many macro objects — including projects, modules, and forms and their controls — have property sheets that can be modified.

*The **Properties** window, with the properties of a form displayed*

The **Properties** window is automatically updated when you select an object, or when you change the properties of the selected object by using other methods (for example, by using the mouse to move and resize form controls).

To display or hide the **Properties** window, do any of the following:

- Click **View** } **Properties Window**.
- Click the **Properties Window** button on the **Standard** toolbar.
- Press **F4**.

Copyright 2014 Corel Corporation. All rights reserved.

Using the Macro Editor toolbars

The Macro Editor features four toolbars — **Standard**, **Debug**, **Edit**, and **UserForm** — that you can use to perform macro-related tasks.

The **Standard** toolbar is the default toolbar.

*The **Standard** toolbar (CorelDRAW version shown)*

The **Debug** toolbar contains buttons for common debugging tasks (see Debugging macros).

*The **Debug** toolbar*

The **Edit** toolbar contains buttons for common code-editing tasks.

*The **Edit** toolbar*

The **UserForm** toolbar contains buttons specific to designing dialog boxes (see Designing dialog boxes).

*The **UserForm** toolbar*

To display or hide a toolbar, click **View** } **Toolbars**, and then click the corresponding command. A check mark next to a command indicates that its toolbar is currently displayed.

- You can “float” a toolbar by dragging it from the menu bar.
- You can dock a toolbar by dragging it to the menu bar.

Copyright 2014 Corel Corporation. All rights reserved.

Using the VSTA Editor

CorelDRAW provides an integrated development environment (IDE) for creating VSTA macro projects. Called the VSTA Editor, this IDE is similar to the Macro Editor (which is the IDE for VBA macro projects in CorelDRAW). You can use the VSTA Editor to perform various tasks related to VSTA macros.

- To use the VSTA Editor, you must have Microsoft Visual Studio 2012 installed.

You can open the VSTA Editor from within CorelDRAW. To open the VSTA Editor, do any of the following:

- Click **Tools** } **Macros** } **VSTA Editor** on the main menu in the application.
- Press **Alt + Shift + F12**.

- To switch between the VSTA Editor and the application, use the Windows taskbar, or press **Alt + Shift F12** or **Alt + Tab**.

For more detailed information on VSTA and its programming environment, please consult the **Help** menu in the VSTA Editor.

Copyright 2014 Corel Corporation. All rights reserved.

Welcome to CorelDRAW X7 Macros Help

Welcome to CorelDRAW X7 Macros Help!

This resource can help you explore the macro-related features and functions of CorelDRAW. An understanding of these features and functions can help you automate tasks or develop commercial solutions that integrate with the software.

In this resource

This resource contains the following sections.

Section	Description
Understanding automation	Introduction to the concepts of automation and macros, and to the macro-programming formats supported by the software
Getting started with macros	Overview of the macro-related tools and features of the software
Creating macros	How to write, record, run, and debug macros
Including queries in macros	How to use Corel Query Language (CQL) to include queries in macros
Making macros user-friendly	How to enhance the usability of macros through dialog boxes, toolbar buttons, user interaction, and documentation
Organizing and deploying macros	How to organize and deploy the macros you create
Understanding the CorelDRAW object model	Overview of the most important features and functions of the CorelDRAW object model
Object Model Reference	Details on all available macro commands for CorelDRAW

Also included is a glossary, which defines many of the key terms used in this documentation.

- Most of the code examples provided in this documentation are written in VBA.

- In the Object Browser of the Macro Editor, you can display a Help topic for any selected object-model element by pressing **F1**.

In this sectionl

This section contains the following topics:

- About this resource
- About additional resources

Copyright 2014 Corel Corporation. All rights reserved.

What are the main elements of automation?

If you've ever developed object-oriented code in C++, Delphi, or Java, you're already familiar with programming-related concepts such as "objects," "classes," "properties," and "methods." However, let's reexamine these terms as they apply to automating CorelDRAW.

In this topic

This topic contains the following subtopics:

- What is an object model?
- What is a class?
- What is a collection?
- What is a property?
- What is a method?
- What is an event?
- What is an enumeration?
- What is a constant?

Copyright 2014 Corel Corporation. All rights reserved.

Which automation environments are supported?

For CorelDRAW versions 6 through 9, the only method of automating tasks was using the Corel SCRIPT language. Solution developers used Corel SCRIPT to create intelligent mini-applications for drawing shapes, repositioning and resizing shapes, opening and closing documents, and setting styles within CorelDRAW.

Although the Corel SCRIPT language was useful for automating basic tasks, a more flexible and powerful solution became necessary. For version 10, CorelDRAW was enhanced with support for the Microsoft Visual Basic for Applications (VBA) engine, which handled behind-the-scenes automation. The addition of VBA made CorelDRAW immediately accessible to millions of VBA and Microsoft Visual Basic (VB) developers around the world. Since then, VBA has been supported by every version of the CorelDRAW software suite.

More recently, the software suite added support for Microsoft Visual Studio Tools for Applications (VSTA), the successor to VBA.

By supporting VBA and VSTA, the software suite offers a platform for the following:

- developing powerful corporate graphical solutions — such as automated ticket generators, customized calendars, and batch file-processors
- streamlining workflows — such as with on-the-fly page-layout mechanisms
- customizing default software features — such as the creation, alignment, or transformation of objects
- ...and much more!

VBA and VSTA each provide their own fully integrated development environment (IDE), with contextual pop-up lists, syntax highlighting, line-by-line debugging, and visual designer windows. These features are particularly helpful to inexperienced developers.

For more information on VBA and VSTA, see the following topics:

- What is VBA?
- What is VSTA?

Copyright 2014 Corel Corporation. All rights reserved.

What is a class?

A class is the definition or description of an object. A class outlines the properties, methods, and events that apply to a type of object in an application; it acts as a template for all objects of that type class. To use a metaphor, the class “car” is a small vehicle with an engine and four wheels.

An object is an instance of a class. To extend the car metaphor, the actual, physical car purchased for the purposes of driving is an object (that is, an instance of the class “car”).

In the context of CorelDRAW, each open document is an instance of the **Document** class, each page in the document is an instance of the **Page** class, and each layer (and each shape on each layer) are more instances of more classes. For example, `Document` represents the **Document** class in the software program. However, `ActiveDocument` represents an object within that class because it makes specific reference to one object.

As previously discussed, objects are often made up of other smaller objects. For example, a car contains four objects of the class “wheel,” two objects of the class “headlight,” and so on. Each of these child objects has the same properties and methods of its class-type. This parent/child relationship of objects is an important one to recognize, particularly when referencing an individual object.

Some classes “inherit” features from their parents. For example, in the context of CorelDRAW, the **Shape** type has many subtypes (or “inherited types”), including **Rectangle**, **Ellipse**, **Curve**, and **Text**. All these subtypes can make use of the basic members of the **Shape** type, including methods for moving and transforming the shape and for setting its color. However, the subtypes also have their own specialist members; for example, a **Rectangle** can have corner radii, whereas **Text** has an associated **Font** property.

Copyright 2014 Corel Corporation. All rights reserved.

What is a collection?

A collection is similar to an array of objects; it is an object that contains a group of objects that are similar in type. These objects share the same properties, methods, and events, and they are uniquely identified within the collection by their index number or their name. Collection objects act in the same manner and are always plural.

For example, `Documents` represent the **Documents** collection class in the software program. However, `Documents.Item`(1) references the first **Document** object in that collection.

Copyright 2014 Corel Corporation. All rights reserved.

What is a constant?

A constant is an instance of an enumeration, and an enumeration groups similar constants together.

For example, cdrPageOrientation is an enumeration, yet it contains several constants, including cdrPortrait and cdrLandscape.

Copyright 2014 Corel Corporation. All rights reserved.

What is a method?

A method is like a verb in that it represents an action that can be performed by or on an object. In the example of a class “car,” the car can be made to go faster and slower, so two methods for the class are “accelerate” and “decelerate.”

In the context of CorelDRAW, documents have methods for creating new pages, layers have methods for creating new shapes, and shapes have methods for applying transformations and effects. For example, `ActiveDocument.Close` represents the **Close** method of a **Document** object; it closes the active document.

Copyright 2014 Corel Corporation. All rights reserved.

What is a property?

A property is like an adjective in that it represents an attribute or characteristic quality of an object. Properties can be returned or set, or they can be read-only.

Most classes have properties. As an illustration, the properties of the class “car” are that it is small, it has an engine, and it has four wheels. Every instance of the class “car” (that is, every object in that class) also has properties such as color, speed, and number of seats. Read-only properties are fixed by the design of the class; for example, the number of wheels or seats does not (usually) vary from car to car. However, other properties can be changed after the object has been created; for example, the speed of the car can go up and down, and, with a bit of help, its color can be changed.

In the context of CorelDRAW, **Document** objects have a name, a resolution, and horizontal and vertical ruler units; individual shapes have outline properties and fill properties, as well as a position and a rotation factor; and text objects have text properties, which may include the text itself. For example, `ActiveDocument.Name` represents the **Name** property of a **Document** object; it specifies the name of the active document.

Copyright 2014 Corel Corporation. All rights reserved.

What is an enumeration?

An enumeration (also called an “enumerated type”) represents a fixed value in the procedures and functions of the coding for a macro. Whereas a variable temporarily stores a changing data value, the value of an enumeration does not change.

Copyright 2014 Corel Corporation. All rights reserved.

What is an event?

An event is like a noun in that it represents an action that takes place within an object. An event is triggered by an action, such as a mouse click, a key press, or a system timer. An event can be coded to trigger appropriate response in its object.

For example, the `ActiveDocument.AfterSave` event triggers an action in the **Document** object after it has been saved.

Copyright 2014 Corel Corporation. All rights reserved.

What is an object model?

An object model represents the hierarchy of items (or “objects”) that make up an application and defines the interrelationships of the objects within that hierarchy. In an object model, each object is a child of another object, which is a child of yet another object, and so on. Furthermore, each object in an object model is defined by a property, a method, or an event, or a combination of these items.

Besides providing a high level of structure, an object model also lets you use object types (or “classes”) in various ways. For example, a **Shape** object of type “group” is used to contain other **Shape** objects, each of which is from type “group” or some other type, such as “rectangle,” “curve,” or “text.”

This high level of organization makes the object model easy to use, yet powerful.

How is an object model used in automation?

Automating CorelDRAW is accomplished by using the object model of the application to access the various objects in a document and make changes to those objects.

In CorelDRAW, the **Application** object represents the top of the object hierarchy: the program itself. All objects are children or grandchildren (or great-grandchildren, and so on) of the application.

Starting with the **Application** object, you can “drill down” through the layers of hierarchy in the object model until you find the desired, and usually the more specific, object. To reference the desired object, you must use a standard notation to separate each level of the object hierarchy. As in many object-oriented languages, the automation environment requires the use of a period or “dot operator” (`.`) to indicate that the object on the right is a member (or child) of the object on the left.

```
Application.Documents(1).Pages(1).Layers(1).Shapes(1).Name = "ABC"
```

An object requires its full hierarchical (or “fully qualified”) reference unless a shortcut is available to it (or unless it has an implicit or implied meaning). An object shortcut is merely a syntactic replacement for the long-hand version of an object. For example, the shortcut object `ActiveLayer` replaces the long-hand version `Application.ActiveDocument.ActivePage.ActiveLayer`, while the object shortcut `ActiveSelection` replaces the long-hand version `Application.ActiveDocument.Selection`.

- For more information on object shortcuts, see Using object shortcuts.

Copyright 2014 Corel Corporation. All rights reserved.

What is automation?

Many actions that you perform in the software can be combined with other, related actions into a single automated solution. Automating repetitive tasks saves time, reduces effort, and lets you perform operations that are too complex to perform manually.

Automation can be used by programmers and nonprogrammers alike.

- This documentation does not teach programming skills to nonprogrammers; rather, it helps experienced programmers develop useful solutions within the software. If you are not a programmer, you may want to refer to other programming-related resources before continuing to read this documentation.

What is a macro?

Most macros are created to automate a series of tasks within an application. The simplest meaning of the term “macro” is the recording of a group of related actions that can be played back automatically, in sequence, whenever you need to perform them. Macros consist of instructions that are written in a programming language, and some programming languages provide access to additional, more advanced, actions that cannot be recorded.

- For the purposes of this documentation, a macro refers to a coded solution that performs tasks in the application by automating functions and subroutines (see Using functions and subroutines).

Although you can record a sequence of actions in the software, the real power of automation is that you can add conditions and looping mechanisms to a recording. As an example, let’s consider a simple macro that applies a red fill and a 1-point outline to a selected shape. By adding a condition and a looping mechanism to the code, you can produce a macro that seeks out each selected shape and applies only the fill to text shapes and only the outline to all other shape types.

After you create a macro, you can ensure that it has the desired outcome by stepping through its code one line a a time, or “debugging” it. When you are happy with the macro, you can keep it for future use and even share it with others.

Which sample macros are available?

The software includes sample VBA macros, which supply additional functionality, demonstrate automation in the suite, and provide sample code.

The following sample VBA macros are included for CorelDRAW:

- File converter (**FileConverter.gms**) — converts a vector or bitmap to a specified vector or bitmap format. You can choose export parameters by using dialog boxes associated with particular filters. You can also save each page as a separate file and set various page properties, such as size, orientation, and background color. The following file formats are supported: AI, BMP, CDR, CGM, CMX, CPT, DSF, DXF, EPS, GIF, JPEG, PCT, PNG, PPF, SVG, SWF, TIF, WMF, and WPG.
- Calendar wizard (**CalendarWizard.gms**) — generates customized calendars. You can choose the date range, layout, font, color, language, and many other options. You can also add holidays and moon phases.

Copyright 2014 Corel Corporation. All rights reserved.

What is VBA?

Microsoft Visual Basic for Applications (VBA) is a built-in programming environment that can be used to automate repetitive functions and create intelligent solutions in the software that supports it. VBA is a subset of the Microsoft Visual Basic (VB) object-driven programming environment. Usually, VBA is integrated into another application to customize functionality within that application.

VBA is both a language and an editor. The VBA language cannot be used without its editor, and the VBA editor is the only utility in which VBA code can be edited or VBA programs can be run.

The VBA language is an event-driven programming language. In other words, it is used to write code that produces a response to an action, such as clicking a button or choosing an option from a list box. When the action occurs, the appropriate event is called, and the code for that event is executed. Events can be simple or complex. For instance, you can code a single line that displays a message box or write an entire procedure that interacts with a database.

- With traditional procedural programming (or “object-driven programming”), the program starts at the first line and executes one line at a time. VB provides an example of an object-driven programming environment.
- Most of the code examples provided in this documentation are written in VBA.

The VBA editor — called the “Macro Editor” (formerly the “Visual Basic Editor”) in CorelDRAW — is an integrated development environment (IDE) that lets you manipulate the objects that are exposed by the object model of the application. To help you code macros, the Macro Editor provides context-sensitive Help for all the object-model elements that are available to you.

VBA is an in-process automation controller. In other words, VBA can be used to control software features that can be automated, and VBA runs efficiently by bypassing the interprocess synchronization mechanisms. However, the automation that the in-process VBA can access can also be accessed by the following:

- external out-of-process automation controllers (OLE clients)
- applications that are developed in programming languages (such as VB, Visual C++, Windows Script Host, and C++) that can be used to develop OLE clients
- the VBA engines of other applications

VBA provides a set of tools for customizing the graphical user interface of the software. These tools let you process and present data efficiently and effectively. Advantages of using VBA include the following:

- familiarity of the VB language
- rapid application development (RAD) IDE
- fast run-time performance of the resulting integrated solutions
- extensible forms package that supports ActiveX controls for creating user interfaces
- access to the full Windows application programming interface (API) and the underlying file system
- connectivity to corporate data
- integration with other software that is based on component object models (COMs)

VBA lets you customize an application to suit your needs, or even integrate it with another VBA-enabled application by referencing the object-model components of the second application. Although VBA was developed by Microsoft and is built into almost all its desktop applications (including Microsoft Office), Microsoft licenses the technology to other companies (including Corel Corporation; Autodesk, Inc., in AutoCAD; and IntelliCAD Technology Consortium, in IntelliCAD). Software products that support VBA can typically be used to control each other, and they can even be used to control some software products that do not support VBA. Consequently, you can use VBA to build solutions in CorelDRAW that access a wide variety of other software products: databases, word processors, XML editors, and more.

- For a complete list of applications that support VBA, see the Microsoft website.

How does VBA differ from VB and VBScript?

The VB programming system is an advanced set of programming tools that provides advanced functionality and components for the Windows operating system and other Windows-based programs. For example, unlike VBA or VBScript, VB lets you create application extensions (DLL files) and stand-alone executable programs (EXE files). (The programs that you create with VBA must run inside the host application.)

VB is a “visual”version of the BASIC programming language — that is, it provides visual cues within the editor. As a result, VB is an easy language to learn. In addition, Microsoft has greatly enhanced the original BASIC language, so that VB is both powerful and fast (although not as powerful as Java or C++, nor as fast as C).

VBA is a subset of the VB programming language, and it uses the programming structure of VB to manipulate the object-model elements that are exposed by an application. The manipulation of these objects results in small packets of code procedures within the application. These code procedures and resulting projects are called “add-ins.”

VBScript (sometimes referred to as Microsoft Visual Basic, Scripting Edition) is also a subset of the VB programming language. VBScript is a Web-based HTML document scripting language.

How does VBA differ from Java and JavaScript?

VBA is similar to Java and JavaScript in that it is a high-level, procedural programming language with full garbage collection and very little memory-pointer support. (For more information, see Allocating memory.) In addition, code that is developed in VBA, much like code developed in Java and JavaScript, supports on-demand compilation and can be executed without being compiled.

VBA is also similar to JavaScript in that it cannot be executed as a stand-alone application. JavaScript is embedded within Web pages as a mechanism for manipulating the document object model (DOM) of the Web browser. Likewise, VBA programs are executed inside a host environment (such as CorelDRAW) to manipulate the object model of the host.

Most VBA applications can be compiled to p-code to make them run more quickly, although the difference is hardly noticeable because of the sophistication of today's computer hardware. Similar compilation is possible with Java, but not with JavaScript.

Finally, whereas VBA uses a single equals sign (=) for both comparison and assignment, Java and JavaScript use a single equals sign (=) for assignment and two equals signs (==) for Boolean comparison. (For more information, see Using Boolean comparison and assignment.)

How does VBA differ from C and C++?

Like C and C++, VB uses functions. In VB, functions can be used to return a value, but subroutines cannot be used in that way. However, functions are used in C and C++, regardless of whether you want to return a value. (For more information, see Using functions and subroutines.)

VBA allocates and frees memory transparently. In C and C++, however, the developer is responsible for most memory management. As a result, using strings in VBA is even simpler than using the **CString** class in C++.

Finally, whereas VBA uses a single equals sign (=) for both comparison and assignment, C and C++ use a single equals sign (=) for assignment and two equals signs (==) for Boolean comparison. (For more information,see Using Boolean comparison and assignment.)

How does VBA differ from WSH?

Windows Script Host (WSH) is an out-of-process automation controller that lets you do occasional scripting and automation of Windows tasks and can be used to control software. Although WSH is a useful addition to the Windows operating system, WSH scripts tend to be slow because they must run out of process, and they cannot be compiled (and must be interpreted as they are executed).

WSH is a host for a number of scripting languages, each of which has its own syntax. However, the standard language that WSH uses is a macro language that resembles VB, so for standard scripts, the syntax is the same as in VBA.

Copyright 2014 Corel Corporation. All rights reserved.

What is VSTA?

The successor to VBA, Microsoft Visual Studio Tools for Applications (VSTA) is based on Microsoft Visual Studio 2012. In CorelDRAW, the VSTA feature supports the .NET framework and enables development in two programming languages: Visual Basic .NET and Visual C#.

- Most of the code examples provided in this documentation are written in VBA.

The VSTA Editor in CorelDRAW is an integrated development environment (IDE) that lets you create VSTA solutions for the software.

- To use the VSTA macro features with CorelDRAW, you must have Microsoft Visual Studio 2012, Professional, Ultimate, or Premium installed.
- If you install Microsoft Visual Studio after installing CorelDRAW Graphics Suite, you must re-install the VSTA macro features by modifying your CorelDRAW Graphics Suite installation.

How does VSTA compare with VBA?

Both VSTA and VBA allow you to create powerful macro solutions. With VSTA, you use the VSTA Editor as an IDE, and you use Visual Basic .NET or Visual C# as a programming language. With VBA, you use the Macro Editor as an IDE, and you use VBA as a programming language.

If you want to perform any of the following tasks, you can use either VSTA or VBA:

- customize or extend the features of the software
- interact with other applications that use Visual Basic 6 — or with other compatible components that are external to the software
- interact with Web-based services
- customize the IDE with add-ins
- create macro projects — with multi-threading support, if desired
- access macro projects programmatically
- generate macro code dynamically
- store macro code in a pre-compiled format
- hide macro code from other macro authors
- debug macro projects
- create customized user interfaces for macro projects

However, if you want to perform any of the following tasks, you must use VSTA:

- access the .NET framework natively — to support using Managed Add-in Framework (MAF), referencing .NET assemblies directly, running customized code on the Common Language Runtime (CLR), enforcing .NET security policies, or creating user interfaces by using .NET WinForms
- fully customize the IDE
- create macro projects that are certified to run on Windows
- create macro projects that support 64-bit processors
- create macro projects that support server-side customizations
- create macro projects that support all data types, including `BigDecimal` and `Int64`
- create macro projects and macro assemblies that persist without the use of structured storage
- open and modify macro projects in Visual Studio
- compile macro projects to DLL assemblies
- run macro projects out of process
- run macro projects without causing the host application to stop execution at errors or breakpoints
- isolate macro projects from one another; run macro projects independently, and stop them during runtime without affecting other running projects
- author macros within managed code
- prevent servers from running customized user interfaces for macro projects

Copyright 2014 Corel Corporation. All rights reserved.

Working with the Application object

In CorelDRAW, the **Application** object is the root of all other objects and is used if no other root object is specified. You can use the application's **Application** object to reference the application's object model from an out-of-process controller.

```
Dim cdr As CorelDRAW.Application
Set cdr = CreateObject("CorelDRAW.Application")
```

- If desired, you can avoid using the **CreateObject** keyword in the preceding example by importing the target type library and using the data types directly.

Copyright 2014 Corel Corporation. All rights reserved.

Working with documents

Each open document, or **Document** object, is a member of the **Application.Documents** collection. The documents in that collection appear in the order in which they were created or opened.

CorelDRAW provides a number of properties, methods, and events for working with documents, the most useful of which are listed in the following table.

Class	Member	Description
ActiveView	**OriginX** property and **OriginY** property	Combine to specify the origin of the active view For more information, see Panning.
ActiveView	**SetViewPoint** method	Specifies the origin of the active view For more information, see Panning.
ActiveView	**Zoom** property	Specifies the zoom factor of the active view For more information, see Zooming.
Application	**ActiveDocument** property	Provides direct access to the active document For more information, see Activating documents.
Application	**CreateDocument** method or **CreateDocumentFromTemplate** method	Create a document For more information, see Creating documents.
Application	**DocumentAfterExport** event	Is triggered when a document is exported (that is, when the **Export** dialog box closes) For more information, see Exporting files from documents.
Application	**DocumentAfterPrint** event	Is triggered when a document is printed (that is, when the **Print** dialog box closes) For more information, see Printing documents.
Application	**DocumentAfterSave** event	Is triggered when a document is saved (that is, when the **Save** dialog box closes) For more information, see Saving documents.
Application	**DocumentBeforeExport** event	Is triggered when the **Export** dialog box opens For more information, see Exporting files from documents.
Application	**DocumentBeforePrint** event	Is triggered when the **Print** dialog box opens For more information, see Printing documents.
Application	**DocumentBeforeSave** event	Is triggered when the **Save** dialog box opens For more information, see Saving documents.
Application	**DocumentClose** event	Is triggered when a document is closed For more information, see Closing documents.
Application	**DocumentNew** event	Is triggered when a document is created For more information, see Creating documents.
Application	**DocumentOpen** event	Is triggered when a document is opened For more information, see Opening documents.
Application	**Documents** property	Contains the collection of open documents For more information, see Activating documents.
Application	**OpenDocument** method	Opens a document

		For more information, see Opening documents.
Application	**QueryDocumentClose** event	Is triggered when the user responds to a request to close a document For more information, see Closing documents.
Application	**QueryDocumentExport** event	Is triggered when the user responds to a request to export a document For more information, see Exporting files from documents.
Application	**QueryDocumentPrint** event	Is triggered when the user responds to a request to print a document For more information, see Printing documents.
Application	**QueryDocumentSave** event	Is triggered when the user responds to a request to save a document For more information, see Saving documents.
Application	**WindowActivate** event	Is triggered when a window is activated For more information, see Activating documents.
Application	**WindowDeactivate** event	Is triggered when a window is deactivated For more information, see Activating documents.
Document	**Activate** method	Activates a document For more information, see Activating documents.
Document	**ActiveWindow** property	Provides direct access to the active window for a document For more information, see Working with windows.
Document	**AfterExport** event	Is triggered when a document is exported (that is, when the **Export** dialog box closes) For more information, see Exporting files from documents.
Document	**AfterPrint** event	Is triggered when a document is printed (that is, when the **Print** dialog box closes) For more information, see Printing documents.
Document	**AfterSave** event	Is triggered when a document is saved (that is, when the **Save** dialog box closes) For more information, see Saving documents.
Document	**BeforeExport** event	Is triggered when the **Export** dialog box opens For more information, see Exporting files from documents.
Document	**BeforePrint** event	Is triggered when the **Print** dialog box opens For more information, see Printing documents.
Document	**BeforeSave** event	Is triggered when the **Save** dialog box opens For more information, see Saving documents.
Document	**BeginCommandGroup** method and **EndCommandGroup** method	Combine to create a “command group” that reduces a series of programmed, document-related actions to a single, undoable step For more information, see Creating command groups for documents.
Document	**Close** event	Is triggered when a document is closed For more information, see Closing documents.
Document	**Close** method	Closes a document For more information, see Closing documents.
Document	**CreateView** method	Creates a document view For more information, see Working with views.
Document	**Export** method,	Exports a file from a document

	ExportEx method, or **ExportBitmap** method	For more information, see Exporting files from documents.
Document	**FilePath** property, **FileName** property, or **FullFileName** property	Specifies the file path or filename (or both) of a saved document For more information, see Activating documents.
Document	**GetUserArea** method	Returns information about a document area that the user drags with the mouse For more information, see Capturing mouse drags.
Document	**GetUserClick** method	Returns information about a document position that the user clicks with the mouse For more information, see Capturing mouse clicks.
Document	**Open** event	Is triggered when a document is opened For more information, see Opening documents.
Document	**PrintOut** method and **PrintSettings** property	Combine to print a document by using the specified settings For more information, see Printing documents.
Document	**PublishToPDF** method and **PDFSettings** property	Combine to publish a document to PDF by using the specified settings For more information, see Publishing documents to PDF.
Document	**QueryClose** event	Is triggered when the user responds to a request to close a document For more information, see Closing documents.
Document	**QueryExport** event	Is triggered when the user responds to a request to export a document For more information, see Exporting files from documents.
Document	**QueryPrint** event	Is triggered when the user responds to a request to print a document For more information, see Printing documents.
Document	**QuerySave** event	Is triggered when the user responds to a request to save a document For more information, see Saving documents.
Document	**ReferencePoint** property	Specifies the reference point for the document For more information, see Setting document properties.
Document	**SaveAs** method or **Save** method	Saves a document For more information, see Saving documents.
Document	**Unit** property	Specifies the unit of measurement for the document For more information, see Setting document properties.
Document	**WorldScale** property	Specifies the drawing scale for the document For more information, see Setting document properties.
Document	**Views** property	Contains the collection of views for a document For more information, see Working with views.
GlobalMacroStorage	**DocumentAfterExport** event	Is triggered when a document is exported (that is, when the **Export** dialog box closes) For more information, see Exporting files from documents.
GlobalMacroStorage	**DocumentAfterPrint** event	Is triggered when a document is printed (that is, when the **Print** dialog box closes) For more information, see Printing documents.
GlobalMacroStorage	**DocumentAfterSave** event	Is triggered when a document is saved (that is, when the **Save** dialog box closes)

		For more information, see Saving documents.
GlobalMacroStorage	**DocumentBeforeExport** event	Is triggered when the **Export** dialog box opens For more information, see Exporting files from documents.
GlobalMacroStorage	**DocumentBeforePrint** event	Is triggered when the **Print** dialog box opens For more information, see Printing documents.
GlobalMacroStorage	**DocumentBeforeSave** event	Is triggered when the **Save** dialog box opens For more information, see Saving documents.
GlobalMacroStorage	**DocumentClose** event	Is triggered when a document is closed For more information, see Closing documents.
GlobalMacroStorage	**DocumentNew** event	Is triggered when a document is created For more information, see Creating documents.
GlobalMacroStorage	**DocumentOpen** event	Is triggered when a document is opened For more information, see Opening documents.
GlobalMacroStorage	**QueryDocumentClose** event	Is triggered when the user responds to a request to close a document For more information, see Closing documents.
GlobalMacroStorage	**QueryDocumentExport** event	Is triggered when the user responds to a request to export a document For more information, see Exporting files from documents.
GlobalMacroStorage	**QueryDocumentPrint** event	Is triggered when the user responds to a request to print a document For more information, see Printing documents.
GlobalMacroStorage	**QueryDocumentSave** event	Is triggered when the user responds to a request to save a document For more information, see Saving documents.
GlobalMacroStorage	**WindowActivate** event	Is triggered when a window is activated For more information, see Activating documents.
GlobalMacroStorage	**WindowDeactivate** event	Is triggered when a window is deactivated For more information, see Activating documents.
View	**Activate** method	Applies a saved view to the document window For more information, see Working with views.
Window	**Activate** method	Activates a document window For more information, see Working with windows.
Window	**ActiveView** property	Provides direct access to the active view for a document window For more information, see Working with views.
Window	**Close** method	Closes a document window For more information, see Working with windows.
Window	**NewWindow** method	Creates a document window For more information, see Working with windows.
Window	**Next** property	Accesses the next window for a document For more information, see Working with windows.
Window	**Previous** property	Accesses the previous window for a document

For more information, see Working with windows.

- For detailed information on any property, method, or event, see the Object Model Reference.

In this topic

For more information on document-related activities, see the following subtopics:

- Creating documents
- Opening documents
- Activating documents
- Setting document properties
- Displaying documents
- Modifying documents
- Creating command groups for documents
- Saving documents
- Exporting files from documents
- Publishing documents to PDF
- Printing documents
- Closing documents

- Files of all supported formats can be imported. Imported files are placed on document layers, so information on importing files is provided in the section on working with layers (see Importing files into layers) rather than in this section on working with documents.

Copyright 2014 Corel Corporation. All rights reserved.

Working with export filters

The following VBA example demonstrates how to save a document as an AutoCAD DXF file by using an export filter:

```
Sub SaveRectangleDXF()
Dim FilterObject As DXFExport
Dim BitmapType As DxfBitmapType
Dim TextAsCurves As Boolean
Dim Units As DxfUnits
Dim Version As DxfVersion
'Initialize FilterObject
Set FilterObject = ActiveDocument.ExportEx("C:\devo\rect.dxf", _
cdrDXF)
'Set the advanced features of the filter
BitmapType = dxfBitmapGIF
FilterObject.BitmapType = BitmapType
Units = dxfInches
FilterObject.Units = Units
TextAsCurves = False
FilterObject.TextAsCurves = TextAsCurves
Version = dxfVersion2000
FilterObject.Version = Version
'Invoke the filter
FilterObject.Finish
End Sub
```

In the preceding example, a call is made to **ActiveDocument.ExportEx** method, and the interface for the export filter (**DXFExport**) is invoked. However, you can use the generic export interface (**ExportFilter**) rather than the filter-specific interface (**DXFExport**), as in the following VBA example:

```
Sub SaveRectangle()
Dim FilterObject As ExportFilter
'Initialize FilterObject
Set FilterObject = ActiveDocument.ExportEx("C:\devo\rect.dxf", cdrDXF)
'Set the advanced features of the filter
FilterObject.BitmapType = 1 'GIF
FilterObject.Units = 0 'Inches
FilterObject.TextAsCurves = False
FilterObject.Version = 1 'AutoCAD 2000
'Invoke the filter
FilterObject.Finish
End Sub
```

The following VBA example demonstrates how to invoke the **Export** dialog box:

```
Sub ShowExportDialog()
Dim FilterObject As ExportFilter
Dim vntReturn As Variant
'Initialize FilterObject
Set FilterObject = ActiveDocument.ExportEx("C:\devo\rect.dxf", cdrDXF)
'If FilterObject supports a dialog, invoke it
If (FilterObject.HasDialog = True) Then
vntReturn = FilterObject.ShowDialog
'Verify that the user clicked "OK" and not "Cancel"
If (vntReturn = True) Then
'Invoke the filter
FilterObject.Finish
End If
End If
End Sub
```

The preceding example requires you to check the return value of the dialog box, and to invoke the **Finish** method for when the user clicks **OK**.

Copyright 2014 Corel Corporation. All rights reserved.

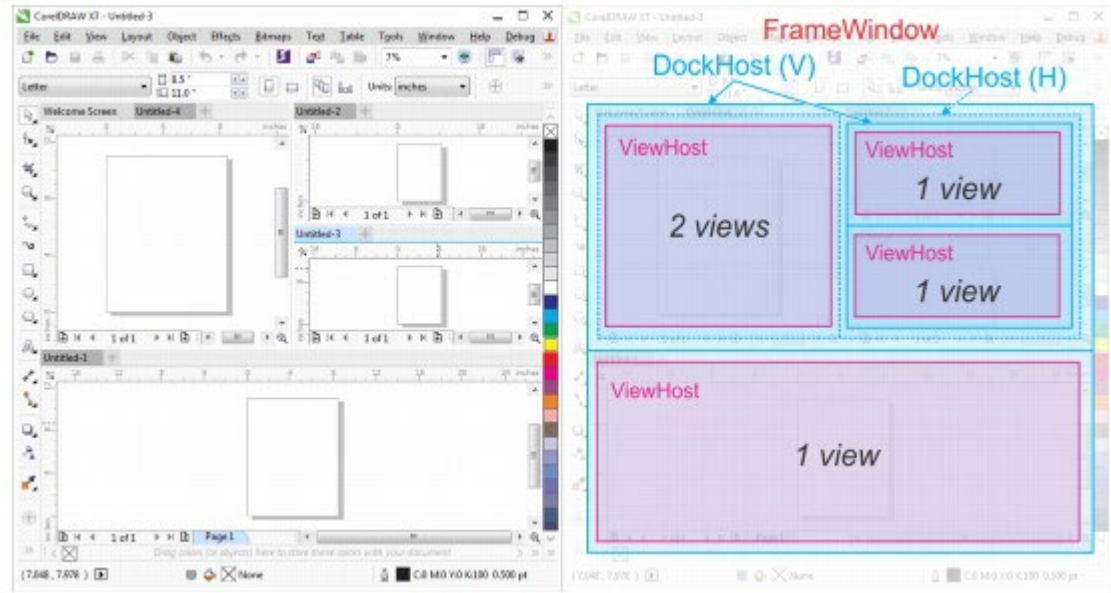
Arranging open documents

You can arrange and dock multiple open documents to suit your needs. When you drag a document out of the main window, it is placed in a new floating window. In the VBA object model, these windows are called FrameWindows. The main frame window hosts the menu, status bar and all the dockers by default; and, when closed, causes the application to quit.

Application.FrameWork.MainFrameWindow represents the main frame window. **Application.FrameWork.FrameWindows** is a collection of all the frame windows, including the main frame and frame windows that host floating dockers and floating toolbars.

A DockHost is a linear docking region that can host one or more items which, in turn, could be DockHost or ViewHost items. A DockHost is similar to a one-dimensional table. It can be either horizontal or vertical. To create complicated two-dimensional layouts, you must nest the DockHosts.

The end-node of this DockHost tree is a ViewHost that contains one or more view windows. They are the tabbed window controls as seen in the UI with each document view as a tab.The following screen capture and diagram show an example of CorelDRAW containing 5 views and how this layout was achieved by using DockHosts and ViewHosts.



Sample layout

- A horizontal DockHost can contain only vertical DockHosts as immediate children. That is, you cannot split a cell inside a horizontal DockHost horizontally. Instead, you must insert another cell.

For more information about arranging open documents, see the following topics:

- Understanding the layout structure
- Combining views
- Cascading views
- Tiling views

Copyright 2014 Corel Corporation. All rights reserved.

Understanding the layout structure

The following macro walks through the docking structure of each frame window and prints the layout structure (to the VBA Immediate window):

```
Sub DumpLayout()
Dim s As String
Dim f As FrameWindow
For Each f In FrameWork.FrameWindows
DumpFrameWindows f, 0, s
Next f
Debug.Print s
End Sub
Private Sub DumpDockItem(d As DockItem, indent As Long, ByRef s As String)
If d.Type = cuiDockItemDockHost Then
DumpDockHost d.DockHost, indent, s
Else
DumpViewHost d.ViewHost, indent, s
End If
End Sub
Private Sub DumpDockHost(d As DockHost, indent As Long, ByRef s As String)
If d Is Nothing Then Exit Sub
Dim item As DockItem
s = s & Space(indent) & "DockHost: " & d.ID & " - " & IIf(d.Orientation = _
cuiDockHostHorizontal, "H", "V") & vbCrLf
For Each item In d.Children
DumpDockItem item, indent + 2, s
Next item
End Sub
Private Sub DumpViewWindow(w As ViewWindow, indent As Long, ByRef s As String)
s = s & Space(indent) & "View: " & w.ID & " - Kind: " & w.Kind & " - " & w.Title & _
" / " & w.Description & vbCrLf
End Sub
Private Sub DumpViewHost(v As ViewHost, indent As Long, ByRef s As String)
Dim w As ViewWindow
s = s & Space(indent) & "ViewHost: " & v.ID & " - " & v.Views.Count & " view(s)" & _
vbCrLf
For Each w In v.Views
DumpViewWindow w, indent + 2, s
Next
End Sub
Private Sub DumpFrameWindows(f As FrameWindow, indent As Long, ByRef s As String)
s = s & Space(indent) & "FrameWindow: " & f.ID & " - " & IIf(f.IsMainFrame, _
"MainFrame", "Secondary") & vbCrLf
DumpDockHost f.RootDockHost, indent + 2, s
s = s & vbCrLf
End Sub
```

If you run the **DumpLayout** macro for the Sample layout, you will see an output similar to the following:

```
FrameWindow: Framework MainFrame - MainFrame
DockHost: fc796773-64ff-4250-8d64-3bd07fd9352f - V
DockHost: 1304df1c-d014-4daf-a755-c0fbbd983127 - H
ViewHost: 5795efd1-b582-4b80-b05c-469450c3babd - 2 view(s)
View: a961c249-6f05-4fbf-bb83-311d82c2444a - Kind: 1a9379f7-f018-3eb7-41e2-51436969b66f - Welcome Screen /
View: 7a1d8b68-a01f-45ad-80bc-67a8ab2b1a86 - Kind: ab303a90-464d-5191-423f-613c4d1dcb2c - Untitled-4 / Untitled-4
DockHost: ed290c1b-448e-49e6-a1e4-fbee981b9545 - V
ViewHost: 3ad110d1-94cb-4cab-9fab-2b431086aed4 - 1 view(s)
View: 235e9f22-79c5-42ed-ad20-9fa17248c83c - Kind: ab303a90-464d-5191-423f-613c4d1dcb2c - Untitled-2 / Untitled-2
ViewHost: e713b707-d067-4ef8-a60e-7ff492b545df - 1 view(s)
View: 338e338a-0f03-4013-965e-ceac1725bef9 - Kind: ab303a90-464d-5191-423f-613c4d1dcb2c - Untitled-3 / Untitled-3
ViewHost: 14f31f64-f810-4934-8c37-4f733393317e - 1 view(s)
View: dedddb8a-3bd7-46bc-9e14-0179c00963db - Kind: ab303a90-464d-5191-423f-613c4d1dcb2c - Untitled-1 / Untitled-1
```

Each of the elements — FrameWindow, DockHost, ViewHost, ViewWindow— have unique IDs (GUID) that you can use to identify the elements. A view window also has a GUID that lets you differentiate the Welcome screen (kind: 1a9379f7-f018-3eb7-41e2-51436969b66f) from a regular document window (kind: ab303a90-464d-5191-423f-613c4d1dcb2c).

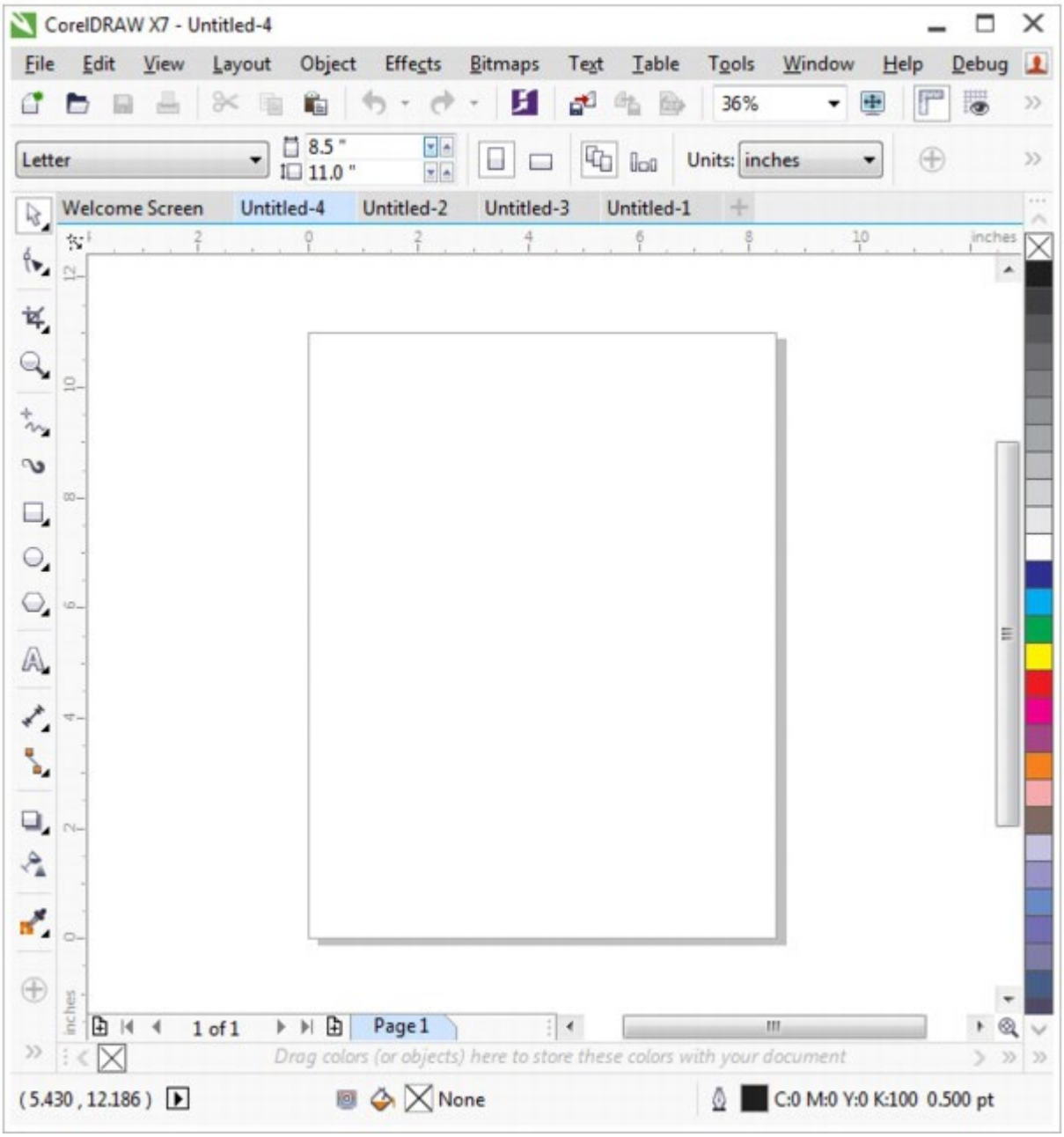
Copyright 2014 Corel Corporation. All rights reserved.

Combining views

The following VBA example takes all the document views and places them all in one ViewHost in the main frame window. Any docked or floating documents are placed in the main frame.

```
Sub CombineAllViews()
Dim vh As ViewHost, vhc As ViewHost
Dim f As FrameWindow
For Each f In FrameWork.FrameWindows
For Each vh In f.ViewHosts
If vhc Is Nothing Then
If FrameWork.MainFrameWindow.ViewHosts.Count = 0 Then _
FrameWork.MainFrameWindow.RootDockHost.InsertViewHost vh, 1
Set vhc = vh
Else
vhc.InsertViewHost vh, vhc.Views.Count + 1
End If
Next vh
Next f
End Sub
```

If the preceding macro is run on the Sample layout, the views are combined, and all open documents appear tabbed.



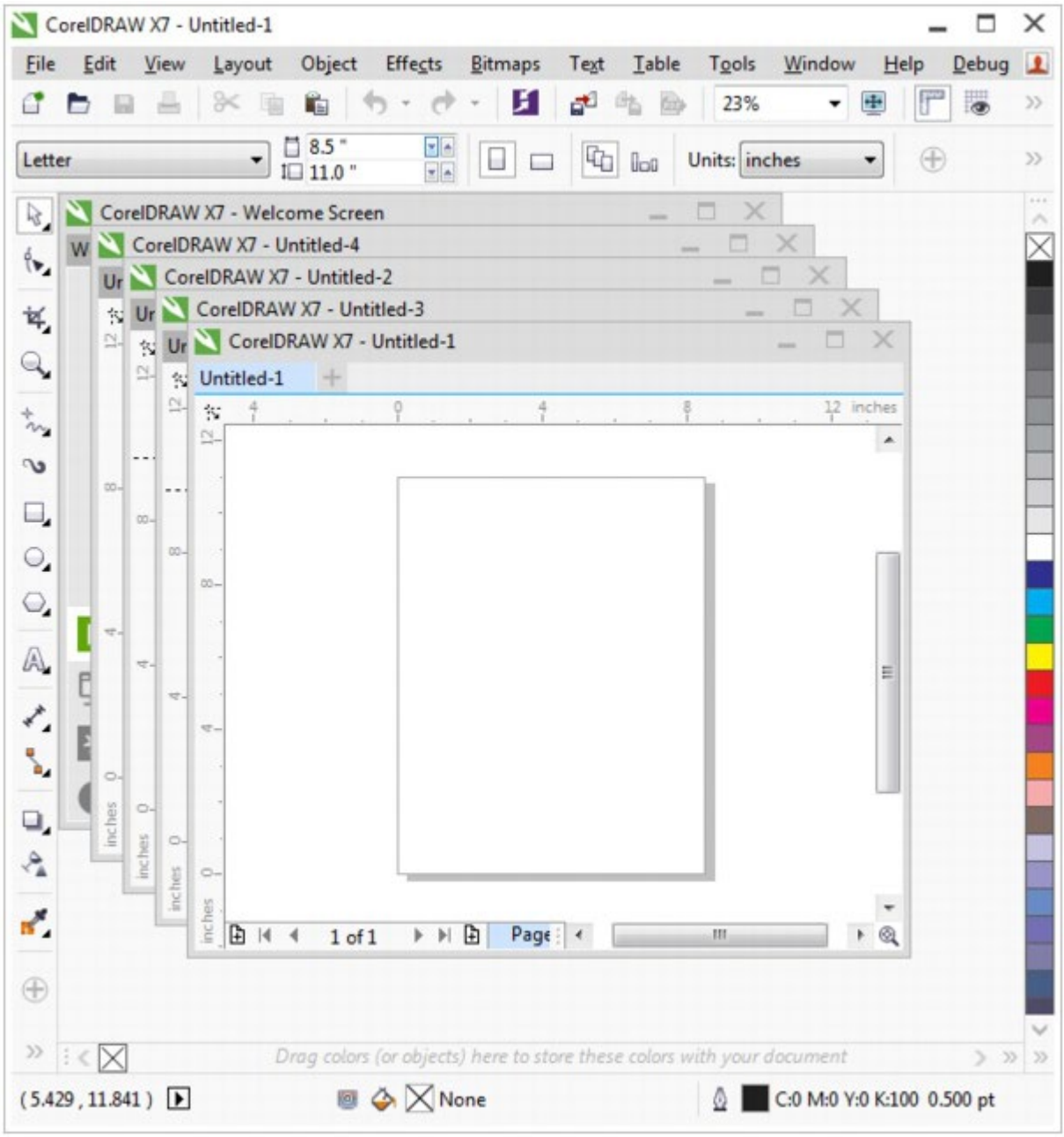
The open documents appear tabbed.

Copyright 2014 Corel Corporation. All rights reserved.

Cascading views

The following VBA example cascades all the views by extracting each view into its own floating window and arranging the views on top of the main frame window.

```
Sub CascadeViews()
Dim vh As ViewHost, rc As ScreenRect, v As ViewWindow, fw As FrameWindow
Set rc = FrameWork.MainFrameWindow.RootDockHost.Position.GetCopy
rc.Resize rc.Width * 0.75, rc.Height * 0.75
For Each vh In FrameWork.MainFrameWindow.ViewHosts
For Each v In vh.views
Set fw = FrameWork.CreateFrameWindowForView(v)
fw.Position.CopyAssign rc
fw.Position.CopyAssign rc
rc.Move rc.Left + 20, rc.Top + 20
Next v
Next vh
End Sub
```



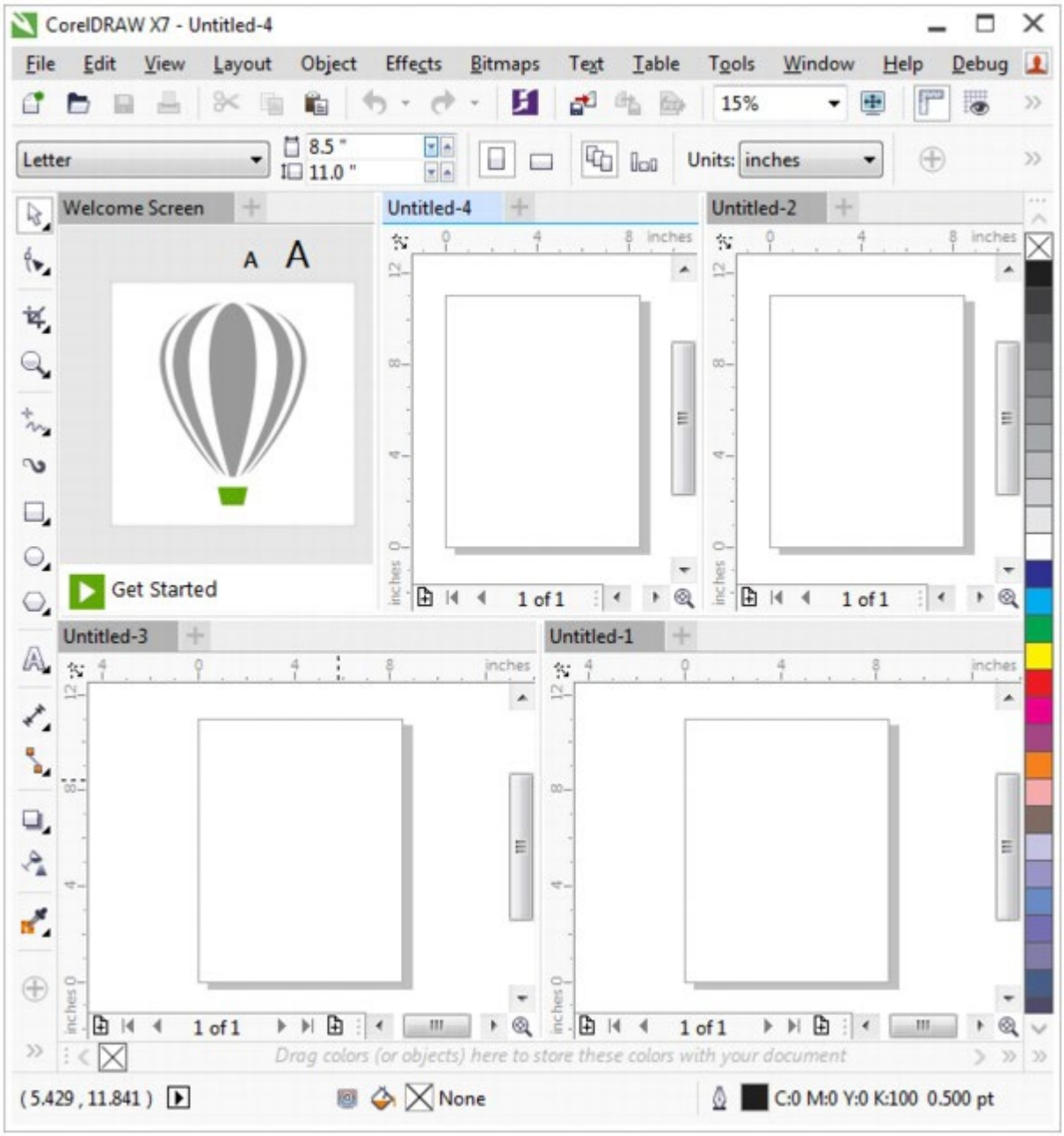
Cascading views

Copyright 2014 Corel Corporation. All rights reserved.

Tiling views

The following VBA macro tiles all the views inside the main frame window by using a grid.

```
Sub TileViews()
 Dim views As New Collection
 Dim fw As FrameWindow
 Dim vh As ViewHost, vw As ViewWindow, dh As DockHost
 Dim rows As Long, cols As Long
 Dim x As Long, y As Long, i As Long
 Dim r As DockItem, c As DockItem
  For Each fw In FrameWork.FrameWindows
   For Each vh In fw.ViewHosts
    For Each vw In vh.views
        views.Add vw
      Next vw
    Next vh
  Next fw
    If views.Count = 0 Then Exit Sub
    rows = CLng(Sqr(views.Count))
  cols = (views.Count + rows - 1) \ rows
    Set fw = FrameWork.MainFrameWindow
    For y = 1 To rows
    i = (y - 1) * cols + 1
    If dh Is Nothing Then
      If fw.RootDockHost.Orientation = cuiDockHostVertical Then
        Set vh = fw.RootDockHost.InsertView(views(i), y)
        Set dh = vh.DockHost
      Else
        Set vh = fw.RootDockHost.InsertView(views(i), 0, cuiDockOperationSplitTopLeft)
        Set dh = vh.DockHost
      End If
    Else
      Set vh = dh.InsertView(views(i), y)
    End If
    For x = 2 To cols
      i = i + 1
      If i <= views.Count Then
        If dh.Children(y).Type = cuiDockItemDockHost Then
          dh.Children(y).DockHost.InsertView views(i), x
        Else
          dh.InsertView views(i), y, cuiDockOperationSplitBottomRight
        End If
      End If
    Next x
  Next y
  For Each r In fw.RootDockHost.Children
    r.RelativeSize = fw.RootDockHost.Position.Height / rows
    If r.Type = cuiDockItemDockHost Then
      For Each c In r.DockHost.Children
        c.RelativeSize = fw.RootDockHost.Position.Width / cols
      Next c
    End If
  Next r
End Sub
```



Tiled views

Copyright 2014 Corel Corporation. All rights reserved.

Working with import filters and export filters

As previously discussed, methods are available for importing files (see Importing files into layers) and exporting files (see Exporting files from documents).

- These file-import and file-export methods can also be used for performing batch conversions or modifying file repositories.

The wide selection of supported file formats is due to the vast number of filters that are available to the application. Each filter lets you work with the files from another graphics application.

In this topic

To learn more about working with filters, see the following subtopics:

- Working with import filters
- Working with export filters

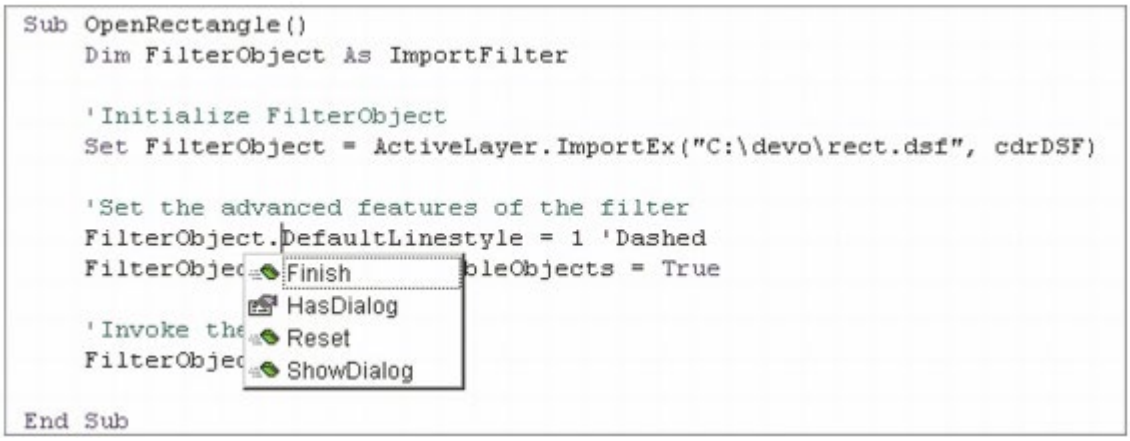
Copyright 2014 Corel Corporation. All rights reserved.

Working with import filters

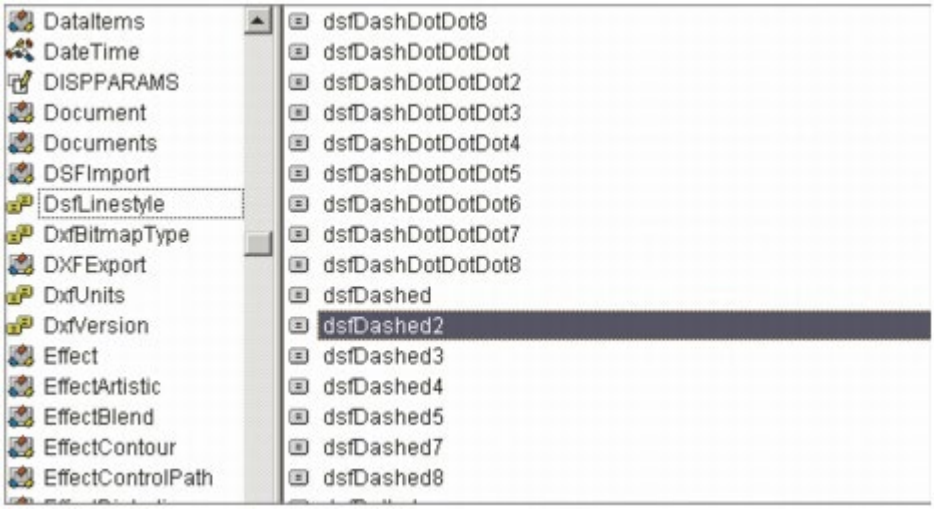
To ensure the portability of a file-import script, you must use the default **ImportFilter** object (rather than the filter-specific object **DSFImport**), as in the following VBA example:

```
Sub OpenRectangle()
Dim FilterObject As ImportFilter
'Initialize FilterObject
Set FilterObject = ActiveLayer.ImportEx("C:\devo\rect.dsf", _
cdrDSF)
'Set the advanced features of the filter
FilterObject.DefaultLinestyle = 1 'Dashed
FilterObject.DeleteInvisibleObjects = True
'Invoke the filter
FilterObject.Finish
End Sub
```

For best results, use the filter-specific object **DSFImport** only to learn the specific interfaces that are supported by a filter. For example, the following screenshot demonstrates that the **ImportFilter** object exposes only generic interfaces in Microsoft® IntelliSense® because the **ImportFilter** interface is generic (and not filter-specific). The **ImportFilter** object does not contain the **DefaultLinestyle** and **DeleteInvisibleObjects** properties; however, you can still set these properties in the **ImportFilter** interface if they are supported by the specified import filter.



As previously discussed, using the **ImportFilter** object (rather than the filter-specific object **DSFImport**) ensures that a file-import script can be used on any other workstation running the same version of the application. To reference the properties, methods, and enumerations for a specific filter, locate that filter in the Object Browser. For example, the following screenshot demonstrates that the line-style **dsfDashed2** can be specified by assigning a value of **7** to the **DefaultLinestyle** property.



To access the object model for a filter, click **Tools** } **References** from within the Macro Editor. In the **References** dialog box that appears, click **Browse**, and navigate to the **Filters** folder of the installed software. Select the dynamic-link library (DLL) file for the desired filter, and then click **OK**. When the **References** dialog box reappears, enable the checkbox that corresponds to the desired filter, and then click **OK**. You can now access the object model for the filter, as in the following VBA example:

```
Sub OpenRectangleDSF()
Dim FilterObject As DSFImport
Dim Style As DsfLinestyle
'Initialize FilterObject
Set FilterObject = ActiveLayer.ImportEx("C:\devo\rect.dsf", _
cdrDSF)
'Set the advanced features of the filter
Style = dsfDashed
FilterObject.DefaultLinestyle = Style
FilterObject.DeleteInvisibleObjects = True
'Invoke the filter
FilterObject.Finish
End Sub
```

```
Sub OpenRectangleDSF()
    Dim FilterObject As DSFImport
    Dim Style As DsfLinestyle

    'Initialize FilterObject
    Set FilterObject = ActiveLayer.ImportEx("C:\devo\rect.dsf", cdrDSF)

    'Set the advanced features of the filter
    Style = dsfDashed
    FilterObject.DefaultLinestyle = Style
    FilterObjec                                  s = True
                DefaultLinestyle
                DeleteInvisibleObjects
    'Invoke the Finish
    FilterObjec HasDialog
                Reset
End Sub         ShowDialog
```

Working with an import filter is made much easier by having the script access the object model for that filter; however, as discussed, this technique reduces the portability of the script. When used at another workstation, the script must first be updated with the correct location of the DLL file for the filter.

Copyright 2014 Corel Corporation. All rights reserved.

Working with layers

Layers are invisible planes that let you organize the objects on a page. You can group related objects into layers, and you can change the vertical order (or “stacking order”) of those layers to change the appearance of the page. Master layers apply to all pages in a document, while local layers apply to a single page.

Each layer, or **Layer** object, is a member of the **Page.Layers** collection for the page on which it appears. The layers in a **Page.Layers** collection appear in the order in which they appear on that page — the first layer is the one at the top of the “stack,” and the last layer is the one at the bottom. If layers are added, reordered, or deleted, the affected **Page.Layers** collection is immediately updated to reflect the new layer order of that page.

CorelDRAW provides a number of properties, methods, and events for working with layers, the most useful of which are listed in the following table.

Class	Member	Description
Document	**ActiveLayer** property	Provides direct access to the active layer for a document For more information, see Activating layers.
Document	**LayerActivate** event	Is triggered when a layer is activated For more information, see Activating layers.
Document	**LayerChange** event	Is triggered when a layer is deactivated For more information, see Activating layers.
Document	**LayerCreate** event	Is triggered when a layer is created For more information, see Creating layers.
Document	**LayerDelete** event	Is triggered when a layer is deleted For more information, see Deleting layers.
Layer	**Activate** method	Activates a layer For more information, see Activating layers.
Layer	**Delete** method	Deletes a layer For more information, see Deleting layers.
Layer	**Editable** property	Controls whether a layer is editable For information, see Locking and hiding layers.
Layer	**Import** method or **ImportEx** method	Imports a file into a layer For information, see Importing files into layers.
Layer	**MoveAbove** method or **MoveBelow** method	Moves a layer For information, see Reordering layers.
Layer	**Name** property	Specifies the name of a layer For information, see Renaming layers.
Layer	**Visible** property	Controls whether the contents of a layer are visible For information, see Locking and hiding layers.
Page	**ActiveLayer** property	Provides direct access to the active layer for a page For more information, see Activating layers.
Page	**CreateLayer** method	Inserts a new layer at the top of the list of non-master layers For more information, see Creating layers.
Page	**Layers** property	Contains the collection of layers for a page

For more information, see Activating layers.

- For detailed information on any property, method, or event, see the Object Model Reference.

In this topic

For more information on layer-related activities, see the following subtopics:

- Creating layers
- Activating layers
- Locking and hiding layers
- Reordering layers
- Renaming layers
- Importing files into layers
- Deleting layers

Copyright 2014 Corel Corporation. All rights reserved.

Working with pages

Each page, or **Page** object, is a member of the **Document.Pages** collection for the document in which it appears. The pages in a **Document.Pages** collection appear in the order in which they appear in that document — for example, the fifth page in the active document is `ActiveDocument.Pages.Item(5)`. If pages are added, reordered, or deleted, the affected **Pages** collection is immediately updated to reflect the new page order of that document.

CorelDRAW provides a number of properties, methods, and events for working with pages, the most useful of which are listed in the following table.

Class	Member	Description
Application	**ActivePage** property	Provides direct access to the active page of the active document For more information, see Activating pages.
Application	**PageSizes** property	Contains the collection of defined page sizes for the application For more information, see Using defined page sizes.
Document	**ActivePage** property	Provides direct access to the active page of a document For more information, see Activating pages.
Document	**AddPages** method or **AddPagesEx** method	Adds blank pages to the end of a document For more information, see Creating pages.
Document	**InsertPages** method or **InsertPagesEx** method	Inserts pages at the specified location in a document For more information, see Creating pages.
Document	**MasterPage** property	Specifies the default page size For more information, see Specifying the default page size.
Document	**PageActivate** event	Is triggered when a page is activated For more information, see Activating pages.
Document	**PageChange** event	Is triggered when a page is deactivated For more information, see Activating pages.
Document	**PageCreate** event	Is triggered when a page is created For more information, see Creating pages.
Document	**PageDelete** event	Is triggered when a page is deleted For more information, see Deleting pages.
Document	**Pages** property	Contains the collection of pages for a document For more information, see Activating pages.
Page	**Activate** method	Activates a page For more information, see Activating pages.
Page	**Delete** method	Deletes a page For more information, see Deleting pages.
Page	**MoveTo** method	Moves a page to the specified location in a document For more information, see Reordering pages.
Page	**SetSize** method	Sets the size of a page For more information, see Specifying the size and orientation of pages.

PageSize	**BuiltIn** property	Returns **True** if a page size is built-in (rather than user-defined) For more information, see Using defined page sizes.
PageSize	**Delete** method	Deletes a user-defined page size For more information, see Using defined page sizes.
PageSize	**Height** property	Specifies the height of a defined page size For more information, see Using defined page sizes.
PageSize	**Name** property	Specifies the name of a defined page size For more information, see Using defined page sizes.
PageSize	**Width** property	Specifies the width of a defined page size For more information, see Using defined page sizes.

- For detailed information on any property, method, or event, see the Object Model Reference.

In this topic

For more information on page-related activities, see the following subtopics:

- Creating pages
- Activating pages
- Reordering pages
- Sizing pages
- Modifying pages
- Deleting pages

Copyright 2014 Corel Corporation. All rights reserved.

Working with shapes

Every document is made up of shapes, or **Shape** objects, which are created by using the drawing tools. Any changes that are made to the properties of a shape — such as by moving the shape, changing its size, or giving it a new fill — are immediately visible to the object model.

The shapes on a document page are stored on layers. Each shape is a member of the **Layer.Shapes** collection for the layer on which it appears. The shapes in a **Layer.Shapes** collection appear in the order in which they appear on that layer — the first shape is the one at the top of the “stack,” and the last shape is the one at the bottom. If shapes are added, reordered, or deleted, the affected **Layer.Shapes** collection is immediately updated to reflect the new shape order of that layer.

- In addition, each document page has a **Shapes** collection, which contains all **Layer.Shapes** collections for that page. The first shape in a **Page.Shapes** collection is the one at the very top of that page, and the last shape is the one at the very bottom.

CorelDRAW provides a number of properties, methods, and events for working with shapes, the most useful of which are listed in the following table.

Class	Member	Description
AddinHook	**ShapeCreated** event	Is triggered when a shape is created For more information, see Creating shapes.
Application	**CreateCurve** method	Creates a line or a curve “in memory” For more information, see Creating lines and curves.
Application	**SelectionChange** event	Is triggered when a selection is deactivated For more information, see Deselecting shapes.
Application	**SymbolLibraries** property	Contains the collection of all external symbol libraries for the application For more information, see Creating symbols.
Color	**CopyAssign** method	Copies a color from one shape fill or shape outline to another For more information, see Working with color.
Color	**Type** method	Specifies the color model for a shape color For more information, see Working with color.
Curve	**CreateSubPath** method	Adds a subpath to a line or a curve For more information, see Creating lines and curves.
Document	**ClearSelection** method	Deselects all objects in a document For more information, see Deselecting shapes.
Document	**Selection** method	Returns, as a single **Shape** object, all selected objects in a document For more information, see Accessing selections directly.
Document	**SelectionChange** event	Is triggered when a selection is deactivated For more information, see Deselecting shapes.
Document	**SelectionRange** property	Returns, as a **ShapeRange** object, all selected objects in a document For more information, see Accessing copies of selections.
Document	**ShapeChange** event	Is triggered when a shape is deselected For more information, see Deselecting shapes.
Document	**ShapeCreate** event	Is triggered when a shape is created For more information, see Creating shapes.
Document	**ShapeDelete** event	Is triggered when a shape is deleted

		For more information, see Deleting shapes.
Document	**ShapeDistort** event	Is triggered when a shape is distorted For more information, see Applying distortions.
Document	**ShapeMove** event	Is triggered when a shape is positioned For more information, see Positioning shapes.
Document	**ShapeTransform** event	Is triggered when a shape is transformed For more information, see Transforming shapes.
Document	**SymbolLibrary** property	Returns the internal symbol library for a document For more information, see Creating symbols.
Fill	**ApplyFountainFill** method	Applies a fountain fill to a shape For more information, see Applying fountain fills.
Fill	**ApplyHatchFill** method	Applies a hatch fill to a shape For more information, see Applying hatch fills.
Fill	**ApplyPatternFill** method	Applies a pattern fill to a shape For more information, see Applying pattern fills.
Fill	**ApplyPostScriptFill** method	Applies a PostScript fill to a shape For more information, see Applying PostScript fills.
Fill	**ApplyTextureFill** method	Applies a texture fill to a shape For more information, see Applying texture fills.
Fill	**ApplyUniformFill** method	Applies a uniform fill to a shape For more information, see Applying uniform fills.
Fill	**Fountain** property	Specifies the fountain-fill properties for a shape For more information, see Applying fountain fills.
Fill	**Hatch** property	Specifies the hatch-fill properties for a shape For more information, see Applying hatch fills.
Fill	**Pattern** property	Specifies the pattern-fill properties for a shape For more information, see Applying pattern fills.
Fill	**PostScriptFill** property	Specifies the PostScript-fill properties for a shape For more information, see Applying PostScript fills.
Fill	**Texture** property	Specifies the texture-fill properties for a shape For more information, see Applying texture fills.
Fill	**Type** property	Specifies the type of fill that is applied to a shape For more information, see Coloring shapes.
Fill	**UniformColor** property	Specifies the uniform-fill properties for a shape For more information, see Applying uniform fills.
FountainColor	**Move** method	Moves a color in the fountain fill for a shape For more information, see Applying fountain fills.
FountainColors	**Add** method	Adds a color to the fountain fill for a shape For more information, see Applying fountain fills.

FountainColors	**Count** property	Counts the number of colors between the start color and end color in the fountain fill for a shape For more information, see Applying fountain fills.
GlobalMacroStorage	**SelectionChange** event	Is triggered when a selection is deactivated For more information, see Deselecting shapes.
Layer	**CreateArtisticText** method or **CreateArtisticTextWide** method	Creates an artistic-text object on the specified layer For more information, see Creating text objects.
Layer	**CreateCurve** method	Creates, on the specified layer, a line or a curve that is created “in memory” by using the **Application.CreateCurve** method For more information, see Creating lines and curves.
Layer	**CreateCurveSegment** method or **CreateCurveSegment2** method	Creates a basic curve on the specified layer For more information, see Creating lines and curves.
Layer	**CreateEllipse** method, **CreateEllipse2** method, or **CreateEllipseRect** method	Creates an ellipse on the specified layer For more information, see Creating ellipses.
Layer	**CreateLineSegment** method	Creates a basic line on the specified layer For more information, see Creating lines and curves.
Layer	**CreateParagraphText** method or **CreateParagraphTextWide** method	Creates an paragraph-text object on the specified layer For more information, see Creating text objects.
Layer	**CreateRectangle** method, **CreateRectangle2** method, or **CreateRectangleRect** method	Creates a rectangle on the specified layer For more information, see Creating rectangles.
Layer	**Shapes** property	Contains the collection of shapes for a layer For more information, see Selecting shapes.
Outline	**Color** property	Specifies the color of the outline for a shape For more information, see Applying outlines.
Outline	**Style** property	Specifies the dash settings (that is, style properties) for the outline of a shape For more information, see Applying outlines.
Outline	**Type** property	Specifies whether an outline is applied to a shape For more information, see Applying outlines.
Outline	**Width** property	Specifies, in document units, the width of the outline for a shape For more information, see Applying outlines.
Segment	**AddNodeAt** method	Adds a node to a line segment or a curve segment For more information, see Creating lines and curves.
Shape	**CreateBlend** method	Applies a blend effect to a shape

		For more information, see Applying blends.
Shape	**CreateContour** method	Applies a contour effect to a shape For more information, see Applying contours.
Shape	**CreateCustomDistortion** method	Applies a customized distortion effect to a shape For more information, see Applying distortions.
Shape	**CreateCustomEffect** method	Applies a customized effect to a shape For more information, see Applying customized effects.
Shape	**CreateDropShadow** method	Applies a drop-shadow effect to a shape For more information, see Applying drop shadows.
Shape	**CreateEnvelope** method, **CreateEnvelopeFromCurve** method, or **CreateEnvelopeFromShape** method	Applies an envelope effect to a shape For more information, see Applying envelopes.
Shape	**CreateExtrude** method	Applies an extrusion effect to a shape For more information, see Applying extrusions.
Shape	**CreateLens** method	Applies a lens effect to a shape For more information, see Applying lenses.
Shape	**CreatePerspective** method	Applies a perspective effect to a shape For more information, see Applying perspective.
Shape	**CreatePushPullDistortion** method	Applies a Push-and-pull distortion effect to a shape For more information, see Applying distortions.
Shape	**CreateSelection** method	Creates a selection from a single shape For more information, see Selecting shapes.
Shape	**CreateTwisterDistortion** method	Applies a Twister distortion effect to a shape For more information, see Applying distortions.
Shape	**CreateZipperDistortion** method	Applies a Zipper distortion effect to a shape For more information, see Applying distortions.
Shape	**Duplicate** method	Duplicates a shape For more information, see Duplicating shapes.
Shape	**Evaluate** method	Returns the result of a given expression that evaluates the properties of the current shape For more information, see Searching for shapes.
Shape	**GetBoundingBox** method	Returns the size of a shape based on the size of its bounding box For more information, see Sizing shapes.
Shape	**GetPosition** method	Returns the horizontal and vertical position of a shape For more information, see Positioning shapes.
Shape	**GetSize** method	Returns the size of a shape For more information, see Sizing shapes.
Shape	**PlaceTextInside** method	Places the selected text inside the specified shape For more information, see Creating text objects.

Shape	**PositionX** property	Returns, or sets, the horizontal position of a shape For more information, see Positioning shapes.
Shape	**PositionY** property	Returns, or sets, the vertical position of a shape For more information, see Positioning shapes.
Shape	**Rotate** method or **RotateEx** method	Rotates a shape For more information, see Rotating shapes.
Shape	**Selected** method	Specifies whether a shape is selected For more information, see Selecting shapes.
Shape	**SetBoundingBox** method	Sets the size of a shape based on the size of its bounding box For more information, see Sizing shapes.
Shape	**SetPosition** method or **SetPositionEx** method	Sets the position of a shape For more information, see Positioning shapes.
Shape	**SetSize** method or **SetSizeEx** method	Sets the size of a shape For more information, see Sizing shapes.
Shape	**SizeHeight** property	Returns, or sets, the height of a shape For more information, see Sizing shapes.
Shape	**SizeWidth** property	Returns, or sets, the width of a shape For more information, see Sizing shapes.
Shape	**Skew** method or **SkewEx** method	Skews a shape For more information, see Skewing shapes.
Shape	**Stretch** method or **StretchEx** method	Stretches (or scales) a shape For more information, see Stretching shapes.
Shape	**Type** property	Returns the type for a shape For more information, see Determining shape type.
ShapeRange	**CreateSelection** method	Creates a selection from a range of shapes For more information, see Selecting shapes.
ShapeRange	**Duplicate** method	Duplicates a range of shapes For more information, see Duplicating shapes.
Shapes	**All** method	Returns all shapes from the specified collection of shapes For more information, see Selecting shapes.
Shapes	**FindShape** method	Returns a single shape that has the specified properties For more information, see Searching for shapes.
Shapes	**FindShapes** method	Returns, as a shape range, all shapes that have the specified properties For more information, see Searching for shapes.
SubPath	**AppendCurveSegment** method or **AppendCurveSegment2**	Adds a curve-type segment to a subpath For more information, see Creating lines and curves.

	method	
SubPath	**AppendLineSegment** method	Adds a line-type segment to a subpath For more information, see Creating lines and curves.
Symbol	**Definition** property	Returns the definition of a symbol For more information, see Creating symbols.
SymbolDefinition	**NestedSymbols** property	Contains the collection of all nested symbols for a symbol definition For more information, see Creating symbols.
SymbolLibrary	**Symbols** property	Contains the collection of all symbol definitions for a symbol library For more information, see Creating symbols.
Text	**FitTextToPath** method	Attaches the specified artistic text to the outline of a shape For more information, see Creating text objects.
Text	**Frames** property	Represents a series of text frames (or **TextFrame** objects), each of which has its own text range (or **TextRange** object) For more information, see Creating text objects.
Text	**Story** property	Represents a text range (or **TextRange** object) that includes all text in a series of text frames (or **TextFrame** objects) For more information, see Creating text objects.

- For detailed information on any property, method, or event, see the Object Model Reference.

In this topic

For more information on shape-related activities, see the following subtopics:

- Creating shapes
- Determining shape type
- Selecting shapes
- Duplicating shapes
- Transforming shapes
- Coloring shapes
- Applying effects to shapes
- Searching for shapes
- Deleting shapes

Copyright 2014 Corel Corporation. All rights reserved.

Writing macros

You can manually code a macro by writing it in the Macro Editor or the VSTA Editor. (Alternatively, in CorelDRAW, you can create a VBA macro by recording it. For information, see Recording macros.) Macros that are developed in the Macro Editor or the VSTA Editor can take advantage of full programming control, including conditional execution, looping, and branching. In effect, you can write macros that are programs in their own right.

- In this documentation, all macro code is referred to as a macro. In some contexts, however, a macro is just those parts of the code that can be run by CorelDRAW

To write a macro, you must first add it to a code module for the desired macro project. You can edit, rename, or even delete macros.

For details on writing macros, see the following procedures:

- To add a macro to a macro project
- To edit a VBA macro
- To delete a VBA macro

Copyright 2014 Corel Corporation. All rights reserved.